

TRABAJO DE GRADO

**SIMULACIÓN DE LA AUTOMATIZACIÓN DE UN ROBOT PARA REMOLCAR
AVIONETAS UTILIZANDO EL SOFTWARE WEBOTS**

Autor: Juan Rosas		
Firma: 		
Tutor: Diego Quiroga	Co-Tutor:	Fecha de entrega al comité:
Firma:	Firma:	18/02/2025

Código del Anteproyecto							
P	1	M	2	0	0	0	4

DECLARACIÓN DE AUTORÍA

Yo, Juan Mathias Rosas Chaparro, con documento de identidad 5.135.136-3, estudiante de Ingeniería Mecatrónica de UTEC ITR SO, en relación al Proyecto Final de grado titulado "SIMULACIÓN DE LA AUTOMATIZACIÓN DE UN ROBOT PARA REMOLCAR AVIONETAS UTILIZANDO EL SOFTWARE WEBOTS", código PIM20004 para su defensa y evaluación, declaro que asumo la originalidad de este trabajo siendo elaborada por mí, y todas las fuentes de información que han sido citadas o utilizadas como argumento del trabajo que están referenciados en él, fueron incluidas en las Referencias Bibliográficas.

Fray Bentos, 18 de febrero de 2025

X

Juan Mathias Rosas Chaparro

AGRADECIMIENTOS

Quiero expresar mi más profundo agradecimiento a todos aquellos que me han acompañado en esta travesía tan especial. En primer lugar, a mi familia: a mi madre, Natalia, a mi padre, Gianni, y a mis hermanas, Florencia y Julieta, por su amor incondicional y su apoyo inquebrantable.

También deseo agradecer al capitán de navío, Sergio, su primer oficial, Gianni y su fiel grumete, Joaquín, por navegar conmigo entre los altibajos de este viaje. A los jugadores del Sportivo Rustico, Laje y Rodrigo, por estar en todos los partidos.

Estoy muy agradecido con mis compañeros de clase, con quienes compartí aprendizajes, desafíos y crecimiento a lo largo de toda la carrera. En especial, a Uriel y Nicolás, grandes amigos que estuvieron a mi lado en este camino y cuya amistad valoro enormemente.

Del mismo modo, expreso mi gratitud a los docentes de la carrera, en particular a Robert, Yamile y, especialmente, a Diego, quien, como tutor de este proyecto, brindó su guía y apoyo fundamentales para su desarrollo.

Finalmente, mi gratitud va para Sol y Kat, cuya paciencia y aliento fueron fundamentales para culminar este proyecto.

A Gordon y Atenea, que siempre acompañaron en las noche de estudio.

Y a Coco, que con él empezó todo, muchas gracias.

RESUMEN

Este trabajo presenta el diseño y simulación de un robot autónomo para el remolque de avionetas utilizando el software Webots como plataforma de validación. Se parte del análisis de la problemática en el hangar de Álabé Servicios Aéreos SRL, donde se identifica la necesidad de automatizar el traslado de aeronaves para reducir la carga física sobre los operarios.

El estudio se estructura en cuatro etapas fundamentales: el diseño conceptual del robot, el análisis cinemático y de navegación, la implementación de la simulación en Webots y la verificación del desempeño del sistema.

Los resultados de la simulación confirman la viabilidad del sistema propuesto, mostrando una navegación eficiente en escenarios controlados y la correcta ejecución de maniobras de remolque. Este trabajo sienta las bases para el desarrollo de un prototipo físico y abre la posibilidad de aplicar tecnologías de automatización en el ámbito aeronáutico, con potencial impacto en la reducción de costos operativos y mejora de la seguridad laboral.

ABSTRACT

This study presents the design and simulation of an autonomous aircraft towing robot using the Webots simulation software as a validation platform. The project originates from the need identified at Álabé Servicios Aéreos SRL's hangar, where manual aircraft towing arises a physically demanding task for operators.

The research is structured into four key phases: conceptual design of the towing robot, kinematic and navigation model analysis, simulation implementation in Webots, and performance verification of the system.

The simulation results confirm the feasibility of the proposed system, demonstrating efficient navigation in controlled scenarios and the successful execution of towing maneuvers. This study lays the groundwork for the development of a physical prototype and highlights the potential of automation technologies in the aeronautical sector, contributing to cost reduction and improved workplace safety.

ÍNDICE

DECLARACIÓN DE AUTORÍA	2
AGRADECIMIENTOS	3
RESUMEN	4
ABSTRACT	5
ÍNDICE	6
ÍNDICE DE FIGURAS	8
ÍNDICE DE TABLAS	10
CAPÍTULO 1. EL PROBLEMA.....	11
1.1. PLANTEAMIENTO DEL PROBLEMA	11
1.2. OBJETIVOS GENERAL Y ESPECÍFICOS.....	13
1.2.1. OBJETIVO GENERAL	13
1.2.2. OBJETIVOS ESPECÍFICOS	14
1.3. ALCANCE Y LIMITACIONES	14
CAPÍTULO 2. MARCO TEÓRICO	15
2.1. ANTECEDENTES	15
2.2. BASES TEÓRICAS	17
2.2.1. FUNDAMENTOS DE ROBÓTICA	17
2.2.2. ACTUADORES	19
2.2.3. SENSORES	21
2.2.4. EJECUCIÓN DE TAREAS	22
2.2.5. MAPEO EN ROBOTS MÓVILES	23
2.2.6. NAVEGACIÓN DE ROBOTS MÓVILES	25
2.2.7. ALGORITMOS DE PLANIFICACIÓN DE TRAYECTORIAS	27
2.2.8. SIMULADOR WEBOTS	39
CAPÍTULO 3. MARCO METODOLÓGICO	41
3.1. TIPO Y DISEÑO DE INVESTIGACIÓN	41
3.2. POBLACIÓN Y MUESTRA.....	41
3.3. TÉCNICAS DE RECOLECCIÓN DE DATOS	41
3.4. TÉCNICAS DE ANÁLISIS DE DATOS	42
3.5. INSTRUMENTOS DE TRABAJO	42
3.6. DESARROLLO SISTEMÁTICO DE OBJETIVOS	42
3.6.1. ESTABLECIMIENTO DEL DISEÑO CONCEPTUAL DEL ROBOT REMOLCADOR PARA PREPARAR LOS PARÁMETROS DE LA SIMULACIÓN	42
3.6.1.1. IDENTIFICACIÓN DE REQUISITOS	42
3.6.2. ANÁLISIS DEL MODELO CINEMÁTICO Y DE NAVEGACIÓN DEL ROBOT REMOLCADOR PARA AVIONETAS	44
3.6.2.1. ESTUDIO DEL MODELO CINEMÁTICO DEL ROBOT REMOLCADOR	45
3.6.2.2. ESTUDIO DE NAVEGACIÓN DEL ROBOT REMOLCADOR	47
3.6.3. IMPLEMENTACIÓN DE LA SIMULACIÓN EN WEBOTS PARA EVALUAR UNA ARQUITECTURA DEL SISTEMA DE NAVEGACIÓN	52
3.6.3.1. CREACIÓN DEL MUNDO EN WEBOTS.....	52
3.6.3.2. INTEGRACIÓN DE LOS SENSORES AL ROBOT EN WEBOTS	53
3.6.3.3. CONFIGURACIÓN DE PARÁMETROS DE SIMULACIÓN	54
3.6.3.4. IMPLEMENTACIÓN DE ARQUITECTURA DEL SISTEMA.....	55
3.6.4. VERIFICACIÓN DE LA SIMULACIÓN EN EL SOFTWARE WEBOTS.....	58
3.6.4.1. CONTROL DE MOTORES SIN CARGA	59
3.6.4.2. CONTROL DE MOTORES CON CARGA	60
3.6.4.3. VALIDACIÓN DE MAPEO	61
3.6.4.4. VALIDACIÓN DE PLANIFICACIÓN DE TRAYECTORIAS Y TRASLADO DE AVIONETA CON ALGORITMO	

D* <i>LITE</i>	62
CAPÍTULO 4. RESULTADOS Y ANÁLISIS	64
4.1. RESULTADOS DEL ESTABLECIMIENTO DEL DISEÑO CONCEPTUAL DEL ROBOT REMOLCADOR PARA PREPARAR LOS PARÁMETROS DE LA SIMULACIÓN	64
4.1.1. IDENTIFICACIÓN DE REQUISITOS	64
4.2. RESULTADOS DEL ANÁLISIS DEL MODELO CINEMÁTICO Y DE NAVEGACIÓN DEL ROBOT REMOLCADOR PARA AVIONETAS.....	67
4.2.1. ESTUDIO DEL MODELO CINEMÁTICO	67
4.2.2. ESTUDIO DE NAVEGACIÓN DEL ROBOT REMOLCADOR	71
4.3. RESULTADOS DE LA IMPLEMENTACIÓN DE LA SIMULACIÓN EN WEBOTS PARA EVALUAR UNA ARQUITECTURA DEL SISTEMA DE NAVEGACIÓN	72
4.3.1. CREACIÓN DEL MUNDO EN WEBOTS.....	72
4.3.2. INTEGRACIÓN DE LOS SENSORES AL ROBOT EN WEBOTS	74
4.3.3. CONFIGURACIÓN DE PARÁMETROS DE SIMULACIÓN.....	75
4.3.4. IMPLEMENTACIÓN DE ARQUITECTURA DEL SISTEMA.....	76
4.4. RESULTADOS DE LA VERIFICACIÓN DE LA SIMULACIÓN EN EL SOFTWARE WEBOTS	80
4.4.1. CONTROL DE MOTORES SIN CARGA	80
4.4.2. CONTROL DE MOTORES CON CARGA	83
4.4.3. VALIDACIÓN DE MAPEO DEL ÁREA DE TRABAJO	84
4.4.4. VALIDACIÓN DE PLANIFICACIÓN DE TRAYECTORIAS Y TRASLADO DE AVIONETA CON ALGORITMO D* <i>LITE</i>	85
CONCLUSIONES.....	91
OPORTUNIDADES DE MEJORA.....	93
BIBLIOGRAFÍA	94
ANEXO A. RESPUESTAS DE CUESTIONARIO	97
ANEXO B. PLANOS DE DISEÑO Y ENSAMBLE DE ROBOT REMOLCADOR ADAPTADOS DE FREIRE Y RUBIO (2016)	100
ANEXO C. SCRIPTS DE CÓDIGO DE SOLUCIÓN IMPLEMENTADA	108
ANEXO D. SCRIPTS DE CÓDIGO DE LOS ENSAYOS DE LAS SECCIONES 3.6.4.1 Y 3.6.4.2	141

ÍNDICE DE FIGURAS

FIGURA 1.1. REMOLCADOR TELEDIRIGIDO ESTÁNDAR DE LA EMPRESA TOWBOTS (S.F.)	12
FIGURA 2.1. MODELO 3D DISEÑADO POR FREIRE Y RUBIO (2016)	16
FIGURA 2.2. ROVERT, DESARROLLO DE BERNHEIM, COSTA Y DE LUCA (2019)	16
FIGURA 2.3. APLICACIONES: A. REMOLCADOR, B. CARGADOR DE UNIDADES, C. MONTACARGAS MÓVIL. (MIHELJ ET AL., 2019)	18
FIGURA 2.4. DISEÑO DE RUEDAS: A. RUEDA FIJA STANDARD, B. RUEDA ORIENTABLE STANDARD, C. RUEDA GIRATORIA, D. RUEDA ESFÉRICA, E. RUEDA MECANUM. (MIHELJ ET AL., 2019)	19
FIGURA 2.5. A. TRACCIÓN DIFERENCIAL DE DOS RUEDAS, B. TRACCIÓN DIFERENCIAL DE DOS RUEDAS CON RUEDA GIRATORIA, C. TRES RUEDAS MOTORIZADAS Y DIRIGIDAS SINCRÓNICAMENTE, D. TRES RUEDAS OMNIDIRECCIONALES EN TRIÁNGULO, E. CUATRO RUEDAS CON DIRECCIÓN TIPO AUTOMÓVIL, F. DOS RUEDAS DE TRACCIÓN DIFERENCIAL Y DOS RUEDAS OMNIDIRECCIONALES, G. CUATRO RUEDAS GIRATORIAS MOTORIZADAS Y DIRIGIDAS, Y H. CUATRO RUEDAS OMNIDIRECCIONALES EN CONFIGURACIÓN RECTANGULAR. (MIHELJ ET AL., 2019)	19
FIGURA 2.6. SIMULADOR WEBOTS	39
FIGURA 3.1. DIBUJO 2D VISTA SUPERIOR Y PROYECTADA DEL ROBOT DE FREIRE Y RUBIO (2016)	45
FIGURA 3.2. DIAGRAMA CINEMÁTICO DEL ROBOT (MILEHJ, 2019)	46
FIGURA 3.3. NODO <i>WORLDINFO</i> EN SOFTWARE WEBOTS	54
FIGURA 3.4. NODO <i>PHYSICS</i> EN SOFTWARE WEBOTS	55
FIGURA 3.5. ENSAYO DE ALGORITMO D* <i>LITE</i>	63
FIGURA 4.1. GRADOS DE LIBERTAD DEL ROBOT REMOLCADOR	68
FIGURA 4.2. MOVIMIENTO DE AVANCE (IZQ) Y RETROCESO (DER)	69
FIGURA 4.3. ROTACIÓN SOBRE SU EJE SENTIDO ANTI-HORARIO (IZQ.) Y HORARIO (DER.)	70
FIGURA 4.4. AVANCE CON CURVA HACIA LA DERECHA (SUP. IZQ.), AVANCE CON CURVA HACIA LA IZQUIERDA (SUP. DER.), RETROCESO CON CURVA HACIA LA DERECHA (INF. IZQ.) Y RETROCESO CON CURVA HACIA LA IZQUIERDA (INF. DER.)	71
FIGURA 4.5. ÁREA DE TRABAJO CON HANGAR EN SIMULADOR WEBOT	73
FIGURA 4.6. AVIÓN EN SIMULADOR WEBOTS	73
FIGURA 4.6. ROBOT REMOLCADOR EN SIMULADOR WEBOTS	74
FIGURA 4.7. ACOPLAMIENTO DEL AVIÓN AL ROBOT REMOLCADOR EN SIMULADOR WEBOTS	74
FIGURA 4.8. INTEGRACIÓN DE LOS SENSORES AL ROBOT REMOLCADOR EN SOFTWARE WEBOTS	75
FIGURA 4.9. FALLO DEL SIMULADOR WEBOTS	75
FIGURA 4.10. DIAGRAMA DE ÁRBOL DE COMPORTAMIENTO DEL ROBOT	77
FIGURA 4.11. DIAGRAMA DE FLUJO DE LA NAVEGACIÓN DEL ROBOT	78
FIGURA 4.12. PLANIFICACIÓN DE TRAYECTORIAS CON ALGORITMO D* <i>LITE</i>	80
FIGURA 4.13. GRÁFICA DE VELOCIDAD DEL ROBOT EN EL TIEMPO EN EL ENSAYO DE MOTORES SIN CARGA	82

FIGURA 4.14. GRÁFICA DE TRAYECTORIA DEL ROBOT EN EL TIEMPO EN EL ENSAYO DE MOTORES SIN CARGA	83
FIGURA 4.15. GRÁFICA DE VELOCIDAD DEL ROBOT EN EL TIEMPO EN EL ENSAYO DE MOTORES CON CARGA	84
FIGURA 4.16. GRÁFICA DE TRAYECTORIA DEL ROBOT EN EL TIEMPO EN EL ENSAYO DE MOTORES CON CARGA	84
FIGURA 4.17. MAPA DE OCUPACIÓN	85
FIGURA 4.18. GRÁFICA DE COMPARACIÓN DE VELOCIDADES EN EL TIEMPO	86
FIGURA 4.19. GRÁFICA DE COMPARACIÓN DE TRAYECTORIAS.....	88
FIGURA 4.20. GRÁFICA DE COMPARACIÓN DE DISTANCIA AL DESTINO EN EL TIEMPO	89
FIGURA A.1. RESPUESTA DE LA PREGUNTA 10	97
FIGURA A.2. RESPUESTA DE LA PREGUNTA 11	98
FIGURA A.3. RESPUESTA DE LA PREGUNTA 12	98
FIGURA A.4. RESPUESTA DE LA PREGUNTA 13	98
FIGURA A.5. RESPUESTA DE LA PREGUNTA 14	98
FIGURA A.6. RESPUESTA DE LA PREGUNTA 15	99
FIGURA B.1. PLANO DE ENSAMBLE ROBOT REMOLCADOR.....	100
FIGURA B.2. PLANO DE DISEÑO DE ROBOT REMOLCADOR ENSAMBLADO	100
FIGURA B.3. PLANO DE DISEÑO DE BASE DEL ROBOT	101
FIGURA B.4. PLANO DE ENSAMBLE DE ACOPLE DE LA RUEDA AL ROBOT REMOLCADOR	101
FIGURA B.5. PLANO DE DISEÑO DE ACOPLE DE RUEDA ENSAMBLADO.....	102
FIGURA B.6. PLANO DE DISEÑO DE BASE CIRCULAR	102
FIGURA B.7. PLANO DE DISEÑO DE BARRA DELANTERA	103
FIGURA B.8. PLANO DE DISEÑO DE RODILLO.....	103
FIGURA B.9. PLANO DE DISEÑO DE BIELA	104
FIGURA B.10. PLANO DE DISEÑO DE PASADOR.....	104
FIGURA B.11. PLANO DE DISEÑO DE BARRA POSTERIOR.....	105
FIGURA B.12. PLANO DE DISEÑO DE BARRA CIRCULAR	105
FIGURA B.13. PLANO DE DISEÑO DE TRAMPA	106
FIGURA B.14. PLANO DE DISEÑO DE CHUMACERA UCP 205	106
FIGURA B.15. PLANO DE DISEÑO DE EJE	107

ÍNDICE DE TABLAS

TABLA 1. EVALUACIÓN DE SISTEMAS DE NAVEGACIÓN	47
TABLA 2. EVALUACIÓN DE SENSORES DE ORIENTACIÓN	48
TABLA 3. EVALUACIÓN DE SENSORES DE LOCALIZACIÓN	49
TABLA 4. EVALUACIÓN DE SENSORES DE DETECCIÓN DE OBSTÁCULOS	50
TABLA 5. EVALUACIÓN DE ALGORITMOS DE PLANIFICACIÓN DE TRAYECTORIAS	51
TABLA 6. REQUISITOS OPERATIVOS DE ÁREA DE TRABAJO	64
TABLA 7. REQUISITOS TÉCNICOS LEARJET35 (LEARJET 35/35A/36/36A MAINTENANCE MANUAL, 2012)	65
TABLA 8. CARACTERÍSTICAS DEL ROBOT DISEÑADO POR FREIRE Y RUBIO (2016)	65

CAPÍTULO 1. EL PROBLEMA

1.1. PLANTEAMIENTO DEL PROBLEMA

La empresa Álabé Servicios Aéreos SRL, ubicada en el hangar de mantenimiento del Aeropuerto Internacional de Melilla Ángel S. Adami, Montevideo, brinda servicios de mantenimiento a diversas avionetas de transporte privado y recreativo cuyas capacidades de tripulación varían entre 2 a 6 personas. Con el fin de mejorar su gestión usando los criterios de mejora continua, esta empresa identificó que una de las tareas más frecuentes y físicamente más demandante para sus trabajadores es la de remolcar y mover a las avionetas dentro y fuera del hangar, así como realizar sus movimientos en el interior de éste para su inspección y mantenimiento. Por ello buscan una solución robótica para la tarea mediante la incorporación de un remolcador automatizado capaz de transportar este tipo de aeronaves.

Este estudio aborda la integración de un robot colaborativo en la industria aeronáutica, destacando su potencial para revolucionar los procesos de trabajo en este sector. Un análisis de la literatura y las últimas estadísticas revela que la automatización y la robotización mantienen una trayectoria ascendente en la industria a nivel mundial. Según el informe de la *International Federation of Robotics* (IFR) del 2023, la instalación de nuevos robots industriales alcanzó un récord de 553000 unidades en 2022, lo que representa un crecimiento del 5 % respecto al año anterior, y se espera que el stock global de robots industriales supere los 3.9 millones de unidades, marcando un aumento del 12%. Este crecimiento subraya la tendencia hacia una mayor adopción de la robotización en diversas industrias (IFR, 2023). Por otro lado, *Interact Analysis* (2023) proyecta un crecimiento significativo en el mercado de robots colaborativos, particularmente en los robots móviles autónomos (*Autonomous Mobile Robots*, AMR) y vehículos guiados automatizados (*Automated Guided Vehicle*, AGV), los que se prevé que van a capturar una porción considerable del mercado. Solo en el 2023, la demanda en el sector de la logística para estos robots alcanzó los 8.010 millones de dólares (USD), lo que refleja el potencial de estos sistemas para transformar las operaciones de manufactura y logística (IA, 2023). Estas tendencias indican una clara dirección hacia la automatización avanzada y la colaboración entre humanos y robots, subrayando la importancia de adaptarse a las innovaciones tecnológicas para mantener la competitividad en la industria aeronáutica.

Otro aval importante para el desarrollo de esta investigación se puede encontrar en el

informe presentado por Morris et al., 2015, donde se expone la importancia que representaría el desarrollo de remolcadores de aviones o avionetas autónomos, además de proponer una infraestructura de control y operación con estos equipos en hangares aeronáuticos y aeropuertos, mejorando su organización logística y operativa.

Para atender la solicitud de Álabé Servicios Aéreos SRL, actualmente existen múltiples empresas internacionales que ofertan remolcadores teledirigidos como alternativa para abordar esta necesidad. Entre estas empresas se encuentra Towbots (s.f.), y uno de sus remolcadores se puede ver en la Figura 1.1. Otras opciones incluyen TowFLEXX (s.f.) y Mototok (s.f.). Estas soluciones incorporan un sistema mecánico para el acoplamiento de la rueda delantera de la nave y motores eléctricos para el desplazamiento del remolcador. En todos los casos, el remolcador es operado de manera inalámbrica por un operador, ya sea por radiofrecuencia o bluetooth.

En el ámbito académico, aunque las investigaciones específicas sobre esta temática son limitadas, se destaca la tesis de Freire Fernández y Rubio Segovia (2016), donde se presenta el diseño y construcción de un prototipo de robot remolcador teledirigido. Otros proyectos académicos se centran en explorar algoritmos de control de trayectoria y localización para vehículos autónomos, como se evidencia en los trabajos de Harapanahalli et al. (2019), Gutiérrez Suárez (2020) y Hichri et al. (2019), aunque estos estudios tienen un enfoque más general y no se limitan únicamente a la problemática específica de los remolcadores teledirigidos.



Figura 1.1. Remolcador teledirigido estándar de la empresa Towbots (s.f.)

En el ámbito nacional, no se han identificado proyectos o soluciones robóticas similares a los mencionados anteriormente. Sin embargo, sí se han llevado a cabo investigaciones en el diseño robótico dentro de universidades locales. Un ejemplo de ello es el desarrollo de un robot autónomo móvil terrestre, que se centra en la fusión sensorial y la localización

y mapeo simultáneos (*Simultaneous Localization and Mapping*, SLAM), detallado por Bernheim, Costa, y De Luca (2019). Este robot está diseñado para navegar por trayectorias no predefinidas. A pesar de constituir un progreso significativo desde una perspectiva teórico-práctica, este avance no atiende directamente las necesidades específicas de la industria aeronáutica.

Este proyecto parte con el diseño conceptual, el cual se fundamenta en los requerimientos técnicos especificados por el cliente. Estos requerimientos son derivados del análisis del entorno operativo y se ajustan a las normativas de seguridad en aviación dictadas por la Dirección Nacional de Aviación Civil e Infraestructura Aeronáutica (DINACIA). Esta entidad referencia estándares internacionales, incluyendo las *Technical Standard Orders* (TSO) y las normativas de la *International Electrotechnical Commission* (IEC). Dado que el robot se clasifica dentro de la categoría de robots colaborativos, se asegura su conformidad con las normas ISO/TS 15066. Estas normas estandarizan los requerimientos de seguridad para este tipo de equipos, garantizando así una operación segura. A continuación, se realiza un análisis cinemático del diseño mecánico para comprender mejor sus capacidades de movimiento y restricciones. Seguidamente, se examina el algoritmo de navegación del robot, elemento crucial para su desplazamiento autónomo y preciso. Este paso precede a la implementación del controlador del remolcador, cuyo desempeño es meticulosamente evaluado mediante el uso del simulador Webots, permitiendo así ajustes y optimizaciones basadas en los resultados obtenidos.

Los hallazgos de esta investigación constituyen un pilar fundamental para el lanzamiento de un emprendimiento innovador. Este no solo permitirá que la empresa de mantenimiento aeronáutico se posicione a la vanguardia del sector en la región, proporcionándole un valor agregado único, sino que también establece una guía metodológica a nivel académico para el diseño de robots. Esta guía tiene el potencial de ser aplicada en diversas áreas industriales, fomentando así una colaboración más estrecha entre el ámbito académico y las empresas tecnológicas regionales.

1.2. OBJETIVOS GENERAL Y ESPECÍFICOS

1.2.1. Objetivo general

Simular la automatización de un robot para remolcar avionetas utilizando el software Webots.

1.2.2. Objetivos específicos

1. Establecer el diseño conceptual del robot remolcador para preparar los parámetros de la simulación.
2. Analizar el modelo cinemático y de navegación del robot remolcador para avionetas.
3. Implementar la simulación en Webots para evaluar una arquitectura del sistema de navegación.
4. Verificar la simulación en el software Webots.

1.3. ALCANCE Y LIMITACIONES

Este proyecto está enfocado principalmente en la simulación de la automatización de un sistema de remolque de avionetas Learjet 35/35A/36/36A. Se consideran elementos de la visita y se modela según un aeropuerto estándar. El acceso a las avionetas y recursos es limitado por lo que el objeto de este proyecto se limita a obtener una simulación del robot remolcando una avioneta en el software Webots.

CAPÍTULO 2. MARCO TEÓRICO

2.1. ANTECEDENTES

Jiménez Cuesta (2012) realizó un estudio enfocado en la simulación de vehículos eléctricos ligeros, categoría bajo la que clasificó a los robots móviles terrestres. El foco de su investigación se centró en el desarrollo y simulación de algoritmos de control, seguido de su aplicación práctica en un robot físico construido con un kit de Lego Mindstorms NXT. Los resultados destacaron por su alta repetibilidad, evidenciando que las simulaciones podían ser reproducidas con precisión en la implementación física del robot. Esta congruencia entre los resultados simulados y los reales corroboró la exactitud y la fiabilidad del enfoque metodológico adoptado por Jiménez Cuesta. Más allá de los resultados empíricos, el trabajo de Jiménez Cuesta es apreciado por su aporte metodológico al campo. Ofreció una guía exhaustiva para el diseño de algoritmos de control de trayectoria y la realización de simulaciones utilizando software especializado, en este caso, MATLAB. Este software es ampliamente reconocido en el ámbito de la ingeniería e investigación por su capacidad para facilitar el desarrollo de simulaciones complejas, lo que subraya la contribución significativa de su investigación tanto en términos prácticos como teóricos.

Almasri, Alajlan y Elleithy (2016) presentaron una técnica avanzada para la robótica móvil autónoma enfocándose en la capacidad del robot de planificar trayectorias y evitar obstáculos. Este estudio destaca por el uso del simulador Webots, que permite evaluar la efectividad de distintos algoritmos en un entorno controlado usando el robot E-puck. Mediante el empleo de sensores infrarrojos de bajo costo, los investigadores lograron desarrollar un sistema capaz de seguir caminos predefinidos y evitar obstáculos en tiempo real, demostrando la importancia del simulador Webots como herramienta clave para la experimentación y validación de algoritmos de navegación autónoma. Este enfoque no solo resalta el potencial de los simuladores en la investigación robótica, sino que también abre nuevas vías para la implementación de robots móviles autónomos en aplicaciones prácticas.

Freire y Rubio (2016) hicieron una contribución destacada al ámbito del transporte de avionetas mediante el diseño y construcción de un robot remolcador teleoperado, como se ilustra en la Figura 2.1. Su metodología se enfocó principalmente en los componentes mecánicos del diseño, con una selección meticulosa de los dispositivos electrónicos de potencia necesarios tanto para el funcionamiento del motor eléctrico como para los

transmisores de radiofrecuencia. El logro más significativo de su investigación fue el desarrollo de un prototipo funcional capaz de remolcar una avioneta Cessna 150 de manera eficiente. La realización exitosa de estas pruebas confirmó la viabilidad y efectividad del diseño propuesto, marcando un hito relevante para el avance de soluciones robotizadas en el sector de la aviación.

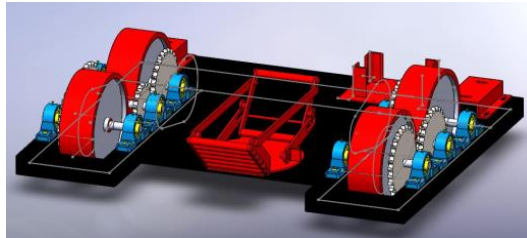


Figura 2.1. Modelo 3D diseñado por Freire y Rubio (2016)

Bernheim, Costa, y De Luca (2019) desarrollaron un robot autónomo móvil terrestre equipado con ruedas omnidireccionales, enfocándose de manera particular en la implementación de algoritmos de control de navegación para Localización y Mapeo Simultáneos (SLAM) y Fusión Sensorial, como se muestra en la Figura 2.2. El resultado de su trabajo fue la creación de un robot inspirado en el diseño Turtlebot de tres niveles. La importancia de esta investigación radica en su enfoque metodológico hacia el desarrollo de un robot de uso general, destacando especialmente en el análisis y aplicación de avanzados algoritmos de navegación.

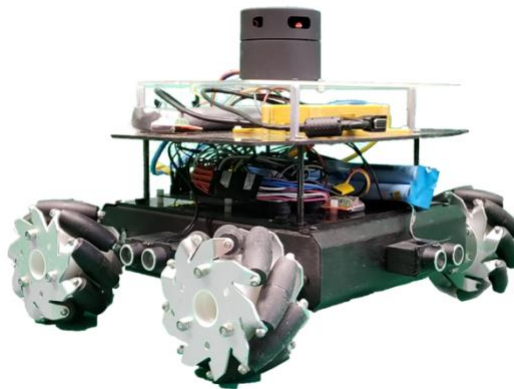


Figura 2.2. Rovert, desarrollo de Bernheim, Costa y De Luca (2019)

Al Khatib, Kareem Jaradat y Abdel-Hafez (2020) introdujeron un sistema de navegación revolucionario diseñado específicamente para robots móviles de bajo costo, fundamentado en la Fusión Sensorial y el algoritmo de control *Extended Kalman Filter*

(EKF). El grupo investigador puso a prueba dos configuraciones del sistema: una optimizada para entornos cerrados y la otra para áreas abiertas. Los experimentos se realizaron ajustando el robot según el entorno de operación, lo que evidenció la adaptabilidad de los algoritmos creados. Un hallazgo clave de este estudio fue la selección de sensores, actuadores y sistemas de control que maximizan la eficiencia y reducen los costos de los sistemas de navegación. Este descubrimiento es de gran importancia para la investigación en curso, ya que ofrece una base práctica y robusta para el diseño de sistemas de navegación robótica económicamente accesibles.

Priyadharshini et al. (2023) exploraron la implementación de SLAM y Fusión Sensorial en el robot E-Puck en el software de simulación Webots. Su objetivo era programar y controlar eficazmente el robot E-Puck para mapear áreas desconocidas, utilizando el filtro EKF para manejar la incertidumbre de la localización del robot durante la navegación. La implementación fue exitosa, demostrando la capacidad del sistema para localizarse y construyendo un mapa del entorno y navegando por él, proceso que marcó un avance significativo en la navegación robótica. Este trabajo destaca por su contribución metodológica, ofreciendo un enfoque detallado para el desarrollo y simulación de algoritmos de navegación SLAM en robots móviles. La utilización de Webots y Python para la simulación subraya la importancia de las herramientas de software en la investigación robótica, pues permite realizar pruebas más eficientes y económicas. Asimismo, el estudio contribuye a nuestro trabajo pues aborda la creación de un entorno de simulación realista y la importancia de la precisión entre las simulaciones y las implementaciones reales.

2.2. BASES TEÓRICAS

En esta sección se exponen los fundamentos teóricos aplicados en el desarrollo de este proyecto.

2.2.1. Fundamentos de robótica

El concepto "robot" se puede definir según los parámetros de distintas normas e instituciones. En este trabajo se opta por la definición elaborada por ISO 8373:2021 (2021) que establece que es un robot es un "mecanismo programado actuado con un grado de autonomía para realizar locomoción, manipulación o posicionamiento". En cuanto a robot de servicio, se define como "robot que realiza tareas útiles para humanos o equipos, utilizado tanto en uso personal como profesional".

Siciliano et al. (2009) definen los componentes fundamentales de un robot, destacando la importancia de un sistema mecánico que incluye el aparato locomotor y los actuadores para la manipulación. Este sistema está compuesto por motores, servomotores, transmisiones y controladores, los que son cruciales para la gestión eficiente de los componentes por medio de su unidad central de procesamiento. La capacidad perceptiva del robot, esencial para la recopilación de datos del entorno, juega un papel clave en la toma de decisiones y en la determinación de las acciones a seguir. Este aspecto incluye sensores de posición, sistemas de visión por cámara y *encoders* para ruedas, entre otros. Además, un sistema de control regula el procesamiento de los datos adquiridos y facilita la ejecución de acciones correspondientes. Por estas razones, la robótica se establece como un campo de conocimiento interdisciplinario que engloba las áreas de mecánica, control, informática y electrónica, resaltando la integración de estos campos para el desarrollo y funcionamiento efectivo de los robots.

Por otro lado, en Mihelj et al. (2019) se identifica a un robot móvil terrestre como un aparato con capacidad de locomoción autónoma que puede desplazarse por su entorno mediante el uso de ruedas, orugas, piernas, rieles, o una combinación de estos elementos. A su vez, realiza una primera clasificación de robots móviles terrestres según su aplicación, la que se representa en la Figura 2.3.

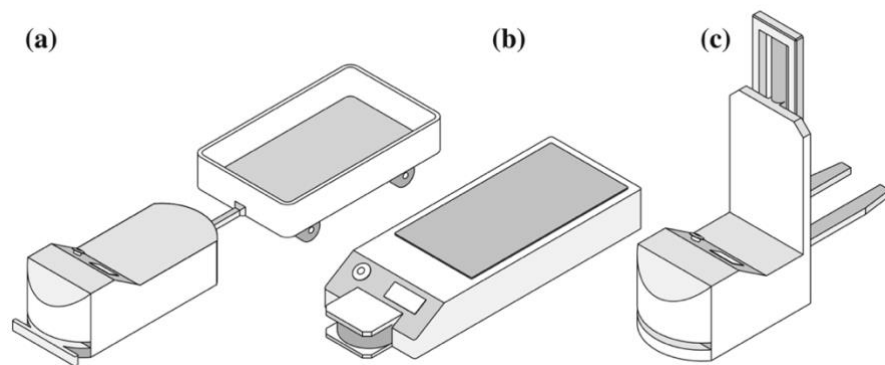


Figura 2.3. Aplicaciones: a. Remolcador, b. Cargador de unidades, c. Montacargas móvil. (Mihelj et al., 2019)

Otra forma de diferenciación se encuentra en el tipo de ruedas y el tipo de configuración en el robot, representados en la Figura 2.4 y la Figura 2.5.

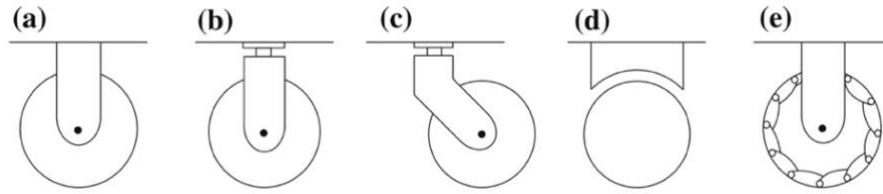


Figura 2.4. Diseño de ruedas: a. Rueda fija standard, b. Rueda orientable standard, c. Rueda giratoria, d. Rueda esférica, e. Rueda mecanum. (Mihelj et al., 2019)

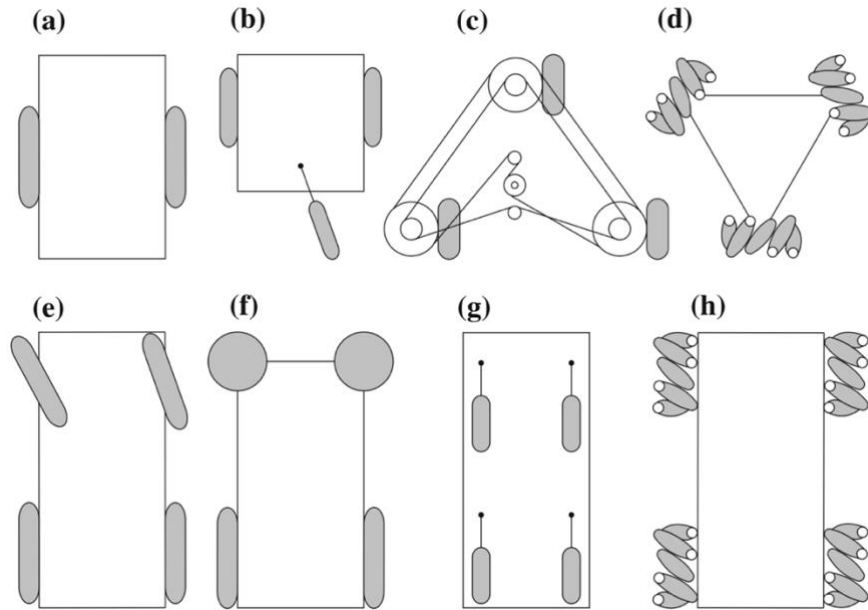


Figura 2.5. a. Tracción diferencial de dos ruedas, b. Tracción diferencial de dos ruedas con rueda giratoria, c. Tres ruedas motorizadas y dirigidas sincrónicamente, d. Tres ruedas omnidireccionales en triángulo, e. Cuatro ruedas con dirección tipo automóvil, f. Dos ruedas de tracción diferencial y dos ruedas omnidireccionales, g. Cuatro ruedas giratorias motorizadas y dirigidas, y h. Cuatro ruedas omnidireccionales en configuración rectangular. (Mihelj et al., 2019)

2.2.2. Actuadores

Un actuador es un dispositivo mecánico que realiza un movimiento controlado en respuesta a una señal de entrada. En el caso de los robots móviles, los actuadores se encargan principalmente de la locomoción, incluyendo movimiento de ruedas, orugas, patas, entre otros. La elección de un actuador depende de las características del robot y las especificaciones de operación, tal como tipos de entorno, velocidad requerida, restricciones de carga y potencia, y fuente de energía.

Tipos de actuadores

- **Motores eléctricos:** son los actuadores más comunes en robótica móvil debido a su simplicidad, eficiencia y capacidad de control. Estos se pueden clasificar en las siguientes categorías:
 - **Motores de corriente continua (DC):** los motores DC convierten energía eléctrica en rotación continua. La velocidad y dirección de giro se controlan mediante la variación del voltaje aplicado y la polaridad. Son ideales para aplicaciones en las que se requiere un movimiento continuo de control sencillo.
 - **Motores sin escobillas (BLDC):** son motores de corriente continua que utilizan un control electrónico capaz de manejar el flujo de corriente en el motor, eliminando las escobillas mecánicas asociadas a este tipo de motor. Ofrecen alta eficiencia y durabilidad, y suelen aplicarse en sistemas de propulsión de drones.
 - **Motores paso a paso:** diseñados para realizar movimientos incrementales discretos, lo que permite un control preciso de posición sin necesidades de un sensor externo. Son útiles para robots que requieran movimientos angulares predefinidos, como manipuladores.
 - **Servomotores:** son actuadores controlados por señales de retroalimentación, haciendo uso de la modulación ancha de pulsos (PWM), lo que les permite alcanzar y mantener posiciones específicas de forma dinámica. Estos dispositivos combinan un motor, un sensor de posición y un controlador integrado. Su aplicación principal se encuentra en la dirección de robots con ruedas.
- **Actuadores hidráulicos:** convierten energía de un fluido presurizado en movimiento mecánico. Proporcionan altas fuerzas y se utilizan principalmente en robots que operan en ambientes exigentes de potencia.
- **Actuadores neumáticos:** similares a los hidráulicos, pero utilizando aire comprimido. Son comunes en aplicaciones industriales donde se necesita velocidad y fuerza, pero su uso en robótica móvil es limitado debido a la menor precisión y eficiencia energética.

2.2.3. Sensores

Son los componentes que permiten al robot percibir su entorno y monitorear su propio estado. Estos dispositivos convierten estímulos físicos del mundo real en datos procesables, como ser señales eléctricas, que el robot utiliza para tomar decisiones. En el diseño de un robot móvil, la selección y disposición de sensores son determinantes para garantizar que el robot pueda operar de manera autónoma y eficiente en su entorno.

Los sensores utilizados en robótica se pueden clasificar en función de su propósito y del tipo de datos que recogen:

- **Sensores de percepción del entorno:** estos sensores permiten al robot detectar y analizar su entorno, identificando obstáculos, estructuras y otros objetos relevantes para su operación.
- **LiDAR:** emplea pulsos láser para medir distancias y es capaz de crear mapas tridimensionales del entorno. Es uno de los sensores más utilizados en navegación autónoma debido a su precisión y capacidad para operar en largas distancias.
- **Cámaras:** capturan imágenes en 2D o 3D, proporcionando información visual rica y detallada para detectar distancias y reconocer objetos del entorno.
- **Sensores de ultrasonido:** generan ondas de sonido de alta frecuencia y miden el tiempo que tarda el eco en regresar para calcular distancias.
- **Sensores infrarrojos:** emplean la reflexión de la luz infrarroja para detectar proximidad y medir distancias cortas.
- **Sensores de estado interno:** son capaces de monitorizar los parámetros internos de un robot, como el movimiento de sus actuadores y la orientación del robot.
- **Encoders:** miden la rotación o el desplazamiento lineal de motores y ruedas. La información de los *encoders* se utiliza para calcular la posición y la velocidad del robot.
- **IMU:** combina acelerómetros y giroscopios para medir aceleraciones lineales y rotaciones angulares. Estos datos son fundamentales para estimar la orientación y detectar cambios dinámicos en el movimiento del robot.

- **Giroscopios:** miden directamente la velocidad angular del robot, complementando a los acelerómetros en la estimación de orientación.
- **Sensores de navegación global:** proporcionan información sobre la posición global del robot en relación con un sistema de coordenadas fijo.
- **GPS:** se utiliza principalmente para aplicaciones de exterior y permite determinar la ubicación del robot mediante señal satelital.
- **Sistemas basados en RFID:** utilizan etiquetas RFID ubicadas en el entorno que el robot lee para determinar su posición.

2.2.4. Ejecución de tareas

Es la capacidad de un sistema robótico para realizar acciones específicas, previamente planificadas de manera eficiente, precisa y acorde a los objetivos del sistema. Este concepto no solo implica completar las tareas, sino también gestionar los cambios en el entorno y adaptarse a condiciones dinámicas. En robótica móvil, la ejecución de tareas abarca aspectos como la planificación de trayectorias, la navegación e interacción con su entorno, la coordinación de múltiples subsistemas y, en algunos casos, la colaboración entre varios robots.

Se puede realizar la ejecución de tareas en diferentes niveles según su complejidad y la naturaleza del sistema de control subyacente. Los principales métodos de ejecución se enumeran a continuación:

- **Ejecución reactiva:** se basa en respuestas inmediatas a estímulos del entorno, sin recurrir a modelos internos complejos ni planificación a largo plazo. Este tipo de ejecución de tareas es ideal para tareas simples donde la rapidez de la respuesta es prioritaria. Generalmente operan bajo la premisa de “estímulo-respuesta”. Se suelen emplear en subsistemas descentralizados y diseñados para tareas específicas donde la toma de decisiones se basa únicamente en datos sensoriales instantáneos.
- **Ejecución deliberativa:** se basa en la planificación previa y el uso de modelos internos para tomar decisiones informadas. Este enfoque es fundamental para tareas complejas que requieran evaluar múltiples opciones y prever consecuencias. Por lo general, estos modelos utilizan mapas, modelos cinemáticos y algoritmos de

planificación de rutas para evaluar trayectorias, secuencias de acción y estrategias.

- **Ejecución híbrida:** combina elementos de los sistemas reactivos y deliberativos, aprovechando lo mejor de cada uno. Es la más utilizada en sistemas robóticos modernos, ya que equilibra la planificación estratégica con la adaptabilidad a estímulos inmediatos.

Ahora bien. Para implementar estas estrategias de ejecución, se utilizan herramientas de modelado que organizan y gestionan los comportamientos del robot de manera estructurada. Las más comunes son:

- **Máquinas de estado finito (FSM):** son modelos simples que representan un sistema conjunto de estados finitos, transiciones entre ellos y acciones asociadas. Son especialmente utilizados para implementar comportamientos reactivos y secuenciales.
- **Máquinas de estado finito jerárquicas (HFSM):** extienden las FSM introduciendo una jerarquía en la estructura de estados. Esto permite manejar comportamientos complejos mediante la descomposición de niveles. Se crean superestados, donde se agrupan sub-estados relacionados que simplifican la gestión y las transiciones ocurren tanto entre sub-estados como entre superestados. Este tipo de modelado es ideal para ejecución deliberativa.
- **Árboles de comportamiento (*Behaviour Tree*, BT):** son estructuras arbóreas que organizan comportamientos en nodos jerárquicos. Son ideales para sistemas híbridos, ya que combinan la planificación estratégica con reacciones rápidas a eventos. Los nodos hoja representan acciones o condiciones y los nodos internos actúan como controladores que dirigen la ejecución según reglas específicas.

2.2.5. Mapeo en robots móviles

El mapeo es una función clave en los robots autónomos, ya que les permite crear representaciones del entorno donde operan. Estas representaciones son fundamentales para la navegación, localización y la planificación de trayectorias. El proceso de mapeo en robótica combina la percepción del entorno a través de sensores con algoritmos computacionales que transforman estos datos en mapas utilizables. Se pueden encontrar tres tipos principales de mapas:

- **Mapas métricos:** representan el entorno como un espacio continuo o discreto con medidas físicas precisas.
- **Mapas topológicos:** representan ubicaciones claves como nodos conectados por caminos o relaciones espaciales.
- **Mapas híbridos:** combinan mapas métricos y topológicos, beneficiándose de la precisión de los mapas métricos para navegación y la simplicidad de los mapas topológicos para la planificación.

Uno de los enfoques más avanzados en mapeo robótico es el método conocido como SLAM (*Simultaneous Localization and Mapping*), que aborda simultáneamente los problemas de localización y construcción de mapas. En este proceso, el robot utiliza sensores para percibir su entorno y, al mismo tiempo, calcula su posición dentro de ese espacio. El desafío clave en SLAM radica en la incertidumbre inherente a las mediciones sensoriales y los errores acumulativos en la odometría del robot. Para manejar esta incertidumbre, se utilizan técnicas probabilísticas avanzadas, como el filtro de Kalman extendido (*Extended Kalman Filter*, EKF) o métodos de partículas (*Particle Methods*) (Siciliano & Khatib, 2016), que permiten estimar tanto la posición del robot como la estructura del entorno con un alto grado de confiabilidad. Además, el método SLAM necesita resolver el problema de cierre de bucles, que implica reconocer ubicaciones previamente visitadas para ajustar y corregir errores en el mapa global.

El proceso de mapeo comienza con la adquisición de datos recogidos de los sensores del robot. Sensores como LiDAR utilizan pulsos láser para medir distancias con alta precisión, siendo ideales para crear mapas métricos detallados, tanto en 2 como en 3 dimensiones.

Una vez recopilados los datos, se deben procesar para extraer información significativa que pueda integrarse en un mapa. Este proceso incluye la identificación de características clave, como bordes, esquinas y planos, que sirven como referencias espaciales. Posteriormente, se actualiza el modelo del mapa mediante algoritmos especializados, entre los que se encuentran:

- **Occupancy Grid Mapping:** se divide el entorno en celdas cuadradas a las que se les asigna una probabilidad de ocupación, actualizándose según mediciones de sensores. Principalmente se basa en modelos probabilísticos como el filtro

Bayesiano (Siciliano & Khatib, 2016).

- **FastSLAM:** combina partículas para representar la posición del robot. Es eficiente computacionalmente para el manejo de incertidumbre.
- **Graph-Based SLAM:** modela el problema como un grafo, donde los nodos representan ubicaciones y las aristas indican mediciones relativas. Se utilizan técnicas de optimización para minimizar errores acumulados en los bucles.

2.2.6. Navegación de robots móviles

La navegación robótica es el proceso mediante el cual un robot planifica y ejecuta movimientos para desplazarse desde un punto inicial hacia un objetivo dentro de un entorno determinado, evitando obstáculos y optimizando recursos como tiempo, energía o distancia recorrida. Este proceso es fundamental para la autonomía de los robots móviles y combina técnicas de localización, planificación de trayectorias y control de movimiento. La navegación robótica se apoya en modelos del entorno, datos sensoriales y algoritmos que procesan esta información para guiar al robot de manera eficiente y segura.

La navegación puede desglosarse en varios subprocesos clave que trabajan de manera conjunta para garantizar que el robot se desplace hasta llegar a su destino.

- **Localización:** permite que el robot determine su posición dentro del entorno. La precisión de la localización es crucial para que el robot pueda planificar trayectorias con éxito.
- **Percepción del entorno:** se utilizan sensores para recabar la mayor información posible del entorno y así poder detectar objetos y obstáculos que limiten el espacio navegable. Estos datos se integran al modelo del entorno que guía las decisiones de navegación.
- **Planificación de trayectorias:** este componente se encarga de calcular la ruta de navegación desde la posición inicial del robot hasta el objetivo. Los algoritmos de planificación determinan la secuencia de movimiento óptima, teniendo en cuenta restricciones como la presencia de obstáculos, características del terreno y las capacidades del robot.
- **Control de movimiento:** una vez planificada la trayectoria, el robot debe traducir

esta información en movimientos físicos, lo que implica controlar motores, actuadores y otros sistemas de locomoción, incluyendo ajustes dinámicos a los cambios del entorno.

La navegación robótica puede clasificarse según el nivel de conocimiento que el robot tiene del entorno y las técnicas utilizadas para resolver el problema:

- **Navegación basada en mapas:** principalmente utiliza mapas predefinidos para planificar sus movimientos. Este enfoque es eficiente en entornos estructurados donde los mapas son confiables y detallados, y no se encuentran obstáculos dinámicos.
- **Navegación reactiva:** en este caso, el robot no utiliza mapas previos, sino que responde a estímulos sensoriales en tiempo real para moverse. Los comportamientos reactivos incluyen evitar obstáculos, seguir líneas o ajustarse dinámicamente al entorno.
- **Navegación híbrida:** combina los enfoques basados en mapas y reactivos para aprovechar las fortalezas de ambos. El robot utiliza un mapa para planificar rutas generales y realiza ajustes reactivos ante cambios locales.

Uno de los desafíos más complejos de la navegación robótica es la detección, interpretación y evasión de obstáculos. Estos pueden clasificarse en dos grandes categorías: estáticos y dinámicos, cada uno con sus características y retos particulares. La capacidad de un robot para interactuar de manera segura y eficiente con su entorno depende en gran medida de cómo detecta y responde a ambos tipos de obstáculos. El diseño de estrategias robustas para gestionar estas situaciones es fundamental para garantizar la autonomía, seguridad y eficiencia del robot. Para poder entender esto mejor, se explican las diferencias de los dos tipos de obstáculos a continuación:

- **Obstáculos estáticos:** son elementos del entorno cuya posición no cambia con el tiempo. Incluyen paredes, columnas, muebles, estructuras fijas y cualquier objeto inamovible dentro del espacio de trabajo del robot. La gestión de estos obstáculos se basa principalmente en la planificación de trayectorias y la percepción previa del entorno.
- **Obstáculos dinámicos:** son objetos o entidades que cambian de posición a lo

largo del tiempo. Incluyen personas, vehículos, robots móviles, animales y cualquier objeto que pueda moverse. Manejar obstáculos dinámicos es considerablemente más complejo, ya que implica la capacidad de predecir el movimiento futuro de estos objetos y ajustar la trayectoria en tiempo real.

2.2.7. Algoritmos de planificación de trayectorias

La planificación de trayectorias es un componente esencial de la robótica móvil y de manipulación, cuyo objetivo principal es garantizar que un robot pueda alcanzar su destino de manera eficiente y segura, evitando colisiones y optimizando recursos como el tiempo, la energía y la distancia recorrida. Este proceso implica tanto el modelado del entorno como la selección de algoritmos según la naturaleza del problema y las características del robot (Koubaa et al., 2018).

Las categorías principales de planificación son:

- **Planificación global:** este enfoque asume que el entorno del robot es completamente conocido antes de iniciar la planificación. Se basa en modelos estáticos del espacio de trabajo, proporcionando soluciones óptimas en términos de distancia o costo acumulado. Sus principales ventajas son que produce trayectorias globalmente óptimas y permite planificar antes de la ejecución.
- **Planificación local:** diseñada para entornos dinámicos o desconocidos donde el robot utiliza datos de sensores para ajustar su trayectoria en tiempo real. Como valor, ofrece alta adaptabilidad a cambios del entorno y presenta de una respuesta rápida frente a obstáculos inesperados.
- **Métodos heurísticos y metaheurísticos:** estos algoritmos combinan estrategias aproximadas para encontrar soluciones de alta calidad en problemas complejos. Su principal ventaja es que pueden aplicarse en entornos donde los métodos exactos serían computacionalmente prohibitivos.

El modelado del entorno también juega un papel fundamental en el desarrollo de los algoritmos. Los modelos más comunes son:

- **Mapas de ocupación:** se divide el espacio en celdas libres y ocupadas, representadas como un grafo o matriz binaria.

- **Espacio de configuración:** simplifica las dimensiones y restricciones del robot al modelarlo como un punto en un espacio multidimensional. Se aplica principalmente en manipuladores robóticos.

A raíz de estos tipos de características se desarrollan los siguientes algoritmos:

Algoritmo Dijkstra (1959): es un método clásico de búsqueda en grafos, diseñado para encontrar la ruta más corta desde un nodo inicial hasta todos los demás nodos en un grafo ponderado, siempre que los pesos de las aristas sean no negativos. Este algoritmo es ampliamente utilizado en planificación global de trayectorias en mapas de ocupación estáticos conocidos.

Descripción general:

- **Entradas:**
 - $G = (V, E)$: grafo dirigido o no dirigido, donde V es el conjunto de nodos y E el conjunto de aristas.
 - $w(u, v) \in E$: peso asociado a la arista entre los nodos u y v , que representa el costo de moverse entre ellos.
 - s : nodo fuente donde comienza la búsqueda.
 - g : nodo objetivo donde finaliza la búsqueda.
- **Salidas:**
 - $d(v)$: distancia mínima desde el nodo fuente s a cada nodo $v \in V$.
 - $prev(v)$: nodo anterior en el camino más corto desde s a v .
- **Pasos del algoritmo:**
 - **Inicialización:**
 - Establecer $d(s) = 0$.
 - Para todos los demás nodos $v \in V$, $d(v) = \infty$.
 - Los predecesores de cada nodo se inicializan como indefinidos,

$prev(v)$ es indefinido para todo $v \in V$.

- Crear un conjunto de nodos no visitados $Q = V$, llamado cola de prioridad.
- **Iteración:** mientras Q no esté vacío:
 - Seleccionar el nodo $u = \arg \min_{v \in Q} d(v)$ con la menor distancia objetivo.
 - Para cada vecino v de u , se evalúa si el camino a v a través de u es más corto que el camino registrado hasta ahora. Si es así, se actualiza la distancia y el predecesor de v , es decir: si $d(u) + w(u, v) < d(v)$ entonces: $d(v) = d(u) + w(u, v)$ y $prev(v) = u$.
 - Marcar u como visitado, removiéndolo de Q .
- **Finalización:**
 - Cuando Q esté vacío o el nodo objetivo g ha sido procesado, el camino más corto se reconstruye retrocediendo desde el nodo objetivo hasta el nodo fuente s , siguiendo los predecesores almacenados.

En términos de complejidad computacional, su costo es de $O((|V| + |E|) \cdot \log|V|)$, lo que puede ser elevado en grafos densos o con muchos nodos, afectando su eficiencia en sistemas con recursos limitados o en escenarios que requieren planificación en tiempo real.

Algoritmo RRT (*Rapidly-exploring Random Tree*) (LaValle S. M., 1998): el objetivo de este algoritmo es construir de manera iterativa un árbol que explora el espacio de configuración del robot, extendiéndose hacia áreas no exploradas. A través de una exploración aleatoria y guiada, el RRT genera rutas viables desde un punto inicial hasta un objetivo, respetando las restricciones del entorno. Su principal fortaleza radica en su capacidad para explorar eficientemente espacios de búsqueda grandes, no estructurados y con obstáculos.

Descripción general:

- **Entradas:**

- x_{init} : posición inicial del robot.
- x_{goal} : posición objetivo.
- C_{free} : espacio de configuración libre de obstáculos.
- C_{obs} : espacio de configuración ocupado por obstáculos.
- δ_q : distancia máxima de expansión del árbol en cada iteración.
- N_{max} : número máximo de iteraciones.
- ϵ : umbral de cercanía del objetivo.

- **Salidas:**

- Trayectoria: camino desde x_{init} hasta x_{goal} si se encuentra.
- Árbol de búsqueda T : estructura que almacena los nodos y conexiones generadas.

- **Pasos del algoritmo:**

- **Inicialización del árbol:** se comienza creando un árbol T con un único nodo correspondiente al estado inicial del robot: $T = \{x_{init}\}$.
- **Generación de un punto aleatorio:** se selecciona aleatoriamente un punto x_{rand} dentro del espacio de configuración C . Este nodo actúa como una guía para expandir el árbol hacia nuevas áreas.
- **Búsqueda del nodo más cercano:** se identifica el nodo $x_{nearest}$ en el árbol T que esté más cerca de x_{rand} . Esto se calcula utilizando la distancia euclidiana: $x_{nearest} = \arg \min_{x \in T} \|x - x_{rand}\|$.
- **Expansión hacia el punto aleatorio:** desde $x_{nearest}$, se genera un nuevo nodo x_{new} avanzando en dirección a x_{rand} a una distancia máxima δ_q : $x_{new} = x_{nearest} + \delta_q \cdot \frac{x_{rand} - x_{nearest}}{\|x_{rand} - x_{nearest}\|}$. Este paso limita la expansión del árbol,

controlando el crecimiento y mejorando su eficiencia.

- **Verificación de colisiones:** es necesario comprobar si el camino entre $x_{nearest}$ y x_{new} atraviesa algún obstáculo. En caso de que no haya colisiones, el nuevo nodo se añade al árbol: $T = T \cup x_{new}$.
- **Verificación de cercanía al objetivo:** se evalúa si x_{new} está lo suficientemente cerca del objetivo x_{goal} , utilizando el umbral ε : $\|x_{new} - x_{goal}\| < \varepsilon$. Si la condición se cumple, se ha encontrado un camino desde x_{init} a x_{goal} . En caso contrario, se regresa al paso 2.

En términos de complejidad computacional, el algoritmo tiene un costo de $O(k \cdot \log n)$, donde k es el número de muestras generadas (x_{rand}) y n el número de nodos en el árbol, lo que implica un aumento significativo en el tiempo de cálculo a medida que el espacio de búsqueda y los obstáculos crecen.

Algoritmo A* (Hart et al., 1968): es una modificación del algoritmo Dijkstra que integra una estimación heurística del costo para alcanzar el nodo objetivo, con el fin de reducir el número de estados explorados.

Descripción general:

- **Entradas:**

- $G = (V, E)$: grafo donde V es el conjunto de nodos y E el conjunto de aristas.
- $w(u, v)$: costo o peso de la arista entre los nodos u y v .
- s : nodo de inicio.
- g : nodo objetivo.
- $h(v)$: función heurística que estima el costo desde el nodo v hasta el nodo objetivo g .

- **Salidas:**

- *Camino óptimo*: secuencia de nodos desde el nodo inicial s hasta el nodo objetivo g .

- *Costo total*: la suma de los costos de las aristas que componen el camino óptimo.
- **Pasos del algoritmo:**
 - **Inicialización:** se inicializan tres conjuntos:
 - *OPEN*: conjunto de nodos que están pendientes de ser evaluados. Inicialmente contiene solo el nodo de inicio s .
 - *CLOSED*: conjunto de nodos ya evaluados. Inicialmente vacío.
 - $g(v)$: costo acumulado desde el nodo de inicio s hasta v , inicializado como infinito excepto para s , que vale 0.
 - Se define la función de evaluación total para cada nodo v : $f(v) = g(v) + h(v)$.
 - **Selección del nodo con menor costo:**
 - Se selecciona de la lista *OPEN* el nodo u con el menor valor de $f(u)$: $u = \arg \min_{v \in OPEN} f(v)$
 - **Verificación del objetivo:**
 - Si el nodo seleccionado u es el objetivo g , se ha encontrado el camino más corto y el algoritmo finaliza.
 - **Expansión de vecinos:**
 - Para cada vecino v de u , se calcula el costo tentativo de pasar por u : $g_{tent}(v) = g(u) + w(u, v)$.
 - Si este costo es menor que el valor actual de $g(v)$, se actualizan los valores: $g(v) = g_{tent}(v)$ y $f(v) = g(v) + h(v)$.
 - Se establece u como predecesor de v : $prev(v) = u$.
 - Si v no está en *OPEN*, se añade al conjunto para ser evaluado.

- **Mantenimiento de conjuntos:**
 - Una vez evaluado u , se mueve de *OPEN* a *CLOSED* para evitar reevaluaciones.
- **Iteración:** se repiten los pasos del 2 al 5 hasta encontrar al nodo objetivo g o hasta que el conjunto *OPEN* quede vacío.

Su complejidad computacional es de $O((|E| + |V|) \cdot \log|V|)$, donde $|V|$ es el número de nodos, y $|E|$, el número de aristas, lo que implica que su rendimiento decrece considerablemente en grafos grandes o con muchos obstáculos, afectando su aplicabilidad en sistemas de navegación en tiempo real.

Algoritmo D* (Stentz, 1994): el algoritmo D* (Dynamic A*) es una extensión del algoritmo A* diseñado para entornos dinámicos, donde la planificación de trayectorias debe ajustarse a medida que se obtiene nueva información del entorno. Fue desarrollado para ser una alternativa eficiente en la navegación de robots móviles en escenarios parcialmente conocidos o en los que pueden surgir obstáculos imprevistos.

Descripción del algoritmo:

- **Definiciones:**
 - $G = (V, E)$: grafo donde V es el conjunto de nodos y E el conjunto de aristas.
 - $r(v)$: etiqueta del nodo v , indicando la pertenencia a alguna de las siguientes listas: lista *OPEN*, el nodo es considerado en la expansión del camino, la lista *NEW*, indicando que nunca estuvo en la lista *OPEN* y la lista *CLOSED*, indicando que fue removido de la lista *OPEN*.
 - $w(u, v)$: costo o peso de la arista entre los nodos u y v . Por ejemplo, distancia euclidiana: $w(u, v) = \sqrt{(x_u - x_v)^2 + (y_u - y_v)^2}$.
 - s : nodo de inicio.
 - g : nodo objetivo.
 - $h(v)$: función heurística que estima el costo desde el nodo v hasta el nodo objetivo g . Por ejemplo, distancia manhattan: $h(v) = |x_g - x_v| + |y_g - y_v|$.

- $k(v)$: función clave, cada nodo v en la lista $OPEN$ posee un valor clave $k(v) = \min_{v \in OPEN} \{h_{old}(v), h_{now}(v)\}$.
- Nodo *RAISE*: se considera al nodo v como *RAISE* si $k(v) < h(v)$, lo que indica que el costo estimado hacia el objetivo ha aumentado.
- Nodo *LOWER*: se considera al nodo v como *LOWER* si $k(v) = h(v)$, lo que indica que el costo estimado al objetivo es óptimo o se ha reducido.
- $b(v)$: puntero de retroceso, cada nodo tiene puntero de retroceso que apunta al siguiente nodo del camino óptimo actual.

- **Salidas:**

- *Camino óptimo*: secuencia de nodos desde s hasta g , conformado por el conjunto de punteros de retroceso $b(v)$.
- *Campo de costos*: se obtiene un campo de costos, que se ajusta dinámicamente conforme se detectan discrepancias en el entorno.

- **Pasos del algoritmo:**

- **Inicialización:**

- Se establece $h(g) = 0$ y $h(v) = \infty, \forall v \neq g$.
- Se marcan todos los nodos en la lista *NEW* (nodos no procesados) y se inserta el nodo objetivo g en la lista *OPEN* con su clave $k(g) = 0$.

- **Procesamiento de los nodos:**

- Se remueve el nodo con la clave mínima: $v: \min_{v \in OPEN} \{k(v)\}$. Se registra su valor $k_{old} = k(v)$, se elimina de la lista *OPEN* y se integra en la lista *CLOSED*.
- 1. Si $k_{old} = h(v)$, entonces se considera expansión de nodo *LOWER*. Para cada vecino u de v : si $h(u) > h(v) + w(u, v)$ entonces se actualiza $h(u) = h(v) + w(u, v)$ y $b(u) = v$. Se

inserta (o se re-inserta) u en la lista $OPEN$ con $k(u) = \min \{h(u), \text{valor previo de } k(u)\}$.

2. Si $k_{old} < h(v)$, entonces se considera expansión de nodo $RAISE$. Si $b(u) = v$, se inserta u en la lista $OPEN$ con $k(u) = h(v) + w(u, v)$

- Se continúa removiendo el siguiente nodo óptimo hasta que el valor mínimo de la lista $OPEN$ no sea menor que el costo en el nodo actual del robot. En ese punto, el mapa de costos desde la posición actual del robot hacia la meta se considera óptimo, obteniendo el camino conformado por los punteros de retroceso $b(V)$ y el nodo de la posición del robot se inserta en la lista $CLOSED$.

○ **Replanificación y actualización:**

- Cuando los sensores del robot detectan una discrepancia entre el costo esperado y el costo real de un nodo, el algoritmo debe actualizar ese costo y reparar la porción afectada del mapa de costos de la siguiente forma:

1. Se actualiza el costo, $w(u, v) = \text{nuevo valor del costo}$.

2. Se reinsertan los nodos afectados: si los nodos v o u ya se encuentran en la lista $CLOSED$, se reinsertan en la lista $OPEN$.

3. Se repite el paso de procesamiento de nodos repetidamente para que se propaguen los cambios a lo largo del camino hasta que la clave mínima en la lista $OPEN$ sea mayor o igual al costo en la posición actual del robot.

○ **Movimiento del robot:**

- El robot sigue la ruta calculada siguiente los punteros de retroceso $b(V)$: $s \rightarrow b(s) \rightarrow b(b(s)) \rightarrow b(b(b(s))) \rightarrow \dots \rightarrow g$, hasta llegar a su destino o replanificar el camino.

Algoritmo D* Lite (Koenig, & Likhachev, 2002): es una versión mejorada del algoritmo D* (Stentz, 1994), el cual mantiene su capacidad de reajustar rutas cuando cambia la estructura del entorno. D* Lite sigue un enfoque de búsqueda inversa, comenzando la planificación desde el nodo objetivo y expandiéndose hacia la posición actual del robot. También elimina las listas *NEW*, *OPEN* y *CLOSED* lo cual reduce la gestión de memoria y de cálculos requeridos al momento de reajustar la ruta cuando aparecen nuevos obstáculos.

Descripción del algoritmo:

- **Definiciones:**

- G : mapa del entorno, representado como un grafo de nodos o una cuadrícula de celdas con valores de ocupación. Se actualiza dinámicamente cuando se detectan cambios en el entorno.
- s : nodo inicial, posición actual del robot.
- g : nodo objetivo, destino final al que el robot debe llegar.
- $w(u, v)$: costo o peso de la arista entre los nodos u y v . Por ejemplo, distancia euclidiana: $w(u, v) = \sqrt{(x_u - x_v)^2 + (y_u - y_v)^2}$.
- $h(v)$: función heurística que estima el costo desde el nodo v hasta el nodo objetivo g . Por ejemplo, distancia manhattan: $h(v) = |x_g - x_v| + |y_g - y_v|$.
- $rhs(v)$: valor de un paso de v , definida como: $rhs(v) = 0, v = g, rhs(v) = \min_{u \in pred(v)} \{w(u, v) + h(u)\}, v \neq g$.
- km : variable acumulativa que se incrementa cada vez que el robot se mueve, se utiliza para otorgar coherencia a las claves en la cola de prioridad Q sin tener que ordenarla completamente.
- Q : cola de prioridad ordenada según la clave $k(v)$ y km .
- $k(v)$: función clave, donde se asocia cada nodo v a un vector clave de dos componentes: $k(v) = [\min\{h(v), rhs(v)\} + w(s, v) + km, \min\{h(v), rhs(v)\}]$.

- **Salidas:**

- *Camino óptimo*: secuencia de nodos desde s_{start} hasta s_{goal} .
- M : mapa actualizado, si se detectan nuevos obstáculos, el mapa se modifica y el camino se recalcula.
- $g(s)$ y $rhs(s)$: se actualizan los costos mínimos acumulados para cada nodo en la ruta.

- **Pasos del algoritmo:**

- **Inicialización:**

- El algoritmo comienza estableciendo valores iniciales para cada nodo: $g(v) = \infty$, $rhs(g) = 0$ y $rhs(v) = \infty$, $\forall v \neq g$.
- El nodo objetivo g se inserta en la cola de prioridad con clave $k(g)$.
- Si bien el algoritmo se ejecuta desde el nodo objetivo g hacia el nodo actual del robot, se denotará como s al nodo de la posición actual del robot.

- **Procesamiento de nodos:**

- Se actualiza el costo de los nodos: se calcula $rhs(v)$. Si $h(v) \neq rhs(v)$, se inserta en la cola de prioridad con clave $k(v)$
- Mientras la clave del nodo superior de la cola de prioridad es menor que $k(s)$ o $rhs(s) \neq h(s)$ se repiten los siguientes pasos:
 1. Se extrae de la cola de prioridades el nodo v con menor clave.
 2. Si $h(v) > rhs(v)$: $h(v) = rhs(v)$. Se actualiza el costo de los nodos sucesores u de v .
 3. Si $h(v) \leq rhs(v)$: $h(v) = \infty$. Se actualiza el costo de los nodos sucesores u de v y también se actualiza el costo de v .

4. Este proceso continúa hasta que se cumpla alguna de las condiciones.

- **Replanificación y actualización:**

- Con cada paso del robot, se incrementa $km = km + w(s_{old}, s_{new})$.
- Cuando se detecta un cambio en el costo de algún nodo, se actualiza el costo correspondiente y se procesan los nodos afectados hasta reparar el camino.

Ventajas del algoritmo D* Lite

D* Lite presenta diversas ventajas en comparación con otros algoritmos de planificación de caminos en entornos dinámicos:

- **Replanificación eficiente:** en lugar de recalcular toda la ruta cuando ocurre un cambio en el entorno, D* Lite solo actualiza los nodos afectados, lo que reduce significativamente la cantidad de cómputo requerido en comparación con A*.
- **Búsqueda inversa:** D* Lite busca la ruta óptima desde el objetivo hacia el punto de inicio, lo que permite actualizar de manera más eficiente la trayectoria a medida que el robot se mueve.
- **Menor almacenamiento de memoria:** utiliza una única cola de prioridad para gestionar las expansiones de nodos, lo que simplifica la estructura de datos y mejora la eficiencia en comparación con otros algoritmos que requieren múltiples listas.
- **Adaptabilidad a entornos dinámicos:** es capaz de ajustarse rápidamente a la aparición de nuevos obstáculos o cambios en el terreno sin necesidad de volver a calcular toda la ruta.
- **Optimización en escenarios grandes:** gracias a su mecanismo de actualización localizada, el algoritmo funciona bien en mapas grandes sin un aumento exponencial del tiempo de cómputo.

Por último, es necesario destacar que su complejidad computacional es la misma que

la del algoritmo A* (y Dijkstra), ya que utiliza la misma función heurística. Por lo tanto, es del orden $O((|E| + |V|) \cdot \log|V|)$. Igualmente, posee una ventaja en que cuando recalcula la trayectoria, solo lo hace en base a los nodos afectados, reduciendo considerablemente el costo computacional con respecto a los algoritmos anteriores.

2.2.8. Simulador Webots

Webots (Cyberbotics, s.f.) es un simulador de robótica de código abierto diseñado para el desarrollo, prueba y validación de algoritmos de control de robots en entornos virtuales. Webots se ha consolidado en la industria y en la educación como una herramienta clave que proporciona un entorno de desarrollo para modelar, programar y simular robots móviles y manipuladores. Una de sus ventajas es que dispone de una amplia gama de robots, sensores, actuadores y modelos de objetos configurables. Además de permitir crear nuevos modelos desde cero o importar modelos 3D desde software CAD. En la Figura 2.6 se puede observar el IDE de Webots.

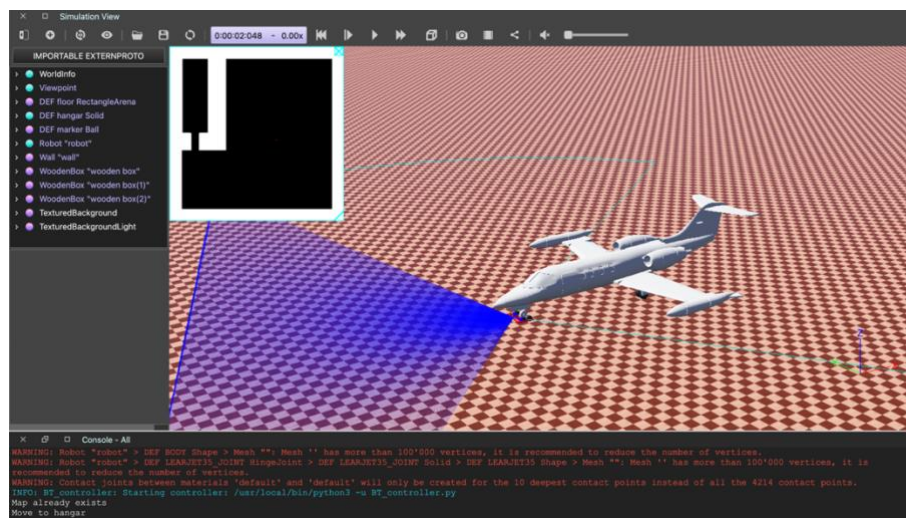


Figura 2.6. Simulador Webots

Características principales:

- **Simulación física realista:** utiliza una versión del motor de física ODE (Open Dynamics Engine) para simular de forma precisa interacciones físicas como fricción, colisiones, fuerzas y momentos. Esto permite emular el comportamiento real de los robots en entornos complejos.
- **Amplia biblioteca de robots y sensores:** el simulador incluye una gran variedad

de modelos de robots prefabricados (como drones, robots móviles y brazos robóticos) y sensores (LiDAR, cámaras, GPS, IMU, entre otros), lo que facilita el desarrollo de aplicaciones sin necesidad de modelar desde cero.

- **Entornos personalizables:** Webots permite la creación de entornos complejos mediante la modificación de terrenos, obstáculos, objetos dinámicos y condiciones ambientales. Esto es ideal para simular escenarios del mundo real y validar algoritmos de navegación, visión por computadora y control.
- **Compatibilidad con varios lenguajes de programación:** los robots en Webots pueden programarse en varios lenguajes de programación, incluyendo Python, C, C++, Java y MATLAB, lo que ofrece flexibilidad para investigadores y desarrolladores.
- **Integración con herramientas de robótica:** el simulador se integra fácilmente con frameworks de desarrollo robótico como ROS (Robot Operating System) y sistema de aprendizaje automático.
- **Simulación en tiempo real y acelerada:** Webots permite ejecutar simulaciones en tiempo real o aceleradas para obtener resultados con más velocidad, lo cual es útil para la validación de algoritmos que requieren largos períodos de prueba.

Componentes de Webots:

La estructura de Webots se basa en la creación de mundos virtuales compuestos por los siguientes elementos:

- **Mundos (Worlds):** archivos con extensión .wbt. Son escenarios que definen el entorno de simulación, incluyendo el terreno, obstáculos y objetos interactivos.
- **Nodos (Nodes):** representan entidades dentro del mundo, ya sean robots, sensores, objetos o luces, entre otros.
- **Controladores (Controllers):** programas que definen el comportamiento de los robots y se comunican con los sensores y actuadores.

CAPÍTULO 3. MARCO METODOLÓGICO

3.1. TIPO Y DISEÑO DE INVESTIGACIÓN

Acorde a los objetivos planteados siguiendo los criterios de Sabino (1992), se define el tipo de investigación como descriptiva ya que el propósito de esta investigación es generar una solución práctica (robot remolcador) para un entorno real (hangar de mantenimiento de aeronaves) haciendo uso de conocimiento científico y tecnológico.

El diseño de la investigación es de campo, debido a que los datos primarios a relevar serán aquellos obtenidos mediante las distintas simulaciones que se realizaron para la evaluación de los diseños durante el proyecto. De esta manera, el diseño final se fundamenta en los resultados de estas pruebas, al igual que en función de los requerimientos de diseño que se establecen al comienzo de la investigación y que pertenecen al contexto real.

3.2. POBLACIÓN Y MUESTRA

El elemento en estudio es la simulación de un robot remolcador de avionetas, por lo que se considera al robot como la población y la muestra. Es necesario especificar que, basado en lo anterior, la muestra es del tipo intencional no probabilística (Sabino, 1992).

3.3. TÉCNICAS DE RECOLECCIÓN DE DATOS

Las técnicas de recolección de datos en este trabajo son:

- **Observación indirecta:** se realiza una revisión bibliográfica de manuales, libros y trabajos de diseño robótico.
- **Observación directa:** se utilizan registros internos de la simulación en Webots mediante variables cinemáticas y sensores virtuales que verifican que los parámetros seleccionados durante la investigación cumplen con los requisitos establecidos. También se utilizan procedimientos de captura mediante la interfaz de registro de Webots tanto para recabar datos en tiempo real como al finalizar la simulación. Dichos datos se almacenan en formato .csv para su posterior análisis.
- **Entrevistas estructuradas y no estructuradas:** se realizaron reuniones de intercambio estructuradas usando cuestionarios y no estructuradas, con

conversaciones informales, para poder establecer los requerimientos de diseño en los cuales basarse para este proyecto. Asimismo se sostienen instancias de consultas a expertos (Ingenieros Mecánicos, Aeronáuticos, Computación y Mantenimiento) en el tema que sirvieron de guía durante esta investigación.

3.4. TÉCNICAS DE ANÁLISIS DE DATOS

Las técnicas de análisis de datos que fueron usadas en este trabajo son:

- **Análisis de contenido:** esta técnica se enfoca en la revisión de datos e información que reúnen diversos trabajos desarrollados sobre la temática, al igual que toda información de expertos sobre requisitos y factores que ayuden a la evolución de este proyecto.
- **Comparación de escenarios:** aunque se use el mismo modelo de aeronave (LearJet 35/35A/36/36A), se puede variar parámetros de la simulación (controladores, obstáculos). De este modo se comparan los resultados bajo distintas configuraciones para evaluar la reacción del robot.

3.5. INSTRUMENTOS DE TRABAJO

Este proyecto se realiza con las siguientes herramientas:

- Macbook Air M1 macOS 15.3.
- Webots R2023b.
- Autodesk Fusion 360 2.0.20981 arm64 [Native].
- Python 3.13.

3.6. DESARROLLO SISTEMÁTICO DE OBJETIVOS

En esta etapa se presenta el desarrollo llevado a cabo en cada etapa del proyecto en base a los objetivos planteados en el Capítulo 1.

3.6.1. Establecimiento del diseño conceptual del robot remolcador para preparar los parámetros de la simulación

3.6.1.1. Identificación de requisitos

El diseño conceptual del robot remolcador se realizó consultando las necesidades del entorno real de trabajo. De este modo se identifican los principales retos de diseño y las características que debe cumplir el robot.

Para comenzar con el diseño de esta solución de transporte de aeronaves, se conformó un equipo multidisciplinario al cual poder solicitar su perspectiva sobre la problemática y pasos a dar en este proyecto. La composición del equipo fue:

- Andrea Helena, Ingeniera Aeronáutica. Responsable del hangar de mantenimiento aeronáutico de Melilla.
- Augusto Rojas, Ingeniero Mecánico.
- Manuel Lara, Ingeniero en Computación.
- Yamile Lara, Ingeniera en Mantenimiento.

Con este equipo se realizó una visita al hangar de mantenimiento con el fin de poder observar e identificar algunas de las problemáticas que se presentaban. En la visita se detectaron varias problemáticas, entre ellas el motivo de estudio de esta investigación. Se procedió luego a realizar una consulta a los profesionales del hangar de mantenimiento Alabé para validar la pertinencia de la solución a desarrollar en este proyecto. La misma se realizó mediante una encuesta anónima hecha en Google Forms, donde participó un total de 9 personas. Las preguntas se registran a continuación:

- ¿Cuáles considera son las tareas más frecuentes realizadas en el hangar?
- ¿Cuáles considera son las tareas que implican mayor esfuerzo físico en el hangar?
- ¿Con qué frecuencia se transporta herramientas de porte liviano en su taller de trabajo y en base a su experiencia laboral?
- ¿Con qué frecuencia se transportan herramientas y componentes aeronáuticos de porte pesado en su taller de trabajo y en base a su experiencia laboral?
- ¿Con qué frecuencia se transportan motores de aviones en su taller de trabajo y en base a su experiencia laboral?

- ¿Con que frecuencia se remolcan y transportan aviones en su taller de trabajo? Ya sea para repostaje de las aeronaves, estacionamiento en el hangar o traslado fuera del hangar
- ¿Se desempeña en un área de trabajo ordenada?
- ¿Con qué frecuencia se realizan inspecciones estructurales de aviones en su taller de trabajo?
- ¿Cuáles son los métodos que se utilizan para la inspección estructural de aviones en su taller de trabajo y en base a su experiencia laboral?
- ¿De las siguientes tareas, cuál considera pueda ser realizada por un robot?
- Inspección estructural del avión: ¿qué método o métodos considera son los más apropiados para implementar por un robot en esta tarea?
- Remolque de aviones: ¿qué método o métodos considera son los más apropiados para implementar por un robot en esta tarea?
- Transporte de equipos pesados (motores, herramientas y componentes aeronáuticos de porte grande): ¿qué método o métodos considera son los más apropiados para implementar por un robot en esta tarea?
- Orden del taller: ¿qué método o métodos considera son los más apropiados para implementar por un robot en esta tarea?
- Método de desplazamiento del robot: ¿cuál considera sería la mejor forma de desplazarse un robot dada las condiciones del terreno del taller y exteriores?

Asimismo, se llevó a cabo una reunión con el equipo multidisciplinario con el fin de analizar los resultados obtenidos del cuestionario, los que se presentan en el Anexo. Esta discusión, en conjunto con el estudio del estado del arte, permitió identificar los requisitos técnicos y operativos del proyecto.

3.6.2. Análisis del modelo cinemático y de navegación del robot remolcador para avionetas

3.6.2.1. Estudio del modelo cinemático del robot remolcador

Para proceder a la automatización del remolcador, primero es necesario realizar un estudio cinemático del diseño seleccionado, el que nos permitirá comprender cómo los actuadores controlaran el movimiento en el espacio. Para desarrollar el análisis cinemático, es necesario identificar el tipo de locomoción y los componentes claves del sistema. Para ello se presenta el diseño original en un dibujo 2D en la Figura 3.1.

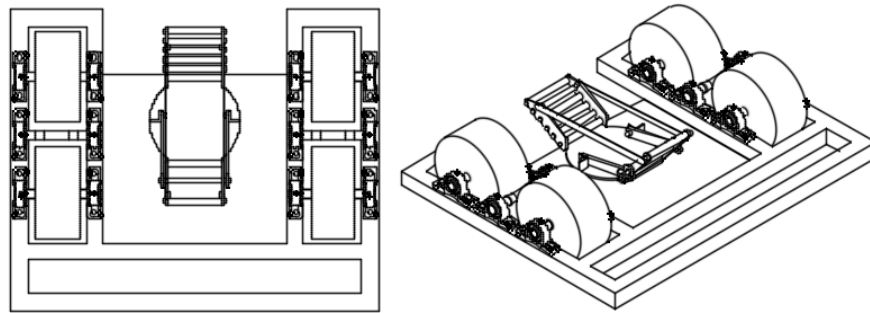


Figura 3.1. Dibujo 2D vista superior y proyectada del robot de Freire y Rubio (2016)

Se puede observar que el robot posee tracción diferencial con cuatro ruedas dispuestas en pares laterales. El centro de masa del robot se encuentra en el centro geométrico del chasis. Su superficie de contacto es plana y sin inclinaciones. Por último, las ruedas no presentan deslizamiento lateral (condición de rodadura pura).

Representación de posición y orientación

Para simplificar el estudio, se puede redefinir el diseño del robot como un cuadrado con dos ruedas, ya que cada una de las ruedas se encuentra actuada por un motor cada par lateral y las ruedas se distribuyen de esa forma por soporte y tracción. Por lo tanto, el robot cumple con las características de un robot de tracción diferencial. Bajo esta premisa, se realiza un diagrama representativo presentado en la Figura 3.2.

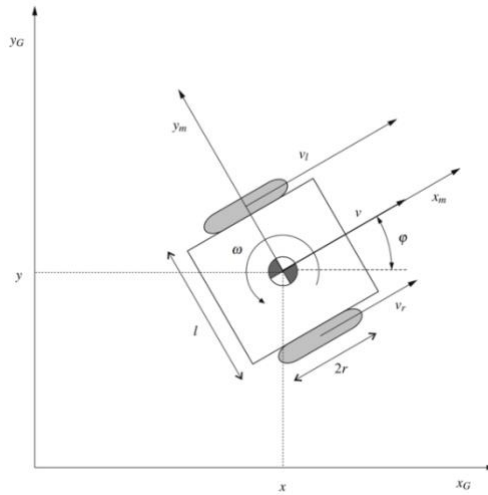


Figura 3.2. Diagrama cinemático del robot (Milehj, 2019)

Donde:

- x_G, y_G : eje de coordenadas globales.
- x_m, y_m : eje de coordenadas locales del robot.

La posición del robot se puede definir mediante tres coordenadas, dos que representan la posición en el plano horizontal y una que describe el ángulo de orientación con respecto al eje vertical, es decir:

$p = \begin{bmatrix} x \\ y \\ \varphi \end{bmatrix}$: donde x representa las coordenadas con respecto al eje x_G , y representa las

coordenadas con respecto al eje y_G y φ determina el ángulo de orientación con respecto al eje vertical. También, en base al diagrama se puede definir la orientación del robot en forma

de matriz de rotación: $R = \begin{bmatrix} \cos \varphi & -\sin \varphi & 0 \\ \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{bmatrix}$, y su matriz transformada homogénea es:

$$T = \begin{bmatrix} \cos \varphi & -\sin \varphi & 0 & x \\ \sin \varphi & \cos \varphi & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Modelado cinemático directo

La velocidad lineal v y la velocidad angular w del robot están relacionadas con las velocidades angulares de las ruedas w_R (rueda derecha) y w_L (rueda izquierda), por lo

tanto:

- $v = \frac{r}{2}(w_R + w_L)$, donde r es el radio de las ruedas.
- $w = \frac{r}{l}(w_R - w_L)$, donde l es la distancia entre las ruedas.

Modelo cinemático inverso

Para determinar las velocidades de las ruedas a partir de la velocidad deseada del robot, se emplean las siguientes ecuaciones:

$$\begin{aligned} \blacksquare \quad w_R &= \frac{2v+l \cdot w}{2r} \\ \blacksquare \quad w_L &= \frac{2v-l \cdot w}{2r} \end{aligned}$$

3.6.2.2. Estudio de navegación del robot remolcador

Evaluación de sistemas de navegación

El remolcador desarrollado por Freire y Rubio (2016) no posee elementos de automatización, ya que lo opera de forma manual un operador que direcciona al robot mediante un dispositivo de radiocontrol (RC). Un microcontrolador PIC F887 se encarga de procesar el accionamiento de dos motores eléctricos mediante un driver de potencia y comunicarse con el dispositivo RC.

Para dotar al robot remolcador de capacidades autónomas, es fundamental evaluar y comparar distintos sistemas de navegación. Por esto, a continuación se analizan los enfoques generales de navegación, considerando aspectos como su precisión, capacidad de adaptación, eficiencia computacional y complejidad de implementación en la Tabla 1.

Tabla 1. Evaluación de sistemas de navegación

Criterio	Navegación basada en mapas	Navegación reactiva	Navegación híbrida
Requiere mapa previo	Sí	No	Parcial

Capacidad de adaptación de obstáculos	Baja	Alta	Alta
Eficiencia computacional	Alta	Baja	Media
Complejidad de implementación	Alta	Baja	Alta
Precisión en entornos estructurados	Muy alta	Baja	Alta
Robustez en entornos dinámicos	Baja	Alta	Alta

La evaluación de los sistemas de navegación permite identificar las diferencias significativas que existen en cuanto a precisión, adaptabilidad y complejidad de implementación. Mientras que los sistemas basados en mapas brindan gran precisión en entornos estructurados, su capacidad de respuesta a cambios del entorno es reducida. En cambio, la navegación reactiva se adapta mejor a entornos dinámicos, pero posee una eficiencia computacional baja. Por su parte, la navegación híbrida equilibra estos aspectos, ofreciendo una alternativa versátil.

Evaluación de elementos para automatización del sistema

En esta sección se busca identificar los sensores necesarios para llevar a cabo la navegación seleccionada anteriormente.

Tabla 2. Evaluación de sensores de orientación

Sensor	Ventajas	Desventajas
---------------	-----------------	--------------------

Giroscopio	Alta precisión en detección de cambios de orientación, rápida respuesta	Puede presentar deriva acumulativa, no mide posición
Unidad de Medición Inercial (IMU)	Mide aceleración y velocidad angular en múltiples ejes, compacto	La deriva acumulativa requiere calibración frecuente
Magnetómetro	Proporciona orientación absoluta respecto al campo magnético terrestre	Sensibilidad a referencias magnéticas, menor precisión en interiores
GPS con brújula digital	Permite orientación global con precisión en exteriores	Dependencia de la señal satelital, baja precisión en interiores

En la Tabla 2 se pueden observar diferencias en los sensores de orientación en cuanto a su precisión y estabilidad. Mientras que el giroscopio y el IMU permiten una detección rápida de cambios de orientación, ambos pueden presentar errores acumulativos. El magnetómetro ofrece una referencia absoluta, aunque su precisión puede verse afectada por interferencias magnéticas. Finalmente, el GPS con brújula digital es útil para la navegación global, aunque su desempeño en interiores es limitado.

Tabla 3. Evaluación de sensores de localización

Sensor	Ventajas	Desventajas
GPS	Alta disponibilidad en exteriores, fácil de integrar	Baja precisión en interiores, precisión limitada (± 1 m)
RTK-GPS (Real-Time Kinematic GPS)	Precisión en centímetros. Ideal para exteriores	Costo alto de implementación, requiere estación base

Balizas Ultra Wideband (UWB)	Alta precisión en interiores, baja latencia	Necesita infraestructura adicional, alcance limitado
Sensores RFID	Costo bajo, fácil instalación de etiquetas en puntos clave	Bajo alcance de lectura, baja precisión espacial

En la Tabla 3 se puede observar que los sensores de orientación varían en términos de precisión y aplicabilidad según el entorno. Mientras que el GPS convencional es de fácil integración, pero con precisión limitada, el RTK-GPS mejora considerablemente la exactitud pero tiene mayor costo. Las balizas UWB representan una alternativa ideal solo para espacios interiores debido a su alta precisión en ese entorno, aunque implican un alto costo de infraestructura adicional para su implementación. Por otro lado, los sensores RFID ofrecen una solución económica y flexible, aunque su alcance y precisión son menores en comparación con el resto de las tecnologías.

Tabla 4. Evaluación de sensores de detección de obstáculos

Sensor	Ventajas	Desventajas
LiDAR	Alta precisión en la detección y mapeo, gran alcance	Costo alto, sensible a la lluvia
Sensor de ultrasonido	Bajo costo, eficaces a corta distancia	Baja resolución, limitados a entornos cerrados
Cámaras estéreo 3D	Proporcionan imágenes detalladas, detectan formas y texturas	Requieren un costo alto de procesamiento computacional
Sensor IR	Compactos y económicos, rápida respuesta a corta distancia	Alcance muy limitado, sensibles a la luz ambiental

En la Tabla 4 se observa que en la selección de un sensor de detección de obstáculos, se debe considerar la precisión y el costo. El LiDAR ofrece alta precisión y alcance, pero

su costo es elevado y su rendimiento pudo verse afectado por condiciones climáticas adversas. Los sensores de ultrasonido presentan una alternativa económica y efectiva en distancias cortas, aunque con menor resolución. Las cámaras estéreo 3D permiten obtener una gran cantidad de información sobre el entorno, pero requieren procesamiento computacional intensivo. Finalmente, los sensores infrarrojos son una opción asequible y rápida, aunque su alcance es muy limitado y pueden ser sensibles a la iluminación del entorno.

Evaluación de algoritmos de planificación de trayectorias

Tras seleccionar los sensores apropiados para automatizar la navegación del robot remolcador, es fundamental evaluar distintos algoritmos de planificación de trayectorias. Estos algoritmos utilizarán los datos capturados por los sensores para diseñar una ruta segura y estable que permita el traslado eficiente de la avioneta, al mismo tiempo que evitan cualquier obstáculo que pueda surgir en su recorrido.

Tabla 5. Evaluación de algoritmos de planificación de trayectorias

Criterio	Dijkstra	RRT	A*	D* Lite
Adaptabilidad a cambios	Replanificación total	Replanificación total	Replanificación total	Replanificación parcial, solo nodos afectados
Eficiencia computacional	Baja	Media	Media	Muy alta
Facilidad de integración	Alta	Media	Alta	Baja
Precisión de trayectoria	Alta	Media	Alta	Alta
Capacidad de respuesta	Media	Baja	Media	Alta

De la tabla 5 se puede observar que la selección del algoritmo de planificación de trayectorias depende del entorno operativo específico y los requisitos del robot. Para entornos controlados y estáticos, A* ofrece un equilibrio entre eficiencia y velocidad de respuesta. En contraste, en escenarios donde el entorno es altamente impredecible y cambia activamente, el algoritmo D* *Lite* proporciona flexibilidad necesaria para adaptarse a los cambios en tiempo real. Por último, el algoritmo RRT resulta idóneo cuando el mapa del entorno es desconocido y es necesario explorar áreas desconocidas rápidamente.

3.6.3. Implementación de la simulación en Webots para evaluar una arquitectura del sistema de navegación

3.6.3.1. Creación del mundo en Webots

El desarrollo del mundo virtual en Webots constituye un paso crítico en la simulación del robot remolcador diseñado en esta tesis. Este mundo no solo permite validar los sistemas de control y los sensores, sino también observar el comportamiento dinámico del robot en condiciones que simulan escenarios operativos reales, como pistas de aterrizajes y hangares.

Diseño del modelo 3D del área de trabajo

Cuando se visitó el hangar de mantenimiento del Aeropuerto Internacional de Melilla “Ángel S. Adami” no se sacaron fotos ni se solicitaron diagramas de las instalaciones, ya que la pista de aterrizaje era asfaltada y lisa, con acceso a varios hangares de mantenimiento y estacionamiento. El área no poseía obstáculos aparentes ni desniveles. Por lo tanto, se decide crear un mundo en el simulador cercano a lo que se puede encontrar en cualquier aeropuerto y hangar de mantenimiento genérico.

La superficie de trabajo se diseña directamente en el software Webots. Sin embargo, para el hangar de mantenimiento, se optó por utilizar un modelo prediseñado en formato .stl para su importación en Webots que cumpliera con las similitudes de un hangar estándar, el cual se obtuvo de uamodeling3d (s.f.).

Diseño del modelo 3D de la avioneta y del robot remolcador

Dado que el diseño 3D de un avión podría presentar desafíos significativos en términos de tiempo y complejidad, se decidió adquirir un modelo prediseñado del Learjet 35/35A/36/36A en formato .stl, obteniéndolo de leha111dx (s.f.).

Para integrar el robot remolcador en la simulación, fue necesario diseñar un modelo 3D utilizando un software de diseño CAD, para luego importarlo al simulador Webots. Se seleccionó el software Autodesk Fusion 360 (Autodesk, s.f.) debido a su facilidad de uso, su capacidad para gestionar versiones en la nube y su funcionalidad de actualización dinámica en ensamblajes. Los planos de cada pieza se pueden encontrar en el Anexo B.

Para importar a Webots, es necesario guardar el modelo CAD en formato .stl con la base en el plano XY. Este modelo no debe contar con las ruedas, ya que éstas se deben colocar desde el simulador como atributo del objeto Robot, pues se definen sobre ellas las características de velocidad y carga del motor definidos en la Tabla 8.

Para la integración mecánica final del robot con el avión acoplado, fue necesario rediseñar el modelo 3D del robot remolcador, eliminando el acople de la rueda de la avioneta. Este acople se colocó directamente en el modelo de la avioneta, y se definió como un eje de rotación libre en el simulador Webots, acoplando de esta forma la avioneta al robot remolcador. El resultado de este ejercicio es asegurar una interacción realista entre ambos elementos.

3.6.3.2. Integración de los sensores al robot en Webots

Ya seleccionados los elementos de automatización en la sección 3.3.1.3, estos se deben integrar en el simulador. Una de las facilidades de Webots es que cuenta con una amplia librería de sensores, lo que nos permite seleccionar con facilidad los más adecuados para la simulación. Es así que se eligen los siguientes sensores:

- **Giroscopio:** se selecciona el nodo hijo “Compass” en Webots. Se coloca en la mitad de la parte frontal del chasis.
- **GPS:** se selecciona el nodo hijo “GPS” en Webots. Se coloca en la mitad de la parte frontal del chasis.
- **LiDAR:** se pueden encontrar varias opciones para sensores LiDAR. Se selecciona el modelo “Hokuyo UTM-30LX”, cuyas características son: posee un campo de visión de 270°, un rango de 0.1-30 m y una resolución de 1080*0.25°. Se coloca en la mitad de la parte frontal del chasis.
- **Display:** se selecciona el nodo hijo “Display” en Webots. No se

monta en el robot, sino que es una herramienta visual en la IDE de Webots para representaciones gráficas.

Por último, es necesario aclarar que los sensores no generan peso al robot y son ideales, es decir, no se obtiene ruido en la captura de datos del entorno, por lo que no es necesario generar ningún filtro de señales.

3.6.3.3. Configuración de parámetros de simulación

Para lograr una simulación fidedigna, es necesario configurar el simulador con los parámetros físicos del mundo. El simulador ya cuenta con varios parámetros configurados previamente en el nodo *WorldInfo*, entre ellos el de gravedad, configurado en 9.81, como se puede visualizar en la Figura 3.3.

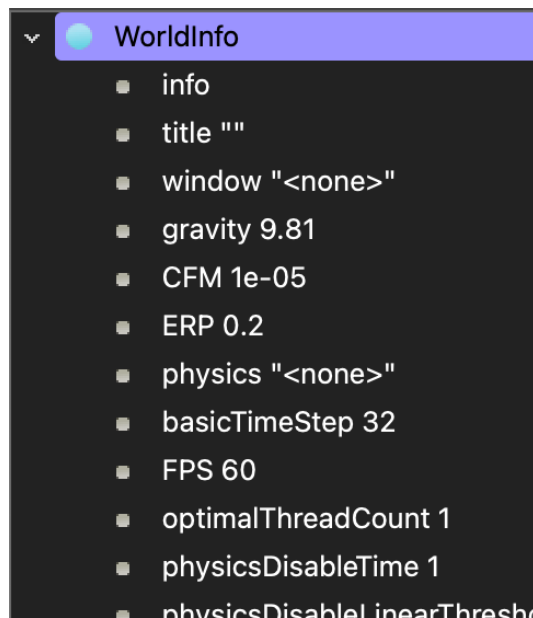


Figura 3.3. Nodo *WorldInfo* en software Webots

Además de los parámetros predefinidos por Webots, es necesario configurar el peso de los actores de la simulación, es decir, el robot remolcador y la avioneta. Este parámetro se configura en el nodo *physics* del robot y de la avioneta. Este nodo se puede visualizar en la Figura 3.4.

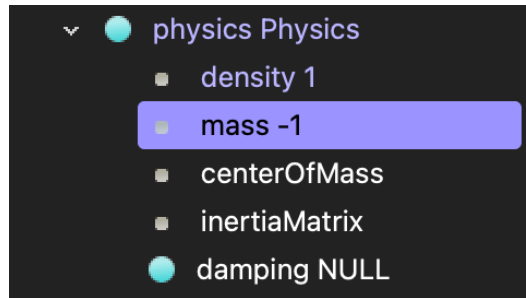


Figura 3.4. Nodo *physics* en software Webots

3.6.3.4. Implementación de arquitectura del sistema

Para la arquitectura del controlador del robot, se implementó una estructura basada en modularidad y reutilización de código, utilizando el lenguaje Python como núcleo, debido a su facilidad para manejar la lógica de control, manipular estructuras de datos y comunicarse de manera eficiente con el simulador Webots. A continuación, se detalla la arquitectura del controlador, el cual se encuentra disponible en su totalidad en el Anexo C.

Implementación del ejecutor de tareas

El ejecutor de tareas implementado en el controlador del robot es un árbol de comportamiento (BT) por medio de la librería `py_trees` para gestionar de manera modular y jerárquica las decisiones y acciones del robot. A continuación, se desglosan los elementos clave y el flujo de ejecución del BT, explicando cómo interactúan cada una de sus partes.

Estructura del árbol de comportamiento

El árbol de comportamiento se organiza de forma jerárquica comenzando con un nodo raíz y a partir de ahí se conectan sucesivamente sub-nodos. Cada nodo puede ser:

- Nodo compuesto, que a su vez tiene hijos y controla el flujo de ejecución, como son los tipos *Sequence*, *Selector* o *Parallel*.
- Nodo hoja, que ejecuta una acción o realiza una comprobación puntual, como mover el robot a una posición o verificar un sensor.

La ejecución del BT ocurre en *ticks* o ciclos de decisión donde se recorre el árbol de arriba abajo, evaluando y ejecutando cada nodo según sus reglas.

Elementos de un árbol de comportamiento

- **Nodo *Sequence*:** ejecuta sus hijos de izquierda a derecha, Si uno de sus hijos falla, *Sequence* deja de ejecutar los nodos siguientes y retorna FAILURE al nodo padre. En cambio, si todos los hijos se ejecutan con éxito, el nodo *Sequence* retorna SUCCESS al nodo padre. Este tipo de nodo se usa cuando se requieren acciones ordenadas. Se representa con una flecha en los diagramas.
- **Nodo *Selector*:** evalúa a sus hijos de izquierda a derecha, buscando alguno de ellos que devuelvan SUCCESS. Tan pronto como uno retorna SUCCESS, *Selector* se detiene y devuelve SUCCESS al nodo padre. Si todos fallan, el nodo devuelve FAILURE. Se utiliza cuando se quiere buscar una acción alternativa que funcione. Se representa con un signo de pregunta en los diagramas.
- **Nodo *Parallel*:** ejecuta todos sus hijos simultáneamente o de forma casi simultánea en cada *tick*. Se distinguen las siguientes dos políticas para el retorno de este nodo: SuccessOnAll, retorna SUCCESS sólo si todos los hijos retornan SUCCESS. SuccessOnOne, retorna SUCCESS apenas uno de los hijos complete con éxito la acción. Este tipo de nodos es muy útil para acciones concurrentes. Se representa con dos flechas paralelas en los diagramas.
- **Nodo *Decorators*:** son nodos especiales que modifican el comportamiento de un nodo hijo. Aunque en este proyecto no se implementan, se suelen usar en BT avanzados.
- ***Blackboard*:** es un espacio compartido de datos donde se pueden guardar y leer valores relevantes para distintos nodos. De este modo, un nodo que calcule un mapa puede almacenarlo en el *Blackboard* para que otro nodo lo consuma. Se representa con un cuadrado negro en los diagramas.

- **Propiedad *memory*:** existe una propiedad en los nodos *Sequence* y *Selector* que permiten conservar el estado de ejecución de sus hijos a lo largo de múltiples *ticks*. Es decir que no reiniciarán la evaluación de sus hijos desde el principio de cada ciclo, sino que recordarán en que etapa del proceso iban. Para ello, se debe seleccionar `memory=True` en estos nodos para activar esta propiedad.

Implementación de la navegación del robot

Este módulo integra tanto el control básico de los motores como una navegación autónoma basada en puntos de camino (*waypoints*), técnica que permite guiar al robot desde una posición inicial hasta una posición final por medio de un conjunto de puntos intermedios. A continuación, se describen los aspectos técnicos y metodológicos de la implementación.

Configuración de motores

El robot utiliza un modelo de tracción diferencial, donde las velocidades angulares de las ruedas de la izquierda y de la derecha operan de forma independiente, determinando tanto la traslación como la rotación en el plano. En la sección 3.6.2.1 sobre el análisis cinemático del robot, se calculan las fórmulas de cinemática directa que permiten convertir las velocidades de las ruedas en velocidad lineal y angular, mientras que las fórmulas de cinemática inversa permiten calcular las velocidades individuales necesarias para alcanzar cada punto de destino. En base a esto, se configuran los motores en cuatro puntos, dos delanteros y dos traseros, como `leftBackMotor`, `leftFrontMotor`, `rightLeftMotor` and `rightFrontMotor`. La configuración inicial incluye:

- `set_motor_positions`: configura los motores en modo de rotación continua para permitir movimientos sin restricciones angulares.
- `set_motor_velocities`: controla las velocidades angulares de los motores traseros y delanteros, permitiendo el movimiento lineal y rotacional.
- `configure_speed`: configura los parámetros p_1 y p_2 que son constantes de multiplicación para el cálculo de la velocidad del robot con respecto a la

distancia al *waypoint*.

Implementación de detección de obstáculos

Para el desarrollo de este módulo, se distinguen dos funciones a cumplir:

- **Generación de un mapa global:** creación de un mapa estático del entorno del hangar previo al traslado de la aeronave, que será utilizado para la localización del robot y la planificación de trayectorias.
- **Actualización dinámica del mapa global:** durante el traslado de la aeronave, los datos del sensor LiDAR se procesan en tiempo real para detectar obstáculos y actualizar el mapa, permitiendo ajustes en la ruta de navegación.

Representación del entorno

El entorno se representa mediante un mapa de ocupación en forma de una matriz 2D, donde cada celda indica el estado de ocupación del espacio correspondiente:

- Valores cercanos a 1 representan celdas ocupadas por obstáculos.
- Valores cercanos a 0 representan celdas libres.

Esta matriz es generada y actualizada de manera dinámica utilizando datos obtenidos por un sensor LiDAR y el sistema de localización.

Implementación del algoritmo de planificación de trayectorias

Para este módulo se implementa el algoritmo D* Lite, basándose en la optimización implementada por (Zhang, C. et al., 2023). Este opera sobre un grafo implícito derivado del mapa de ocupación. Los nodos representan las celdas del mapa y las aristas entre nodos representan posibles movimientos entre celdas adyacentes. Cada arista tiene un costo asociado basado en la distancia y la ocupación de las celdas.

3.6.4. Verificación de la simulación en el software Webots

Para evaluar la viabilidad y el desempeño del sistema diseñado, se llevaron a cabo ensayos específicos en el simulador Webots. Estos ensayos se desarrollaron siguiendo un

enfoque sistemático que incluye la descripción de los escenarios, la metodología empleada, los parámetros de configuración y los resultados obtenidos. Además, se realizó la toma de datos durante las simulaciones con el objetivo de validar cuantitativamente lo observado en el entorno virtual y garantizar la precisión de los análisis.

3.6.4.1. Control de motores sin carga

Objetivo

Validar la configuración y respuesta de los motores en maniobras básicas sin la influencia de carga adicional.

Procedimiento

Para este ensayo, se configuró un entorno libre de obstáculos y sin el avión acoplado al robot. Este ensayo se divide en las siguientes etapas:

- **Movimiento de avance:** se configuran la velocidad de las ruedas a 2 rad/s durante 4 segundos, seguido por una detención de motores de 1 segundo.
- **Movimiento de retroceso:** se configura la velocidad la velocidad de las ruedas a 2 rad/s durante 4 segundos, seguido por una detención de motores de 1 segundo.
- **Rotación hacia la izquierda:** se configuran las ruedas en sentidos opuestas con velocidad de 1 rad/s durante 4 segundos, seguido por una detención de motores de 1 segundo.
- **Rotación hacia la derecha:** se configuran las ruedas en sentidos opuestas con velocidad de 1 rad/s durante 4 segundos, seguido por una detención de motores de 1 segundo.
- **Prueba de velocidad máxima:** se configuran las ruedas a 5.445 rad/s durante 4 segundos, seguido por una detención de motores de 1 segundo.

El robot fue programado para realizar estas maniobras mediante el uso de un controlador en Python que envía comandos directos a los motores. Las velocidades de los motores se ajustaron utilizando la función `setVelocity()`, y las pruebas se llevaron a cabo con los motores configurados en modo sin restricciones de posición

`setPosition(float('inf'))`). En este ensayo se adquieren los datos de posición del robot a lo largo del tiempo con el fin de obtener gráficas de la trayectoria, velocidad lineal y velocidad angular del robot. El código completo del controlador se puede encontrar en el Anexo D.

3.6.4.2. Control de motores con carga

Objetivo

Se busca evaluar el desempeño de los motores del remolcador al ejecutar maniobras básicas mientras transporta la avioneta acoplada. El objetivo principal es analizar el impacto de la carga en la estabilidad, velocidad y respuesta del sistema.

Procedimiento

El ensayo se realizó en el simulador Webots utilizando un modelo de remolcador conectado a una avioneta (Learjet 35/35A/36/36A) mediante un nodo de acoplamiento. Se programaron maniobras básicas en el controlador, replicando las pruebas realizadas en el ensayo sin carga, pero adaptando las velocidades para garantizar estabilidad. Se divide este ensayo en las siguientes etapas:

1. **Movimiento de avance:** se configuran la velocidad de las ruedas a 2 rad/s durante 4 segundos, seguido por una detención de motores de 1 segundo.
2. **Movimiento de retroceso:** se configura la velocidad la velocidad de las ruedas a 2 rad/s durante 4 segundos, seguido por una detención de motores de 1 segundo.
3. **Rotación hacia la izquierda:** se configuran las ruedas en sentidos opuestas con velocidad de 1 rad/s durante 4 segundos, seguido por una detención de motores de 1 segundo.
4. **Rotación hacia la derecha:** se configuran las ruedas en sentidos opuestas con velocidad de 1 rad/s durante 4 segundos, seguido por una detención de motores de 1 segundo.
5. **Prueba de velocidad máxima:** se configuran las ruedas a 5.445 rad/s durante 4 segundos, seguido por una detención de motores de 1 segundo.

El entorno fue configurado libre de obstáculos para reflejar únicamente el impacto de

la carga. En este ensayo se adquieren los datos de posición del robot a lo largo del tiempo con el fin de obtener gráficas de la trayectoria, velocidad lineal y velocidad angular del robot. El código completo del controlador se puede encontrar en el Anexo D, ya que se usó el mismo script que para el ensayo de motores sin carga con el fin de poder evaluarlos correctamente.

3.6.4.3. Validación de mapeo

Objetivo

Se desea evaluar la capacidad del sistema para generar un mapa preciso del entorno utilizando datos del sensor LiDAR y procesar dichos datos para crear un mapa navegable. Este ensayo busca verificar la precisión del sistema de mapeo y su utilidad para la planificación de trayectorias.

Procedimiento

Para este ensayo, se configuró el simulador Webots con el área de trabajo en la que operará el robot remolcador, es decir, el área de $300 \times 300 \text{ m}^2$ con el hangar. Se coloca el robot en las coordenadas $(-130, -18)$, equipándolo con todos los sensores seleccionados en el apartado 3.3.2.1 y programándolo para realizar movimientos predefinidos que abarquen toda el área operativa. Cabe señalar que en el Display de visualización de Webots, se puede visualizar el mapa a medida que se genera, a la vez que también se visualiza una línea roja trazando el movimiento del robot.

El código de este ensayo se encuentra en el código del Anexo C, ya que se activa el mapeo del entorno de trabajo cuando no encuentra en su base de datos el mapa en formato .npy.

El sistema siguió los siguientes pasos:

1. El sensor LiDAR recolectó datos sobre la distancia a los obstáculos en el entorno.
2. Las lecturas fueron convertidas a un mapa de celdas de 300×300 píxeles, representando las zonas ocupadas por obstáculos.
3. Se aplicó un filtro de convolución al mapa generado para expandir los obstáculos detectados, definiendo un espacio navegable que garantiza

márgenes de seguridad.

Los resultados se almacenaron en un archivo de formato .npy y se visualizaron en tiempo real utilizando la herramienta Display de visualización del simulador Webots. El código completo del controlador se puede encontrar en el Anexo C.

3.6.4.4. Validación de planificación de trayectorias y traslado de avioneta con algoritmo D* Lite

En esta verificación, primero se realizó una prueba con un único obstáculo. Luego se realizó una segunda prueba sumando otro obstáculo. Y se fueron realizando varios ensayos más hasta conformar el ensayo que se presenta a continuación, evaluando todos los tipos de obstáculos posibles en el traslado de la avioneta.

Objetivo

Evaluar la capacidad del sistema para detectar obstáculos estáticos y evitarlos mientras transporta una avioneta acoplada, utilizando el algoritmo de planificación de trayectorias D* Lite. El ensayo también busca analizar la eficacia del algoritmo en la generación de rutas óptimas y su capacidad de replanificación dinámica frente a obstáculos detectados. Además, se busca ajustar los parámetros p_1 y p_2 del controlador P de los motores.

Procedimiento

Para este ensayo, se configuró el entorno de operación del robot remolcador en el entorno de simulación Webots que incluye la carga acoplada (la avioneta Learjet 35/35A/36/36A), una serie de obstáculos estáticos en la forma de un muro de 12 m de largo, 3 de alto y un grosor de 0.2 m, en conjunto con tres bloques de $1 \times 1 \times 1 \text{ m}^3$, esparcidos de forma aleatoria a lo largo de la trayectoria al hangar, donde el robot depositará la avioneta. Se definen un punto de inicio con coordenadas (35,-10) y un punto final con coordenadas (-128,5).

Luego, se realizaron 4 ensayos con distintas configuraciones de p_1 y p_2 , presentadas a continuación:

- Configuración 1: $p_1 = 1, p_2 = 1$

- Configuración 2: $p_1 = 1, p_2 = 0.2$
- Configuración 3: $p_1 = 0.5, p_2 = 0.5$
- Configuración 4: $p_1 = 0.5, p_2 = 0.2$

En la Figura 3.5. se puede visualizar la configuración del entorno con los obstáculos colocados en el camino del remolcador.

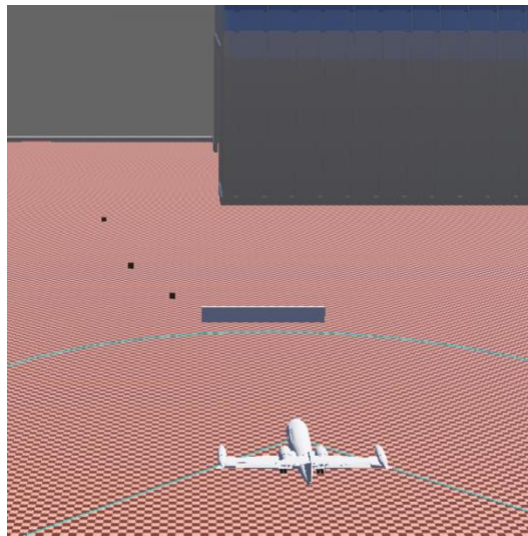


Figura 3.5. Ensayo de algoritmo *D* Lite*

En este ensayo se adquieren los datos de posición del robot a lo largo del tiempo con el fin de obtener gráficas de velocidad lineal, trayectoria y distancia al destino. El código completo del controlador se puede encontrar en el Anexo C.

CAPÍTULO 4. RESULTADOS Y ANÁLISIS

4.1. RESULTADOS DEL ESTABLECIMIENTO DEL DISEÑO CONCEPTUAL DEL ROBOT REMOLCADOR PARA PREPARAR LOS PARÁMETROS DE LA SIMULACIÓN

4.1.1. Identificación de requisitos

De los datos recopilados en la encuesta, presentados en el Anexo A, se destaca que el 80% de los participantes considera que el traslado de avionetas es una de las tareas más exigentes en términos de esfuerzo físico. Asimismo, el 70% de los encuestados indicó que el remolque de aeronaves dentro del hangar es un proceso que requiere mejoras en términos de seguridad y eficiencia. Esta percepción refuerza la justificación para la implementación de una solución robótica automatizada.

Otro aspecto relevante es que los participantes identificaron el remolque de aviones como una de las tareas con alto potencial de automatización, junto con el transporte de equipos pesados y la inspección estructural de aeronaves. Estas respuestas refuerzan la idea de que una solución robótica automatizada no solo sería bienvenida, sino que también responde a una necesidad real del sector.

En la discusión del equipo multidisciplinario del análisis de las respuestas de la encuesta se identificó a la avioneta Learjet 35/35A/36/36A como la de mayor envergadura y esfuerzo de traslado. Además, se identifica el área de trabajo y los requisitos operativos del robot, los cuales se encuentran en la Tabla 6. Luego, se realizó un estudio del manual de mantenimiento de la aeronave (Learjet Inc., 2012), identificando las características principales del robot a trasladar, presentes en la Tabla 7.

Tabla 6. Requisitos operativos de área de trabajo

Requisitos operativos del área de trabajo	
1	Remolque de avioneta de forma autónoma
2	El equipo opera sobre superficie lista, en terreno asfaltado
3	No se dispone de planos del área de operación

4	El acople de ruedas debe adaptarse a cualquier tipo de dimensiones
5	El empuje de la avioneta debe realizarse desde la rueda delantera

Tabla 7. Requisitos técnicos Learjet35 (Learjet 35/35A/36/36A Maintenance Manual, 2012)

Especificaciones de avioneta Learjet 35/35A/36/36A		
1	Peso en vacío	4590 kg
2	Longitud	14.8 m
3	Envergadura	12 m
4	Altura	3.7 m

El estudio del estado del arte no solo permitió validar el estudio de esta solución, sino que permitió identificar el proyecto de Freire y Rubio (2016) como la solución con la que continuar el trabajo de este proyecto. Las especificaciones técnicas de este robot se encuentran en la Tabla 8.

Tabla 8. Características del robot diseñado por Freire y Rubio (2016)

Características del robot de Freire y Rubio (2016)	
1	Dimensiones: 830 mm de largo, 1016 mm de ancho y 254 mm de alto
2	Motores DC de 12/24 V
3	Estructura de acero estructural grado A
4	Baja necesidad de mantenimiento
5	Facilidad de operación

6	Control remoto ergonómico
7	Seguro térmico para alta tensión
8	Capacidad de carga máximo 5000 kg
9	Protección anticorrosiva
10	Ruedas de hierro fundido con cubierta de caucho y de fácil reemplazo
11	El acople de rueda se adapta a cualquier dimensión
12	El direccionamiento del remolcador es diferencial
13	Velocidad máxima de remolque 3 Km/h (0.83m/s)
14	Batería para driver de potencia: 12 V, 40 Amph
15	Batería para sistema de control: 12 V, de ácido
16	Batería para control remoto: 1.5 V, de ion-litio
17	Potencia máxima: 1.9 kW

Por lo tanto, se pueden identificar los siguientes requisitos para la simulación de la automatización del traslado de avionetas:

- Traslado de avioneta Learjet 35/35A/36/36A con las características presentadas en la Tabla 7.
- Terreno asfaltado liso, sin irregularidades. De dimensiones de 300x300 m^2 .
- Traslado de la pista de aterrizaje a un hangar de mantenimiento.
- Realizar el traslado con el robot desarrollado por Freire y Rubio (2016).

- El traslado debe realizarse por el robot remolcador de forma autónoma.

4.2. RESULTADOS DEL ANÁLISIS DEL MODELO CINEMÁTICO Y DE NAVEGACIÓN DEL ROBOT REMOLCADOR PARA AVIONETAS

4.2.1. Estudio del modelo cinemático

Velocidad máxima del robot

El análisis cinemático permitió modelar el comportamiento del robot remolcador en función de sus grados de libertad y sistemas de locomoción. Se estableció que el robot posee una configuración de tracción diferencial con dos partes de ruedas motrices, lo que le otorga la capacidad de realizar desplazamientos lineales y rotación sobre su propio eje.

Mediante la formulación de ecuaciones cinemáticas, se determinó la relación entre la velocidad angular de las ruedas y la trayectoria del robot. Se validó que la estrategia de control propuesta permite realizar movimientos precisos en espacios restringidos, garantizando la capacidad de posicionamiento adecuado. Este análisis sirvió de base para el diseño del sistema de navegación.

Por las características del robot presentadas en la Tabla 8, sabemos que la velocidad máxima del robot es de 0.83 m/s, por lo tanto, podemos obtener la velocidad de rotación de las ruedas, considerando el caso de mayor velocidad que es cuando ambas ruedas giran en el mismo sentido, ya sea avanzando o retrocediendo, en línea recta, podemos obtener la velocidad angular máxima de las ruedas del robot:

$$\bullet \quad v = \frac{r}{2}(w_R + w_L) \rightarrow v = \frac{r \cdot 2w}{2} \rightarrow w = \frac{v}{r}$$

En este caso, el radio de las ruedas es de 0.1524 m (Freire & Rubio, 2016). Por lo tanto, la velocidad angular máxima de las ruedas es:

$$\bullet \quad w = \frac{0.83}{0.1524} = 5.446 \text{ rad/s}$$

Se utiliza este valor para definir la velocidad de las ruedas y del motor, se toman como uno solo en el software Webots.

Análisis de grados de libertad del robot

El análisis de los grados de libertad (DOF) del robot es fundamental para comprender su capacidad de movimiento en el entorno. Los grados de libertad representan el número de variables independientes necesarias para definir la posición y orientación del robot en el espacio.

Dado que el robot cuenta con un sistema de tracción diferencial por pares de ruedas, donde las ruedas del lado derecho giran solidariamente con una velocidad angular w_R y las del lado izquierdo con velocidad angular w_L , se identifican los siguientes movimientos:

1. Movimiento lineal de avance o retroceso cuando $w_R = w_L$.
2. Rotación sobre su eje central de rotación cuando $w_R \neq w_L$, especialmente cuando $w_R = -w_L$.

Por lo tanto, el robot posee dos grados de libertad, presentados en la Figura 4.1:

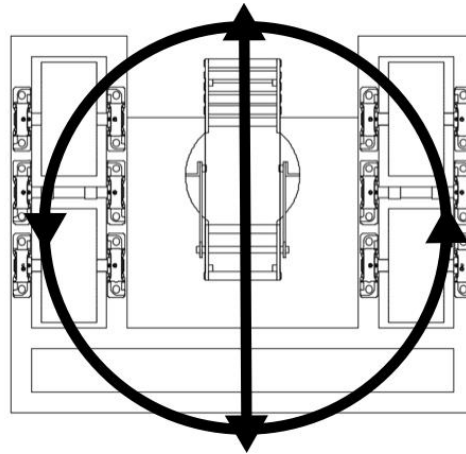


Figura 4.1. Grados de libertad del robot remolcador

- **DOF 1:** traslación en el plano XY , movimiento de avance y retroceso.
- **DOF 2:** rotación alrededor de su eje vertical Z , cambio de orientación θ .

Esta configuración le permite realizar desplazamientos y rotaciones combinados para alcanzar cualquier orientación y posición en el plano. Sin embargo, al no contar con ruedas omnidireccionales, el robot es un sistema no holonómico, es decir, no puede desplazarse lateralmente de forma directa.

Gracias a la combinación de estos dos grados de libertad, el robot es capaz de ejecutar

hasta ocho tipos de movimientos distintos, que integran desplazamientos rectos y rotaciones, los cuales se presentan a continuación:

1. **Avance:** cuando ambos pares de ruedas giran en el mismo sentido y con la misma velocidad $w_R = w_L > 0$, el robot avanza en línea recta, como se presenta en la Figura 4.2 (izq).
2. **Retroceso:** si ambos pares de ruedas giran en el mismo sentido a la misma velocidad negativa $w_R = w_L < 0$, el robot retrocede en línea recta como se presenta en la Figura 4.2 (der).

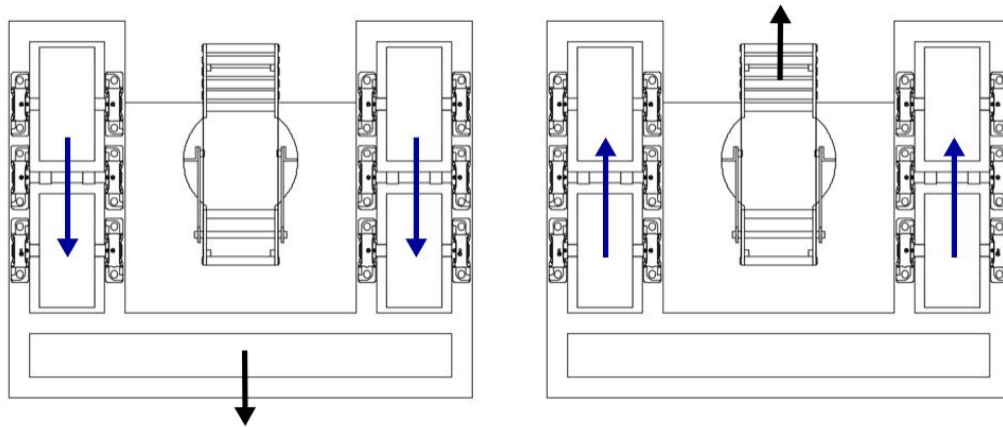


Figura 4.2. Movimiento de avance (izq) y retroceso (der)

3. **Rotación sobre su eje sentido anti-horario:** si las ruedas giran en direcciones opuestas con la rueda derecha avanzando y la izquierda retrocediendo ($w_R > 0$ y $w_L < 0$), el robot gira sobre su eje en sentido horario como se muestra en la Figura 4.3 (izq).
4. **Rotación sobre su eje sentido horario:** si la rueda derecha retrocede $w_R < 0$ y la izquierda avanza $w_L > 0$, el robot gira en sentido horario como se muestra en la Figura 4.3 (der).

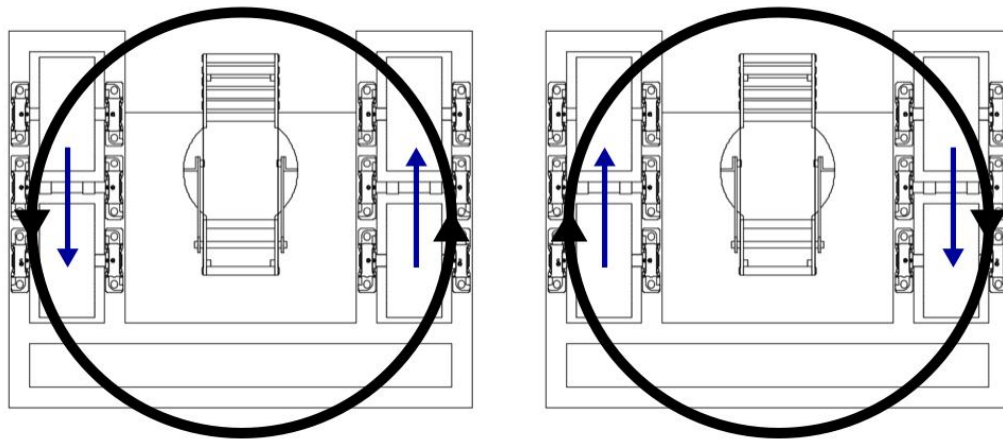


Figura 4.3. Rotación sobre su eje sentido anti-horario (izq.) y horario (der.)

5. **Avance con curva hacia la derecha:** si ambas ruedas giran en la misma dirección hacia adelante, pero la rueda izquierda lo hace más rápido que la derecha $w_R < w_L$, el robot avanza describiendo una curva hacia la derecha como se muestra en la Figura 4.4 (sup. izq.).
6. **Avance con curva hacia la izquierda:** si ambas ruedas giran en la misma dirección hacia adelante, pero la rueda derecha gira más rápido que la izquierda $w_R > w_L$, el robot avanza describiendo una curva hacia la izquierda como se muestra en la Figura 4.4 (sup. der.).
7. **Retroceso con curva hacia la derecha:** si ambas ruedas giran en la misma dirección hacia atrás, pero la rueda izquierda gira más rápido que la derecha $-w_R < -w_L$, el robot retrocede describiendo una curva hacia la derecha como se muestra en la Figura 4.4 (inf. izq.).
8. **Retroceso con curva hacia la izquierda:** si ambas ruedas giran en la misma dirección hacia atrás, pero la rueda derecha gira más rápido que la izquierda $-w_R > -w_L$, el robot retrocede describiendo una curva hacia la izquierda como se muestra en la Figura 4.4 (inf. der.).

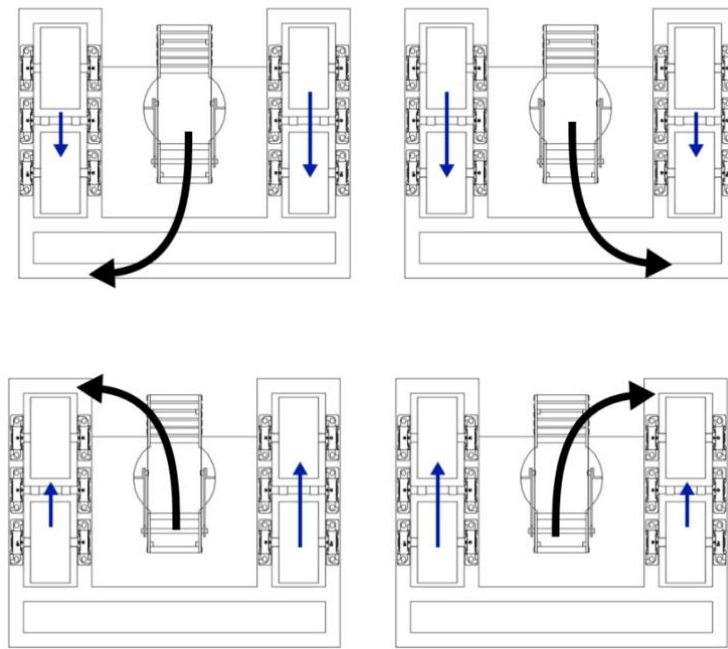


Figura 4.4. Avance con curva hacia la derecha (sup. izq.), avance con curva hacia la izquierda (sup. der.), retroceso con curva hacia la derecha (inf. izq.) y retroceso con curva hacia la izquierda (inf. der.)

De esta forma, se describen todos los movimientos posibles del diseño del remolcador de avionetas.

4.2.2. Estudio de navegación del robot remolcador

Selección del sistema de navegación

Dado que el robot remolcador operará tanto en un entorno controlado como en espacios abiertos con variabilidad, se considera que un enfoque de navegación híbrida es la mejor solución.

Selección de sensor de orientación

La elección de un giroscopio como sensor principal de orientación se fundamenta en su capacidad para proporcionar información rápida y precisa sobre los cambios de orientación, lo cual es esencial para el desplazamiento y las maniobras del robot remolcador. Además, su integración es sencilla, su costo es bajo y cumple eficientemente con los requisitos del proyecto.

Selección de sensor de localización

Se seleccionó el sistema GPS porque ofrece una cobertura total en exteriores, como pistas de aterrizaje, donde el robot realizará gran parte de sus maniobras. Comparado con otros sistemas más avanzados, el GPS ofrece un costo más accesible y fácil de integrar al sistema de control del robot. Por último, es necesario destacar que la precisión que ofrece el GPS de ± 1 m es suficiente para el robot, ya que las maniobras del robot se realizarán dentro de un área de $300 \times 300 \text{ m}^2$.

Selección de sensor de detección de obstáculos

Para cumplir esta función, se selecciona el sensor LiDAR ya que proporciona un largo alcance de detección detallado, elemento esencial en el transporte de avionetas, ámbito donde la seguridad es prioritaria. La detección anticipada de obstáculos permite generar maniobras seguras, protegiendo tanto al equipo como a la nave y los posibles obstáculos detectados. A su vez, se considera en la toma de datos generar un área de seguridad de 6 m alrededor de cada objeto detectado.

Selección del algoritmo de planificación de trayectorias

El algoritmo D* *Lite* sobresale en adaptabilidad a cambios y eficiencia computacional, características cruciales dado que el entorno puede variar debido al movimiento de otras avionetas o equipos. Su eficiencia computacional asegura que el robot puede recalcular trayectorias en tiempo real sin retrasos significativos, lo que es vital para evitar colisiones. La precisión de trayectoria y su capacidad de respuesta son superiores en comparación con otros algoritmos. Estas características garantizan que el robot pueda navegar de manera eficaz y segura, ajustándose rápidamente a los cambios dinámicos del entorno. Sin embargo, ofrece una baja facilidad de integración, pero es un costo a asumir para asegurar calidad en el traslado de avionetas.

4.3. RESULTADOS DE LA IMPLEMENTACIÓN DE LA SIMULACIÓN EN WEBOTS PARA EVALUAR UNA ARQUITECTURA DEL SISTEMA DE NAVEGACIÓN

4.3.1. Creación del mundo en Webots

Modelo 3D del área de trabajo

Como se definió en los requisitos de la sección 4.1.1, se eligió un área de trabajo de

300x300 m², con el fin de aproximarse a una posible área real de operación del robot. Su diseño se realizó directamente en el simulador Webots con una superficie lisa, color parquet para facilitar a la visualización del movimiento del robot.

El modelo adquirido del hangar (uamodeling3d, s.f.) posee unas dimensiones de 180 m de longitud por 87 m de ancho y 52 m de altura. Dicha instalación se posiciona en la esquina superior izquierda dándole un color metálico. La composición del área de trabajo con el hangar incorporado en Webots se visualiza en la Figura 4.5:

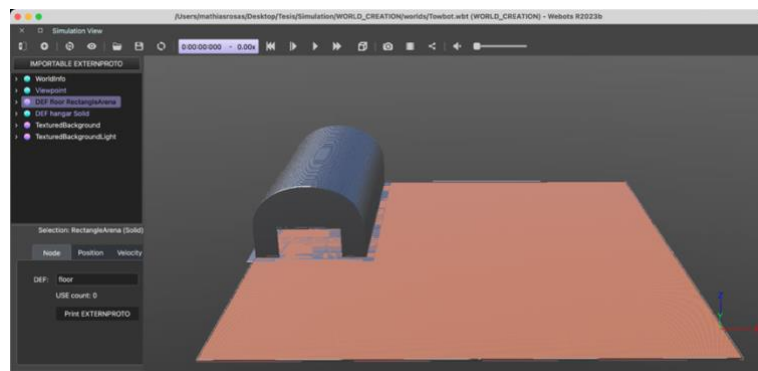


Figura 4.5. Área de trabajo con hangar en simulador Webot

Modelo 3D de la avioneta y el robot remolcador

El modelo adquirido de la avioneta Learjet 35/35A/36/36A (leha111dx, s.f.) cuenta con dimensiones realistas: longitud de 14.8 m, envergadura de 12 m y altura de 3.7 m. En la Figura 4.6, se muestra el modelo del avión, de color blanco, integrado al entorno de simulación y estacionado frente al hangar de mantenimiento.

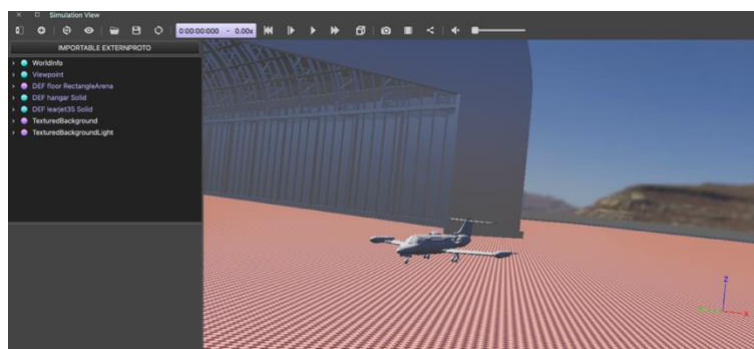


Figura 4.6. Avión en simulador Webots

En la Figura 4.6 se puede observar la integración del robot en el entorno de simulación,

donde el chasis se denota en rojo y las ruedas implementadas desde Webots en negro.

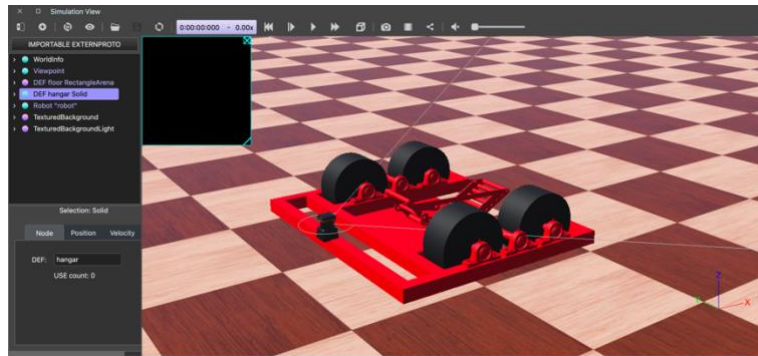


Figura 4.6. Robot remolcador en simulador Webots

El diseño final del robot remolcador con la avioneta acoplada en el simulador Webots se muestra en la Figura 4.7:

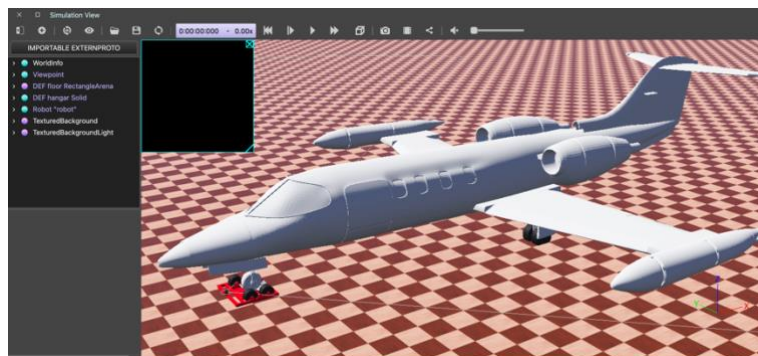


Figura 4.7. Acoplamiento del avión al robot remolcador en simulador Webots

4.3.2. Integración de los sensores al robot en Webots

El GPS y el giroscopio no tienen representación física en el simulador. Sin embargo, el sensor LiDAR sí posee una representación física, la cual se puede ver en la Figura 4.8. Todos los sensores se colocan en el centro de la parte frontal del robot. El *display* se puede ver como un cuadrado negro con borde de color cyan en la Figura 4.8, al igual que los nodos de los sensores virtuales en la sección izquierda como hijos del nodo Robot.

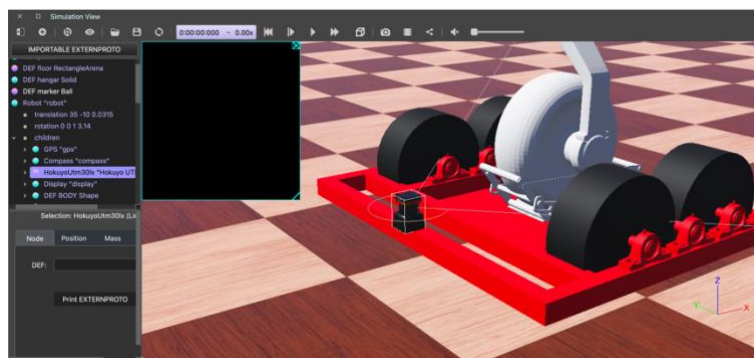


Figura 4.8. Integración de los sensores al robot remolcador en software Webots

4.3.3. Configuración de parámetros de simulación

Primero se configuró el peso del robot remolcador, parámetro del que no se tiene información en el proyecto de Freire y Rubio, por lo cual se decidió otorgarle una masa de 50kg, situando su centro de masa en el punto de acople de la rueda de la avioneta. Se verificó su movilidad sin carga y no se detectaron diferencias con la configuración de este parámetro en el robot remolcador.

Sin embargo, cuando se configuró el peso de la avioneta según la información recogida en la Tabla 7, donde su centro de masa se posiciona en el centro geométrico de la aeronave (Freire y Rubio, 2016), se genera un error en el software Webots, el cual se puede visualizar en la Figura 4.9, donde el robot remolcador y la avioneta se hunden en el piso, dejando inoperante la simulación.

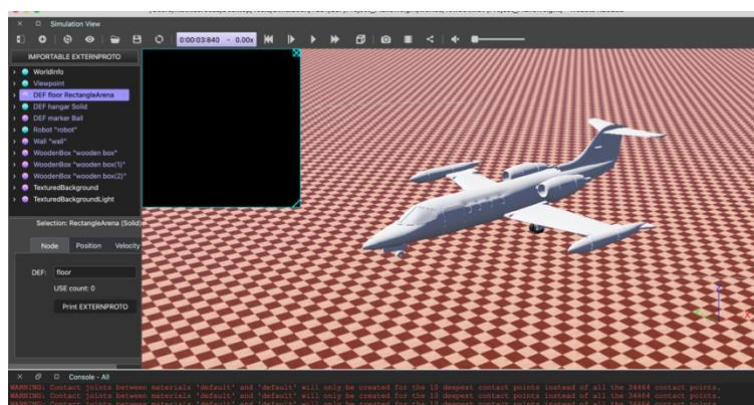


Figura 4.9. Fallo del simulador Webots

Dado este problema en la simulación y considerando que en la Tabla 8 se informa de los parámetros de operación con carga máxima, se decidió eliminar el parámetro *physics*

en el remolcador y la avioneta, con el fin de priorizar el estudio del algoritmo de planificación de trayectorias con los motores operando a la velocidad de carga máxima de la Tabla 8.

4.3.4. Implementación de arquitectura del sistema

En el Anexo C se puede encontrar la implementación completa de la arquitectura desarrollada.

Implementación del ejecutor de tareas

El árbol de comportamiento comienza con un nodo raíz del tipo *Sequence* con dos hijos: nodo *Selector* llamado “Existe el mapa?” y nodo de la función Planning llamado “Trasladar avioneta al hangar”, con la propiedad de memoria activada.

El nodo *Selector* “Existe el mapa?” comprueba si existe un mapa del entorno de trabajo, si existe, se carga el mapa desde disco y se guarda en un *Blackboard* predefinido (clave ‘map’), retornando SUCCESS y continuando en el nodo de la función Planning “Trasladar avioneta al hangar”. Este nodo se encarga de la planificación de trayectoria y traslado de la avioneta al hangar. Si no existe el mapa, retorna FAILURE y ejecuta su nodo hijo del tipo *Parallel* “Mapeo”.

El nodo *Parallel* “Mapeo” tiene dos hijos: el nodo de la función Mapping “Mapear el entorno”, el cual se encarga de detectar obstáculos y crear un mapa, y el nodo de la función Navigation “Mover alrededor del aeródromo”, que se encarga de generar *waypoints* que el robot deberá recorrer para moverse alrededor del hangar para realizar el mapeo del entorno. El nodo padre “Mapeo” tiene una política de éxito del tipo SuccessOnOne y, por lo tanto, se considera alcanzado el estado de SUCCESS de cualquiera de sus dos hijos.

En la Figura 4.10. se representa el diagrama del comportamiento del robot:

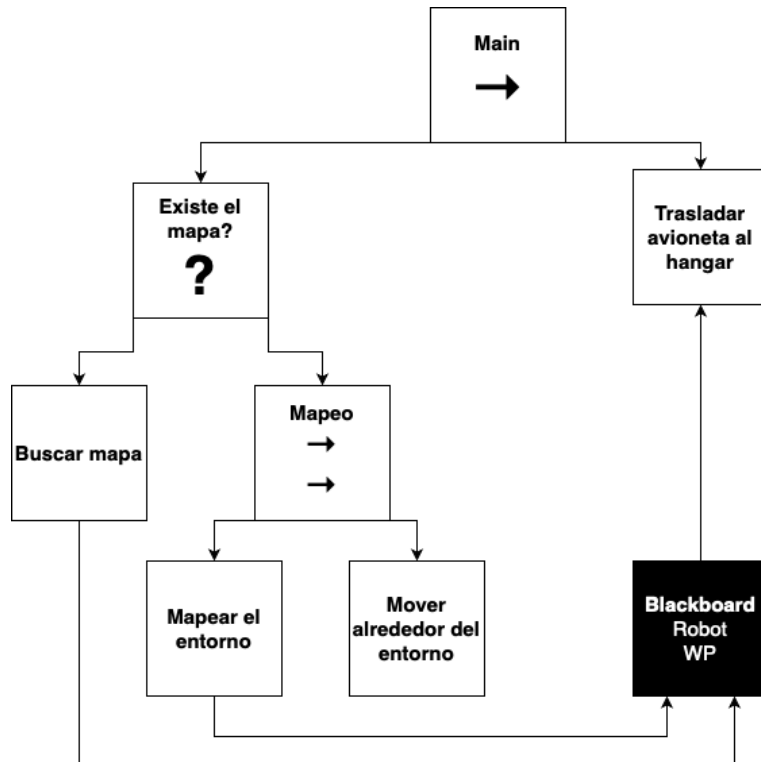


Figura 4.10. Diagrama de árbol de comportamiento del robot

Implementación de la navegación del robot

Los pasos principales de este módulo son los siguientes:

1. **Definición de waypoints:** se establecen como coordenadas cartesianas (x, y) en el entorno de trabajo del robot.
2. **Cálculo de distancias y ángulos:** ρ , distancia euclidiana entre la posición actual del robot y el siguiente waypoint; α , ángulo entre la orientación actual del robot y la línea recta hacia el waypoint. Estos valores se actualizan continuamente utilizando los datos obtenidos por el GPS y el giroscopio del robot.
3. **Control de motores:** las velocidades de las ruedas derecha ($\dot{\phi}_R$) e izquierda ($\dot{\phi}_L$) se calculan mediante:

- $\dot{\phi}_R = \alpha \cdot p_1 + \rho \cdot p_2$

- $\dot{\phi}_L = -\alpha \cdot p_1 + \rho \cdot p_2$

- Donde p_1 y p_2 son los parámetros de control obtenidos de `configure_speed`

que ajustan la respuesta angular y linear, respectivamente de los motores, α y ρ son el ángulo de orientación y la distancia del robot al siguiente *waypoint* respectivamente.

4. **Evaluación de progreso:** si $\rho < 0.3$ m, el *waypoint* se considera alcanzado, y el robot avanza al siguiente. Este proceso se repite hasta que todos los *waypoints* han sido visitados.

El diagrama de flujo de la Figura 4.11 describe el movimiento del robot:

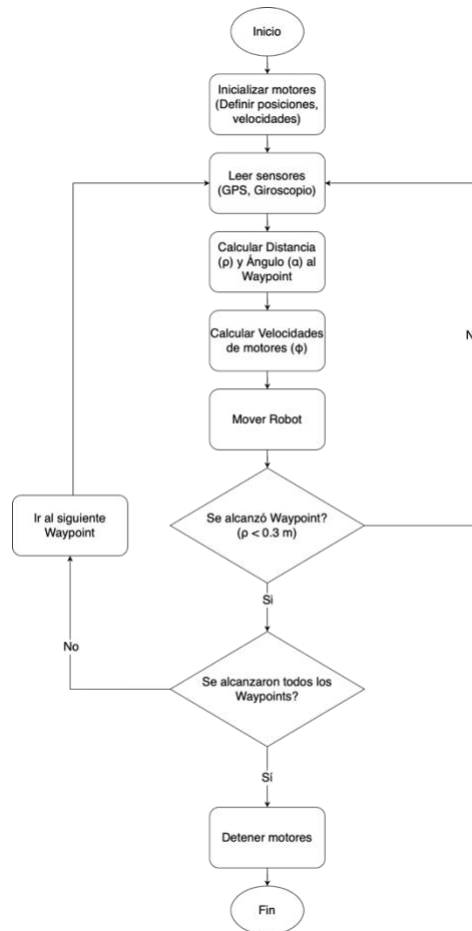


Figura 4.11. Diagrama de flujo de la navegación del robot

Implementación de detección de obstáculos

El proceso de detección de obstáculos se basa en la lectura de datos desde un sensor LiDAR (Hokuyo UTM-30LX). Estos datos se utilizan para actualizar un mapa de ocupación, representado como una matriz bidimensional (conocida como *occupancy grid*). A continuación, se destacan las etapas principales de este módulo:

- **Lectura de datos desde LiDAR:** se obtienen las distancias medidas a lo largo de diferentes ángulos.
- **Filtrado y ajuste de rangos:** se descartan o limitan los valores medidos que estén fuera de un rango de interés (mayores a 12 m).
- **Transformación de coordenadas:** los datos se convierten de un marco de referencia del robot al marco de referencia global.
- **Proyección del mapa:** se mapean dichos puntos a índices matriciales (x, y) en la rejilla del entorno.
- **Generación de un mapa de colisión (cspace):** se aplica una convolución para representar las áreas ocupadas por los obstáculos con un cierto margen de seguridad.
- **Visualización y almacenamiento:** el mapa resultante se visualiza para ver la distribución de obstáculos, y se almacenan los datos si se desea emplearlos en módulos posteriores.

Implementación del algoritmo de planificación de trayectorias

La implementación de este módulo se detalla en los siguientes pasos:

1. **Inicialización:** el algoritmo se inicializa con la posición inicial del robot (nodo de inicio) y la ubicación deseada (nodo objetivo). Se genera una estructura de datos de prioridad para manejar los nodos por explorar, utilizando un árbol de prioridades.
2. **Generación del camino inicial:** se calcula un camino óptimo inicial desde el nodo de inicio hasta el nodo objetivo utilizando una versión modificada del algoritmo D* Lite (Zhang et al., 2023). Este camino se almacena como una lista de coordenadas que representan las celdas a seguir.
3. **Adaptación dinámica:** durante la ejecución, el algoritmo verifica continuamente el entorno mediante datos del sensor. Si se detectan nuevos obstáculos o cambios en el mapa, se recalculan los costos de las aristas afectadas. El algoritmo replantea las partes del camino afectado utilizando la

información actualizada, lo que permite al robot adaptarse a condiciones cambiantes en tiempo real.

4. **Seguimiento de trayectoria:** el sistema de control utiliza la trayectoria calculada para generar comandos motores que guían al remolcador hacia el nodo objetivo. A medida que el robot se aproxima a cada nodo intermedio, verifica su progreso y recalcula el camino si es necesario.

En la Figura 4.12 se puede visualizar el diagrama de flujo del algoritmo:

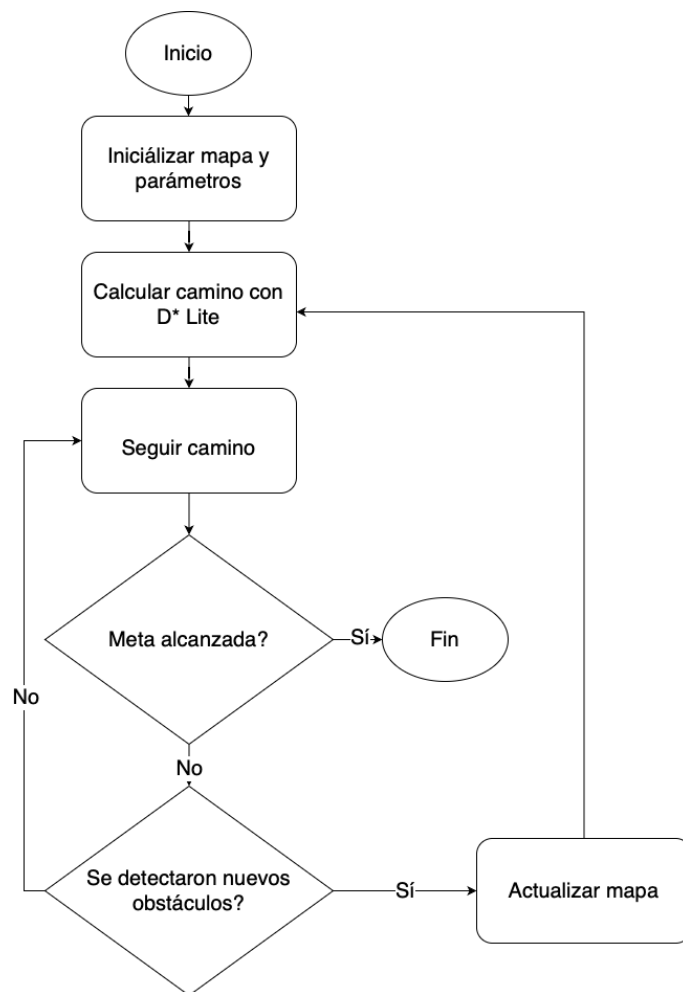


Figura 4.12. Planificación de trayectorias con algoritmo D* Lite

4.4. RESULTADOS DE LA VERIFICACIÓN DE LA SIMULACIÓN EN EL SOFTWARE WEBOTS

4.4.1. Control de motores sin carga

Los datos recopilados durante el ensayo de la sección 3.6.4.1 proporcionan una visión clara del comportamiento del robot durante las maniobras básicas ejecutadas en el simulador.

Análisis de velocidad lineal

La gráfica de velocidad lineal (Figura 4.13) demuestra claramente cada una de las etapas del ensayo.

- **Movimiento de avance:** el robot mantiene velocidad constante y se detiene durante 1 segundo.
- **Movimiento de retroceso:** el robot mantiene velocidad constante y se detiene durante 1 segundo.
- **Rotación hacia la izquierda:** en esta etapa se realiza un movimiento angular, aun así, se registra algo de velocidad lineal, teniendo un deslizamiento al terminar la rotación.
- **Rotación hacia la derecha:** se vuelve a realizar otra rotación en el sentido opuesto, teniendo un desliz considerable al comienzo para luego mantenerse constante.
- **Prueba de velocidad máxima:** en esta etapa se esperaba llegar a velocidades cercanas a 0.83 m/s, y en la gráfica se observa que el robot posee una velocidad cercana a 0.7 m/s.

Los resultados obtenidos en el ensayo de control de motores sin carga confirman que el robot responde de manera adecuada a las maniobras programadas, mostrando una estabilidad general en la ejecución de los movimientos. La gráfica de velocidad lineal evidencia la correcta diferenciación entre las cinco etapas del ensayo: avance (1), retroceso (2), rotaciones (3 y 4) y prueba de velocidad máxima (5). Se observa que, en los movimientos de avance y retroceso (1 y 2), la velocidad del robot se mantiene relativamente estable en torno a 0.25 m/s, con fluctuaciones menores atribuibles a la tracción y ajuste natural del sistema.

Durante las maniobras de rotación (3 y 4), se registra una reducción significativa en la velocidad lineal, lo que sugiere que el robot disminuye su velocidad mientras ejecuta

cambios de dirección, probablemente debido al efecto de deslizamiento y la distribución de fuerzas en las ruedas. Este comportamiento es consistente con el esperado en un sistema de tracción diferencial sin asistencia de control adicional para la corrección de trayectoria.

En la prueba de velocidad máxima (5), el robot logra alcanzar valores cercanos a 0.7 m/s, inferiores a los 0.83 m/s esperados, lo que indica una ligera limitación en la eficiencia de transmisión de potencia a las ruedas. Además, una vez alcanzado el régimen máximo se evidencia una estabilidad en la velocidad sin oscilaciones abruptas, lo que confirma un desempeño adecuado del sistema de motorización en esta condición.

Finalmente, la detección de caídas bruscas en la velocidad en los períodos de detención entre cada fase valida la correcta implementación del control de motores y la respuesta precisa a los comandos enviados. En general, los resultados obtenidos respaldan el correcto funcionamiento del robot en maniobras básicas sin carga y proporcionan una referencia de su comportamiento en condiciones ideales antes de evaluar el impacto de la carga en su desempeño.

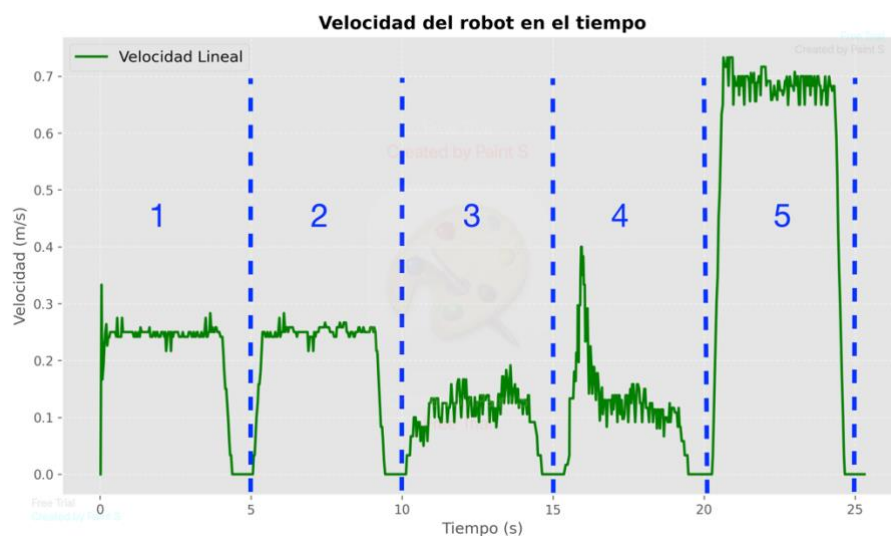


Figura 4.13. Gráfica de velocidad del robot en el tiempo en el ensayo de motores sin carga

Análisis de trayectoria

La gráfica de la trayectoria registrada (Figura 4.14) confirma que el desplazamiento del robot se ajusta a los comandos enviados, evidenciando una ejecución estructurada y predecible. Inicialmente, el robot parte desde la posición 0 (Inicio) y avanza hasta la posición 1. A continuación, ejecuta un retroceso preciso hasta la posición 2, coincidiendo

con su punto de partida.

Posteriormente, se observan maniobras de rotación hacia la izquierda y derecha, lo que indica una evaluación de la capacidad de rotación del sistema de control. Luego, el robot realiza un desplazamiento de avance a mayor velocidad, alcanzando una distancia superior en la posición 5. En esta última etapa, se detecta una leve desviación en la trayectoria, atribuida a un deslizamiento mínimo, lo que sugiere la influencia de factores externos como la fricción con la superficie o pequeñas variaciones en la respuesta de los motores.



Figura 4.14. Gráfica de trayectoria del robot en el tiempo en el ensayo de motores sin carga

En general, los resultados reflejan un desempeño estable del sistema de navegación sin carga, con una respuesta adecuada a los comandos programados y una trayectoria que, salvo el pequeño deslizamiento, se mantiene dentro de los márgenes esperados.

4.4.2. Control de motores con carga

Los datos obtenidos del ensayo de la sección 3.6.4.2 muestran cómo la adición de la avioneta afecta al desempeño del robot en cada una de las maniobras programadas.

Análisis de velocidad lineal

No se observa un cambio destacable en la velocidad del robot (Figura 4.15) en comparación con el ensayo sin carga. Durante los movimientos de avance y retroceso (1 y 2), se observaron menos oscilaciones bruscas, al igual que durante el período de rotación (3 y 4) y velocidad máxima.

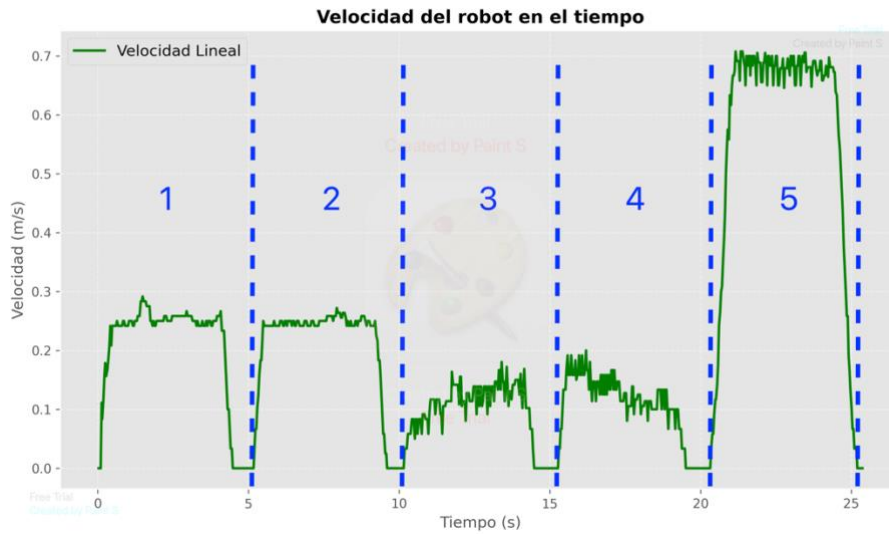


Figura 4.15. Gráfica de velocidad del robot en el tiempo en el ensayo de motores con carga

Análisis de trayectoria

La trayectoria registrada (Figura 4.16) confirma que el comportamiento del robot cambia con la carga. En los tramos rectilíneos, la trayectoria se mantuvo relativamente estable, aunque con ligeras correcciones que sugieren ajustes en la dirección debido a la influencia de la carga.

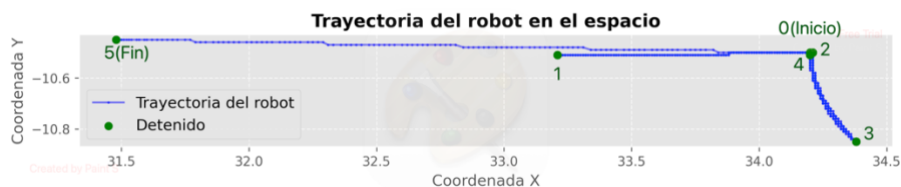


Figura 4.16. Gráfica de trayectoria del robot en el tiempo en el ensayo de motores con carga

4.4.3. Validación de mapeo del área de trabajo

El sistema completó exitosamente el proceso de mapeo especificado en la sección 3.6.4.3 en un tiempo de 53 minutos, generando dos representaciones del entorno:

1. **Mapa crudo:** este mapa muestra la densidad y ubicación de los obstáculos detectados por el sensor LiDAR. Las áreas con mayores valores corresponden a zonas con mayor densidad de obstáculos. Este se encuentra en formato .npy, por lo cual no es visualizable.

2. **Mapa navegable:** este mapa representa las áreas seguras para la navegación, definidas tras expandir los obstáculos con un filtro de convolución. Este mapa se puede visualizar en la Figura 4.17.

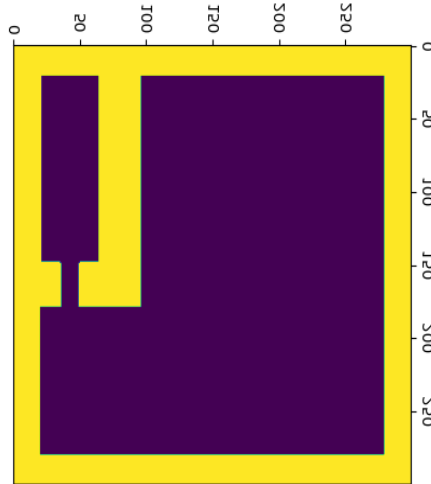


Figura 4.17. Mapa de ocupación

4.4.4. Validación de planificación de trayectorias y traslado de avioneta con algoritmo D* Lite

En el ensayo de la sección 3.6.4.4 se logró completar el traslado de la avioneta desde un punto del aeródromo al hangar de mantenimiento en el simulador Webots en todas las configuraciones.

Análisis de velocidad

En la Figura 4.18 se observa la evolución de la velocidad del robot en el tiempo para distintas configuraciones de p_1 y p_2 del algoritmo D* Lite.

Las configuraciones evaluadas presentan las siguientes trayectorias:

- **Configuración 1 ($p_1 = 1, p_2 = 1$) (azul):** presenta una velocidad más inestable con picos elevados. Este comportamiento sugiere que la configuración puede generar oscilaciones debido a una mayor rigidez en la planificación. Además, es la configuración más rápida en completar su objetivo.

1. **Configuración 2 ($p_1 = 1, p_2 = 0.2$) (verde):** muestra valores de velocidad

relativamente más bajos y con algunos picos dispersos, indicando que esta configuración permite ajustes más suaves. Además, es la configuración que más tiempo toma en completar su objetivo.

2. **Configuración 3 ($p_1 = 0.5, p_2 = 0.2$) (amarillo):** al igual que la Configuración 2, presenta valores de velocidad más bajos, presentando mayor estabilidad. En contraste con la Configuración 2, completa su tarea unos minutos antes.
3. **Configuración 4 ($p_1 = 0.5, p_2 = 0.5$) (rojo):** posee un comportamiento parecido al generado en la Configuración 1, pero con valores menores de velocidad y con mayor estabilidad. A su vez, completa la tarea unos minutos más tarde que en la Configuración 1.

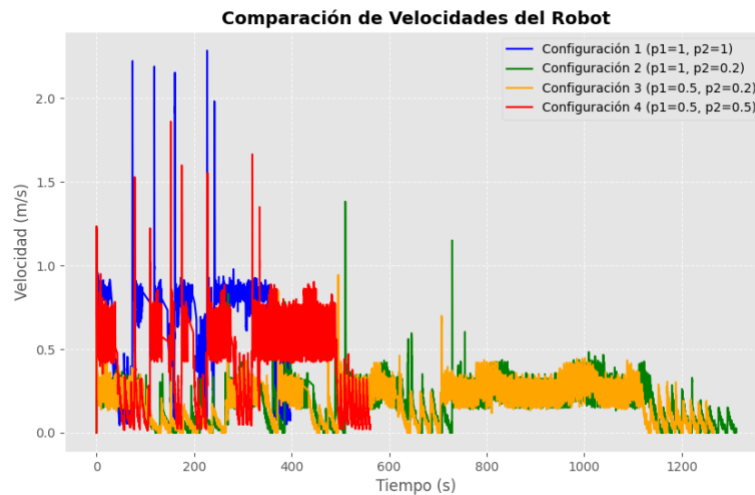


Figura 4.18. Gráfica de comparación de velocidades en el tiempo

El análisis de la velocidad del robot durante la ejecución del algoritmo D* Lite demuestra que la elección de los parámetros p_1 y p_2 influye significativamente en el comportamiento dinámico del sistema de navegación. A partir de la gráfica analizada, se observa que las trayectorias con valores altos de p_1 y p_2 presentan picos más altos, sugiriendo que una planificación con cambios más estrictos puede generar oscilaciones y correcciones más abruptas en la trayectoria. Por el contrario, configuraciones con parámetros más bajos tienden a generar un comportamiento más estable y una navegación más uniforme.

Análisis de trayectoria

En la Figura 4.19 se observa la comparación de trayectorias del robot en el espacio para distintas configuraciones de p_1 y p_2 del algoritmo D* Lite.

1. **Configuración 1 ($p_1 = 1, p_2 = 1$) (azul):** la trayectoria se mantiene estable y ajustada a lo largo del recorrido, con mínimos desvíos. Esto sugiere que una mayor rigidez en la planificación permite generar trayectorias más precisas y consistentes. Sin embargo, en zonas de cambios bruscos de dirección, la rigidez podría dificultar la adaptación a nuevas condiciones del entorno.
4. **Configuración 2 ($p_1 = 1, p_2 = 0.2$) (verde):** presenta una trayectoria muy similar a la Configuración 1, con ligeras desviaciones en ciertas curvas. Este comportamiento indica que la reducción del parámetro p_2 permite ajustes más suaves sin comprometer significativamente la precisión de la trayectoria.
5. **Configuración 3 ($p_1 = 0.5, p_2 = 0.2$) (amarillo):** se observan pequeñas desviaciones en la ruta en comparación con las configuraciones anteriores. La trayectoria se adapta de manera más dinámica en zonas con variaciones en la dirección, lo que sugiere que la reducción de p_1 mejora la capacidad de ajuste del robot en entornos con obstáculos o cambios inesperados.
6. **Configuración 4 ($p_1 = 0.5, p_2 = 0.5$) (rojo):** presenta un comportamiento intermedio entre la Configuración 1 y la Configuración 3. Aunque mantiene una trayectoria cercana a la óptima, su flexibilidad es mayor, permitiendo pequeñas correcciones en el trayecto. Se aprecia una mejor adaptabilidad en curvas y zonas de alta variabilidad, sin comprometer en exceso la estabilidad del recorrido.

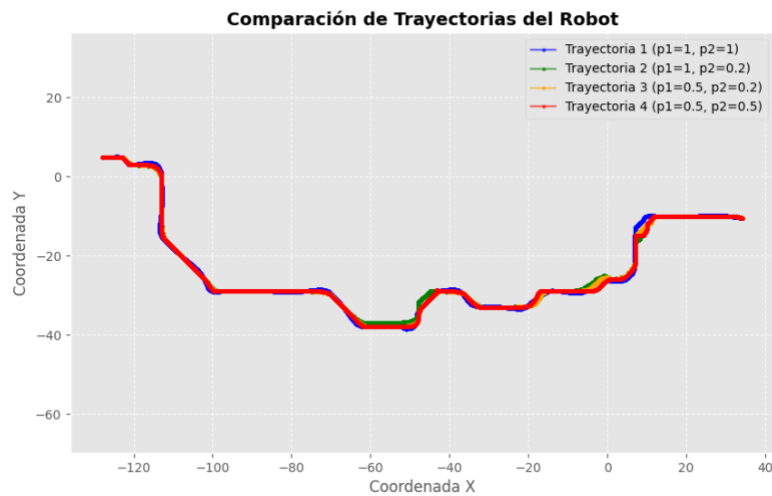


Figura 4.19. Gráfica de comparación de trayectorias

El análisis de la trayectoria del robot durante la ejecución del algoritmo D* Lite demuestra que la selección de los parámetros p_1 y p_2 genera trayectorias más rígidas, mientras que valores más bajos introducen mayor flexibilidad, permitiendo ajustes dinámicos en el trayecto.

Análisis de distancia al destino

En la Figura 4.20 se observa la evolución de la distancia del robot al destino en función del tiempo para distintas configuraciones de p_1 y p_2 del algoritmo D* Lite.

- **Configuración 1 ($p_1 = 1, p_2 = 1$) (azul):** es la configuración que alcanza el destino en el menor tiempo, reduciendo la distancia de manera constante y rápida. Su trayectoria muestra una tendencia descendente pronunciada, lo que sugiere que una planificación más rígida favorece una navegación eficiente en términos de distancia recorrida.
7. **Configuración 2 ($p_1 = 1, p_2 = 0.2$) (verde):** tiene un comportamiento más lento en la reducción de la distancia al destino, con un inicio menos eficiente en comparación con la Configuración 1. La planificación más flexible en p_2 permite ajustes suaves, pero a costa de un mayor tiempo de llegada.
 8. **Configuración 3 ($p_1 = 0.5, p_2 = 0.2$) (amarillo):** muestra una evolución de la

distancia similar a la Configuración 2, con una disminución progresiva, pero sin los descensos abruptos observados en la Configuración 1. Esto sugiere que valores más bajos de p_1 generan una planificación más adaptable, aunque menos eficiente en términos de tiempo total.

9. **Configuración 4 ($p_1 = 0.5, p_2 = 0.5$) (rojo):** tiene un comportamiento intermedio entre las configuraciones anteriores. Su desempeño es más eficiente que la Configuración 3 en la primera mitad del trayecto, pero no tan rápido como la Configuración 1.

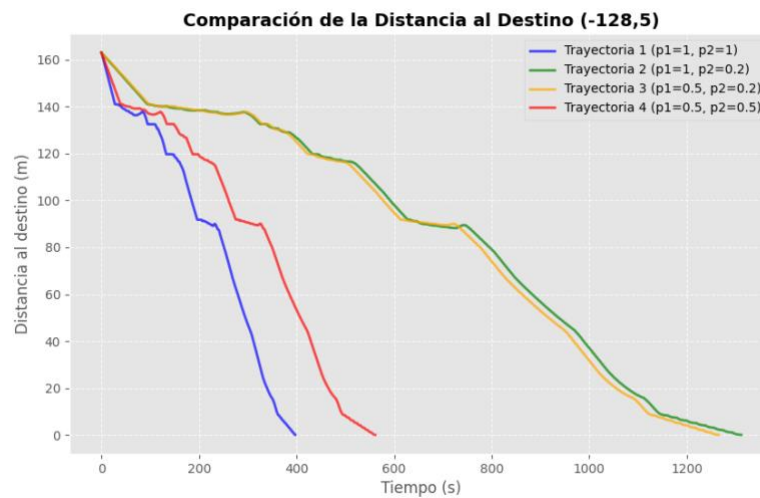


Figura 4.20. Gráfica de comparación de distancia al destino en el tiempo

El análisis de la distancia al destino confirma que valores altos de p_1 permiten una planificación más directa y eficiente, reduciendo el tiempo necesario para completar la trayectoria. Sin embargo, valores más bajos de p_1 y p_2 introducen mayor estabilidad, lo que resulta en trayectorias con mayor capacidad de ajuste. En aplicaciones donde el tiempo de llegada es crítico, configuraciones con parámetros p_1 y p_2 más altos son preferibles, mientras que, en entornos dinámicos, valores más bajos pueden mejorar la respuesta del sistema sin comprometer excesivamente la eficiencia.

El análisis comparativo sugiere que no existe una única configuración óptima para la navegación del robot, sino que la elección de los parámetros p_1 y p_2 debe depender de los objetivos operacionales y las condiciones del entorno. Si la prioridad es minimizar el tiempo de llegada y mantener una trayectoria más rígida y eficiente, la configuración $p_1 = 1, p_2 = 1$ es la más adecuada, ya que permite reducir rápidamente la distancia al destino pero con

mayores fluctuaciones en la velocidad. Si el entorno de operación es dinámico y se requiere una mayor capacidad de ajuste ante obstáculos inesperados, la configuración $p_1 = 0.5, p_2 = 0.2$ proporciona una navegación más estable y flexible, con menores variaciones en la velocidad y ajustes más suaves en la trayectoria.

Por otro lado, configuraciones intermedias como $p_1 = 0.5, p_2 = 0.5$ pueden representar un equilibrio entre estabilidad y adaptabilidad, permitiendo una planificación eficiente sin comprometer la capacidad de reacción del sistema. Estas configuraciones pueden ser útiles en escenarios donde el entorno presenta cambios ocasionales, pero no constantes, asegurando un desempeño más versátil.

Este estudio confirma que, con una adecuada selección de parámetros, el algoritmo D Lite* es una estrategia viable y eficiente para la navegación autónoma del remolcador de avionetas en el simulador Webots. Los resultados obtenidos proporcionan un marco sólido para futuras optimizaciones, permitiendo ajustes estratégicos en función de los requisitos específicos del sistema y el entorno de operación.

CONCLUSIONES

1. La identificación de requisitos en un sistema permite consolidar los parámetros de diseño de una solución, en este caso obtener características para la simulación con parámetros realistas.
2. La selección del proyecto de Freire y Rubio (2016) como referencia técnica facilitó el desarrollo de un diseño basado en experiencias previas, aportando avances en la investigación en el área y optimizando tiempo y recursos.
3. El análisis del modelo cinemático del robot remolcador permitió determinar los parámetros óptimos para la configuración de los motores.
4. La configuración de los parámetros físicos en la simulación dentro del software Webots demostró que existe una limitación en el procesamiento del peso de la aeronave, lo que sugiere la necesidad de explorar simuladores alternativos que permitan incorporar este factor con mayor precisión.
5. La implementación de la arquitectura del sistema validó la eficacia de los árboles de comportamiento, destacando su facilidad de implementación y su capacidad para diagnosticar y resolver problemas de manera estructurada.
6. La recreación del entorno en Webots permitió evaluar con mayor realismo el desempeño del robot, proporcionando un escenario más preciso para el análisis de su funcionamiento y su interacción con el entorno.
7. Las pruebas realizadas con el algoritmo D* Lite demostraron su eficacia en la replanificación de trayectorias en tiempo real, permitiendo al robot evitar obstáculos imprevistos y mejorar la eficiencia del desplazamiento en el simulador.
8. La configuración de los parámetros p_1 y p_2 del control de motores influye directamente en la estabilidad y capacidad de ajuste de las trayectorias del robot. Mientras que valores más altos optimizan el tiempo de llegada en aplicaciones críticas, valores más bajos pueden mejorar la respuesta del sistema en entornos dinámicos sin comprometer significativamente la eficiencia.

9. El diseño del robot remolcador desarrollado para la tarea de traslado de avionetas mostró un desempeño funcional en la simulación, cumpliendo con los requisitos de maniobrabilidad y autonomía, lo que respalda su potencial implementación en entornos reales con futuras mejoras.

OPORTUNIDADES DE MEJORA

- **Optimización del modelo de simulación:** la limitación en el procesamiento del peso de la aeronave en Webots sugiere la necesidad de evaluar simuladores alternativos, como Gazebo o CoppeliaSim, que permitan una simulación más precisa de la física y la interacción con cargas pesadas.
- **Validación del diseño en hardware real:** para evaluar la viabilidad práctica del robot remolcador, se recomienda la construcción de un prototipo físico y la realización de pruebas en un entorno controlado, lo que permitiría validar la funcionalidad del diseño y ajustar su desempeño ante condiciones reales.
- **Optimización del consumo energético:** un análisis más detallado de la eficiencia energética del sistema podría contribuir a mejorar la autonomía del robot, considerando la selección de motores más eficientes y estrategias de optimización del control de movimiento.
- **Incorporación de controlador de velocidad:** la implementación de controladores PID en los motores permitiría mejorar la estabilidad y precisión del movimiento del robot, garantizando un ajuste dinámico de la velocidad más eficiente.

BIBLIOGRAFÍA

Almasri, M. M., Alajlan, A. M., & Elleithy, K. M. (2016). Trajectory planning and collision avoidance algorithm for mobile robotics system. *IEEE Sensors Journal*, 16(12), 5021–5028.

<https://doi.org/10.1109/JSEN.2016.2553126>

Al Khatib, E. I., Jaradat, M. A. K., & Abdel-Hafez, M. F. (2020). Low-cost reduced navigation system for mobile robot in indoor/outdoor environments. *IEEE Access*, 8, 2971169.

<https://doi.org/10.1109/ACCESS.2020.2971169>

Autodesk. (s.f.). Fusion 360. <https://www.autodesk.com/latam/products/fusion-360/overview>

Bernheim, S., Costa, A., & De Luca, A. (2019). Robot autónomo móvil terrestre con énfasis en SLAM y fusión sensorial. Memoria de proyecto para la obtención del título de Ingeniero Electricista, Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay.

Cyberbotics. (s.f.). Webots: Simulador de robots. <https://cyberbotics.com>

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269–271. <https://doi.org/10.1007/BF01386390>

Freire Fernández, J. J., & Rubio Segovia, J. A. (2016). Diseño y construcción de robot remolcador para avionetas. Trabajo final de graduación, Facultad de Ingeniería en Mecánica y Ciencias de la Producción, Escuela Superior Politécnica del Litoral, Guayaquil, Ecuador.

Gutiérrez Suárez, B. C. (2012). Diseño e implementación de un algoritmo de generación de trayectorias para la evasión de un obstáculo para un robot móvil. Tesis para optar el Título de Ingeniero Electrónico, Facultad de Ciencias e Ingeniería, Pontificia Universidad Católica del Perú.

Harapanahalli, S., O'Mahony, N., Velasco Hernandez, G., Campbell, S., Riordan, D., & Walsh, J. (2019). Autonomous Navigation of mobile robots in factory environment. *Procedia Manufacturing*, 38, 1524-1531. <https://doi.org/10.1016/j.promfg.2020.01.134>

Hichri, B., Fauroux, J-C., Adouane, L., Doroftei, I., & Mezouar, Y. (2019). Design of cooperative mobile robots for co-manipulation and transportation tasks. *Robotics and Computer Integrated Manufacturing*, 57, 412-421.

<https://doi.org/10.1016/j.rcim.2019.01.002>

leha111dxe. (s.f.). Learjet 35/36. <https://www.cgtrader.com/3d-models/aircraft/private-aircraft/learjet-35-36>

Interact Analysis. (2023). Mobile Robots – 2023 (6th ed.). Interact Analysis. <https://interactanalysis.com/research/mobile-robots/>

International Federation of Robotics. (2023). World Robotics 2023. IFR. <https://ifr.org/free-downloads/>

International Organization for Standardization. (2021). Robotics — Vocabulary (ISO 8373:2021). <https://www.iso.org/standard/75539.html>

Hart, P., Nilsson, N., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. <https://doi.org/10.1109/tssc.1968.300136>

Jiménez Cuesta, J. (2012). Simulación de vehículos eléctricos ligeros. Proyecto Final de Carrera, Ingeniería Informática, Escola Tècnica Superior d'Enginyeria Informàtica, Universitat Politècnica de València, Valencia, España.

Koenig, S., & Likhachev, M. (2002). D* Lite. En *Proceedings of the AAAI Conference on Artificial Intelligence*. Georgia Institute of Technology, Atlanta, GA y Carnegie Mellon University, Pittsburgh, PA. <https://www.aaai.org/Papers/AAAI/2002/AAAI02-018.pdf>

Koubaa, A., Bennaceur, H., Chaari, I., Trigui, S., Ammar, A., Sriti, M.-F., Alajlan, M., Cheikhrouhou, O., & Javed, Y. (2018). *Robot path planning and cooperation: Foundations, algorithms and experimentations*. Springer International Publishing. <https://doi.org/10.1007/978-3-319-77042-0>

LaValle, S. M. (1998, October). Rapidly-exploring random trees: A new tool for path planning (Technical Report No. TR 98-11). Computer Science Department, Iowa State University.

Learjet Inc. (2012). Learjet 35/35A/36/36A maintenance manual. Wichita, KS: Autor.

Morris, R., Chang, M. L., Archer, R., Cross II, E. V., Thompson, S., Franke, J. L., Garrett, R. C., Malik, W., McGuire, K., & Hemann, G. (2015). *Self-Driving Aircraft Towing Vehicles*:

A Preliminary Report. Papers from the 2015 AAAI Workshop on Artificial Intelligence for Transportation: Advice, Interactivity, and Actor Modeling. Association for the Advancement of Artificial Intelligence.

Mototok. (s.f.). Mototok Aircraft Tugs: Electric, Towbarless, Remote-Controlled. Recuperado el 28 de marzo de 2024, de <https://www.mototok.com>

Priyadharshini, S., Venkatesh, S., Ramkumar, K., Amirtharajan, R., Thanikaiselvan, V., & Mahalingam, H. (2023). SLAM (Simultaneous localisation and mapping) using E-PUCK robot in Webots software. In 2023 2nd International Conference on Vision Towards Emerging Trends in Communication and Networking Technologies (ViTECoN) (pp. 1-6). IEEE. <https://doi.org/10.1109/VITECON58111.2023.10157295>

Sabino, C. (1992). El proceso de investigación. Ed. Panapo.

Siciliano, B., & Khatib, O. (Eds.). (2016). Handbook of robotics (2da ed.). Springer. <https://doi.org/10.1007/978-3-319-32552-1>

Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2009). Robotics: Modelling, Planning and Control. Springer-Verlag London Ltd.

Stentz, A. (1994). Optimal and efficient path planning for partially-known environments. En Proceedings of the 1994 IEEE International Conference on Robotics and Automation (Vol. 4, pp. 3310-3317). IEEE. <https://doi.org/10.1109/ROBOT.1994.351061>

Towbots. (s.f.). Remote Controlled Aircraft Towing | Towbots. <https://www.towbots.us>

Towflexx. (s.f.). TowFLEXX | electric remote controlled towbarless aircraft tugs. <https://towflexx.com>

uamodeling3d. (s.f.). Hangar red metal Free low-poly 3D model. <https://www.cgtrader.com/free-3d-models/industrial/other/hangar-red-metal>

Zhang, C., Li, M., Jiao, J., Guo, P., & Yang, X. (2023). Path Planning of Mobile Robot Based on Optimized D*Lite Algorithm. En 2023 IEEE 3rd International Conference on Information Technology, Big Data and Artificial Intelligence (ICIBA) (pp. 1022-1027). IEEE. <https://doi.org/10.1109/ICIBA56860.2023.10165159>

ANEXO A. RESPUESTAS DE CUESTIONARIO

1. Aquí se registraron diversas respuestas como tareas de mantenimiento y transporte de aeronaves.
2. Se destacan los movimientos de objetos pesados como motores y nuevamente el traslado de avionetas.
3. Las respuestas reflejaron que este tipo de tareas se realizan a diario múltiples veces, por lo que es una tarea de alta frecuencia.
4. Nuevamente se reporta una alta demanda de este tipo de tareas.
5. En este caso, se indica poca frecuencia como respuesta.
6. Aquí se especifica que es una tarea diaria y con una frecuencia mínima de 3 veces por día.
7. Aquí las opiniones se encuentran divididas, resultando en la mitad indicando que se desempeñan en un área ordenada y la otra mitad no.
8. Las respuestas registradas señalan que es un trabajo que se realiza con una frecuencia de 3 veces por semana.
9. Se indican los métodos de tintas penetrantes y ultrasonido.

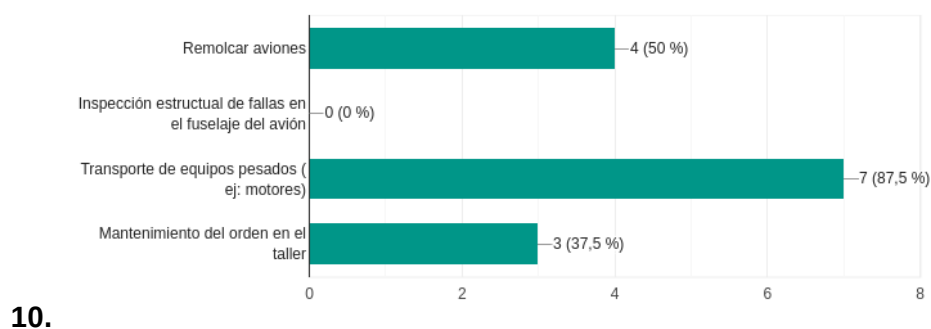
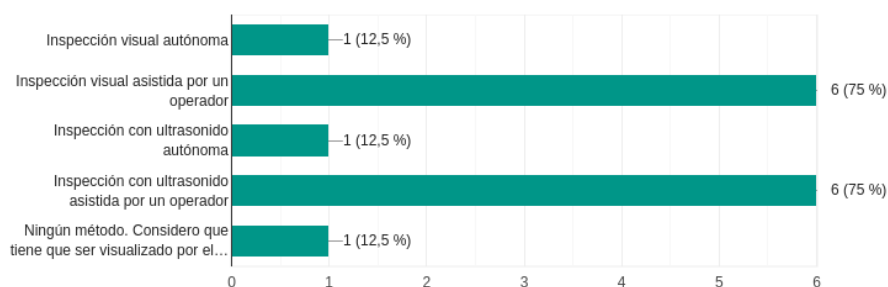
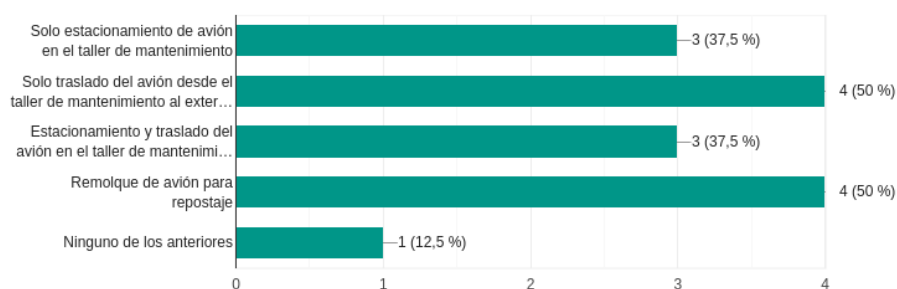


Figura A.1. Respuesta de la pregunta 10



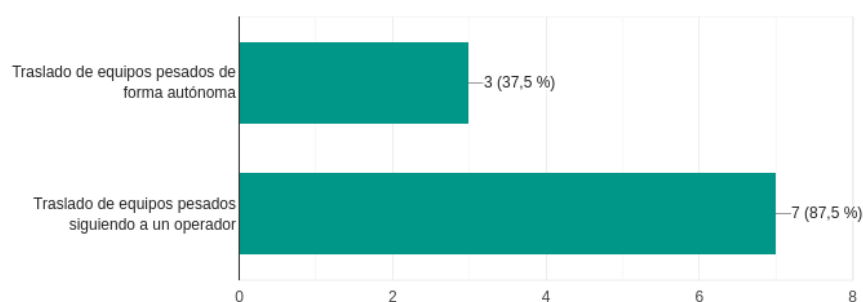
11.

Figura A.2. Respuesta de la pregunta 11



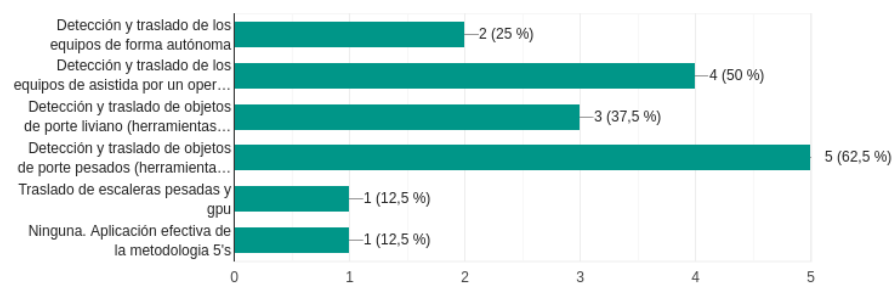
12.

Figura A.3. Respuesta de la pregunta 12



13.

Figura A.4. Respuesta de la pregunta 13



14.

Figura A.5. Respuesta de la pregunta 14

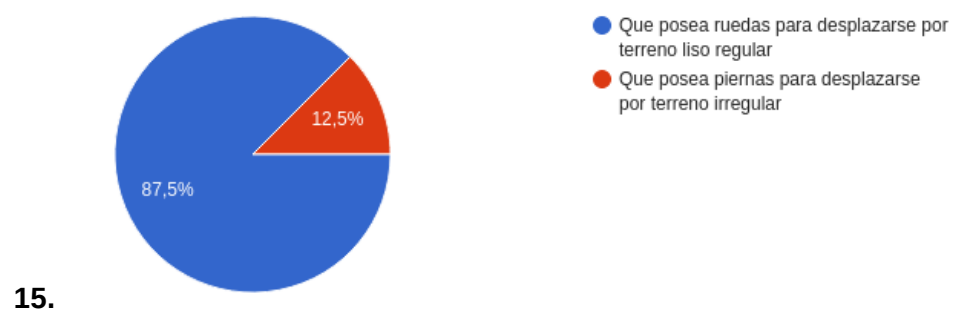


Figura A.6. Respuesta de la pregunta 15

**ANEXO B. PLANOS DE DISEÑO Y ENSAMBLE DE ROBOT REMOLCADOR
ADAPTADOS DE FREIRE Y RUBIO (2016)**

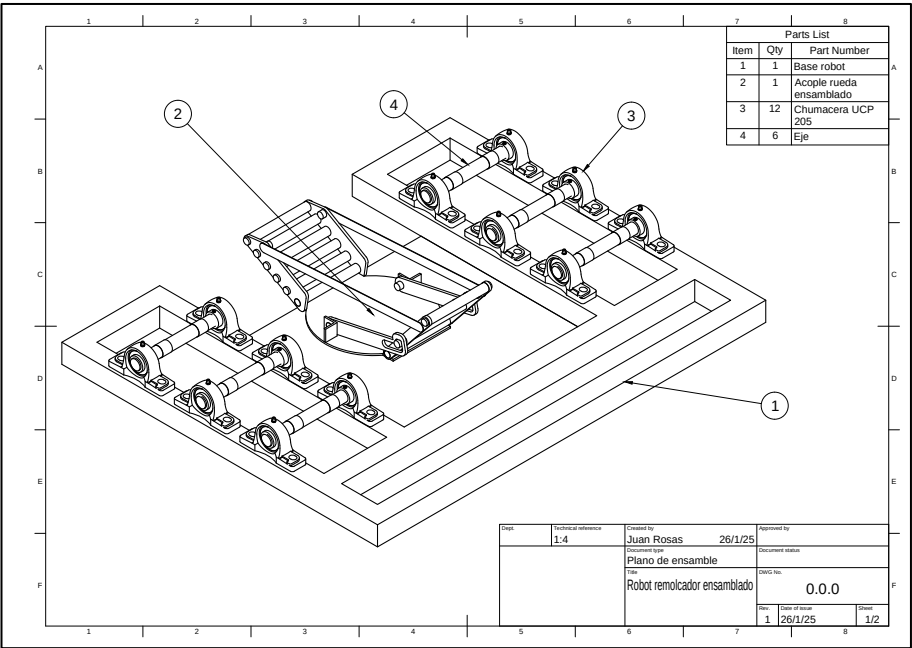


Figura B.1. Plano de ensamble robot remolcador

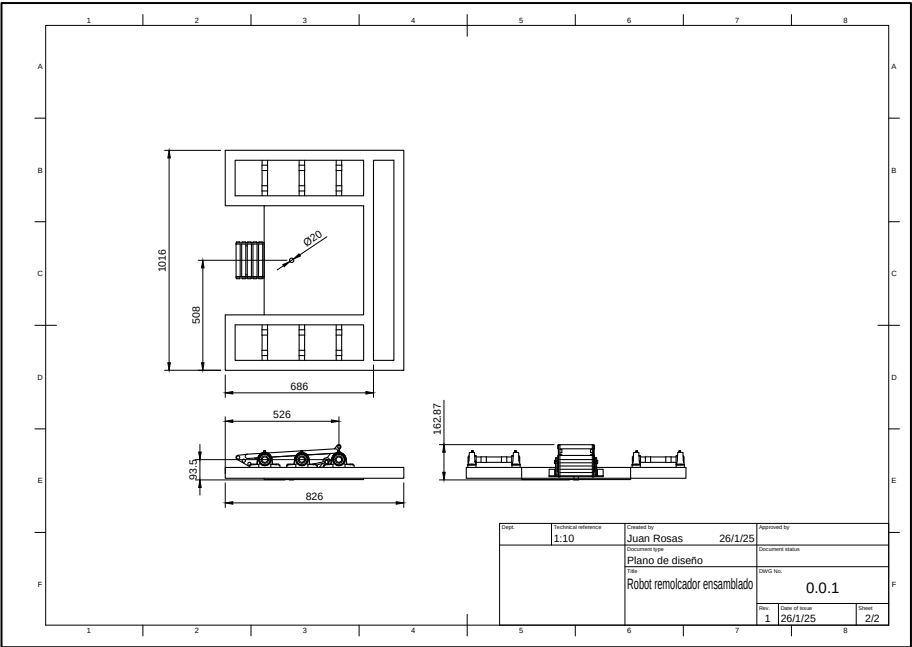


Figura B.2. Plano de diseño de robot remolcador ensamblado

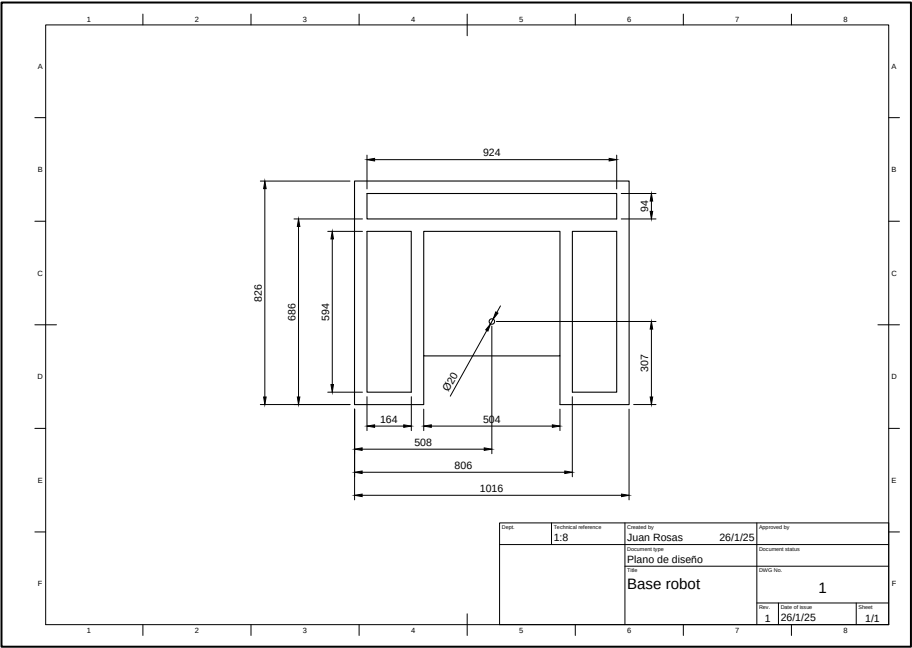


Figura B.3. Plano de diseño de base del robot

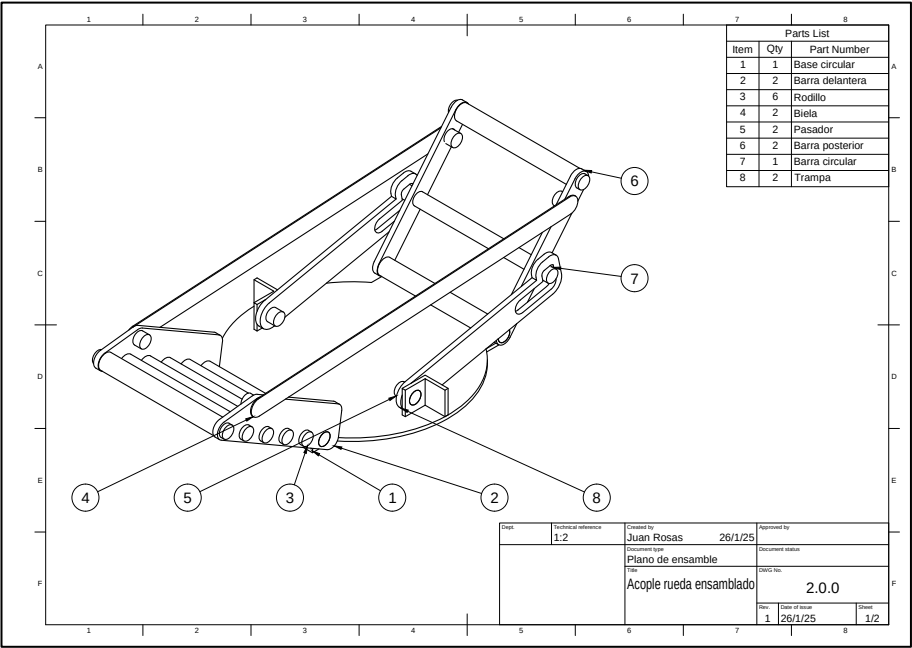


Figura B.4. Plano de ensamble de acople de la rueda al robot remolcador

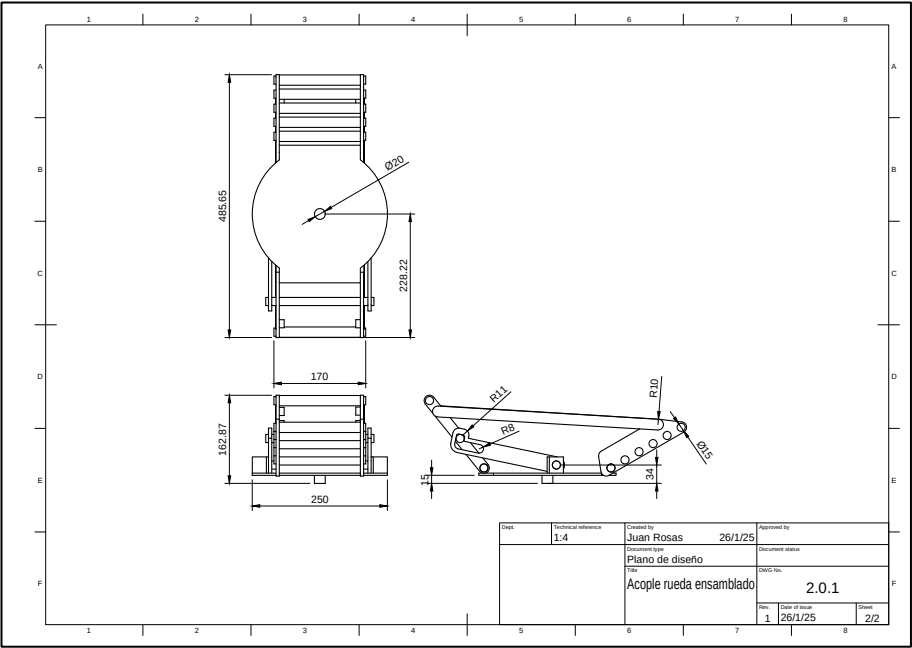


Figura B.5. Plano de diseño de acople de rueda ensamblado

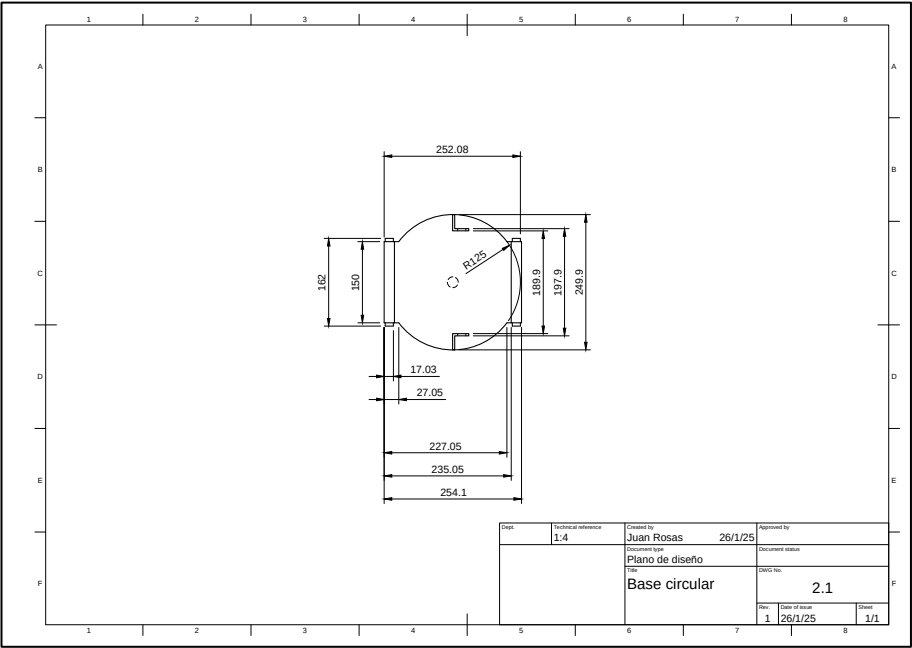


Figura B.6. Plano de diseño de base circular

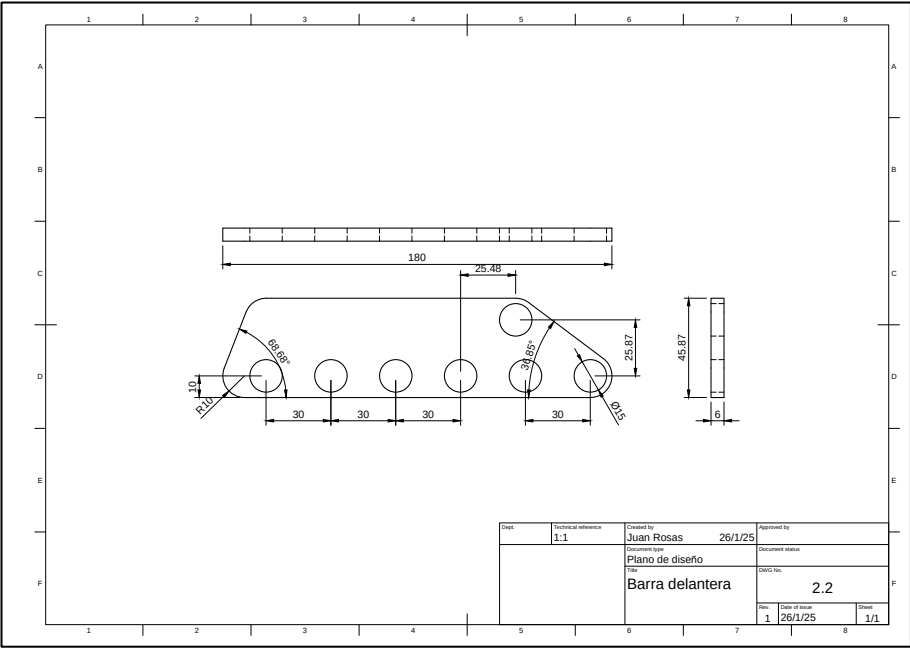


Figura B.7. Plano de diseño de barra delantera

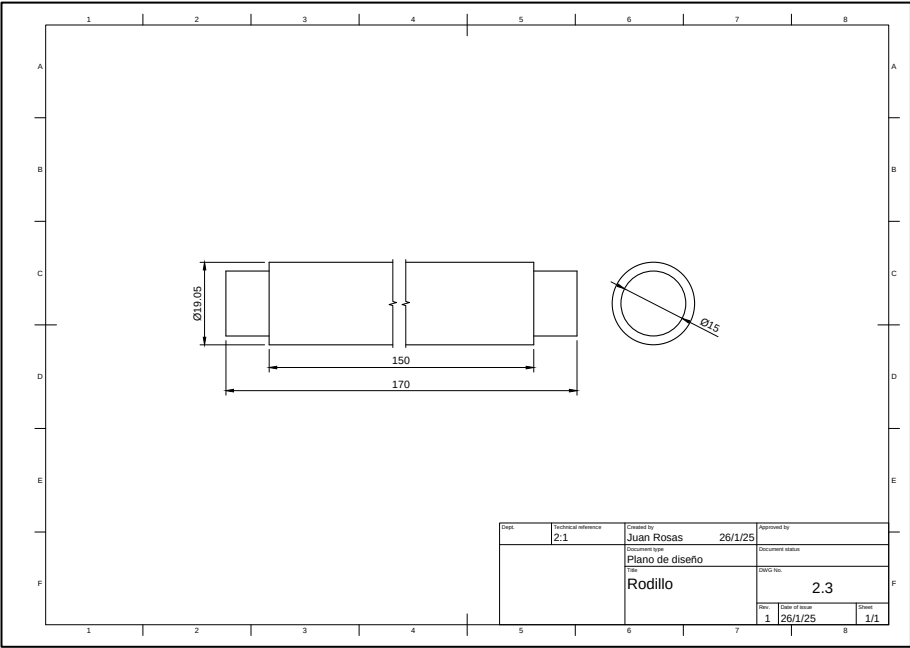


Figura B.8. Plano de diseño de rodillo

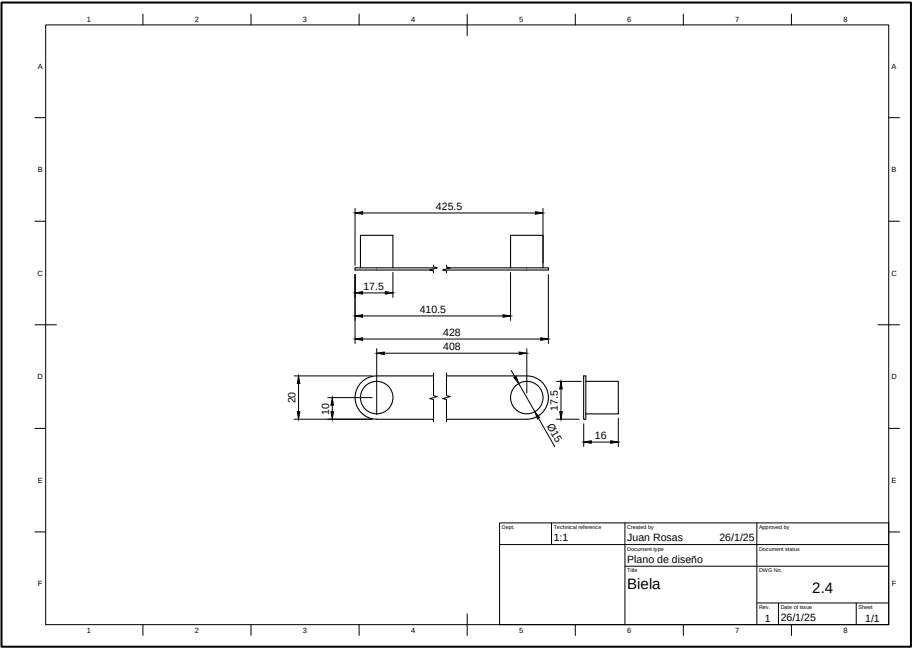


Figura B.9. Plano de diseño de biela

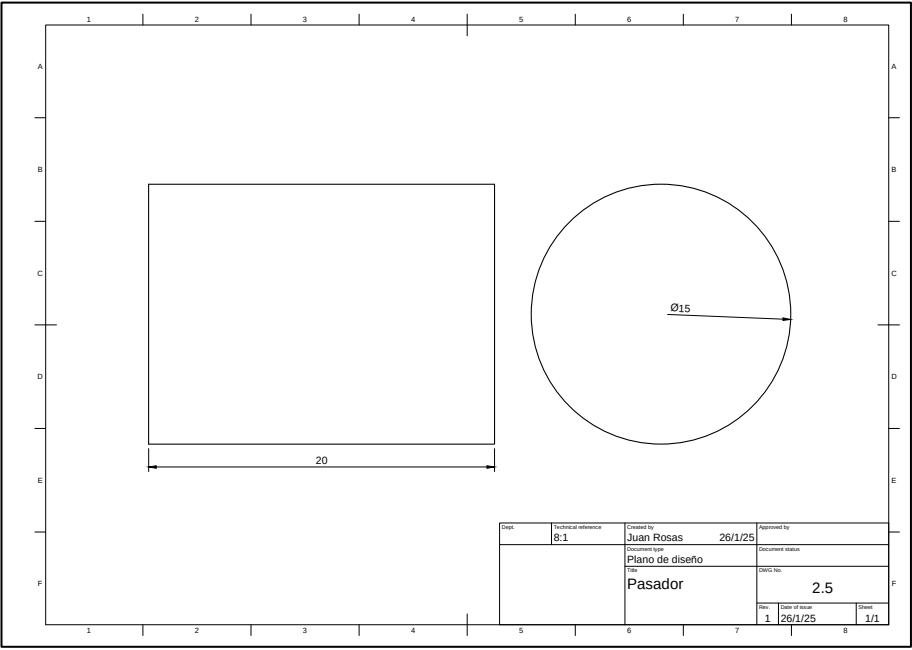


Figura B.10. Plano de diseño de pasador

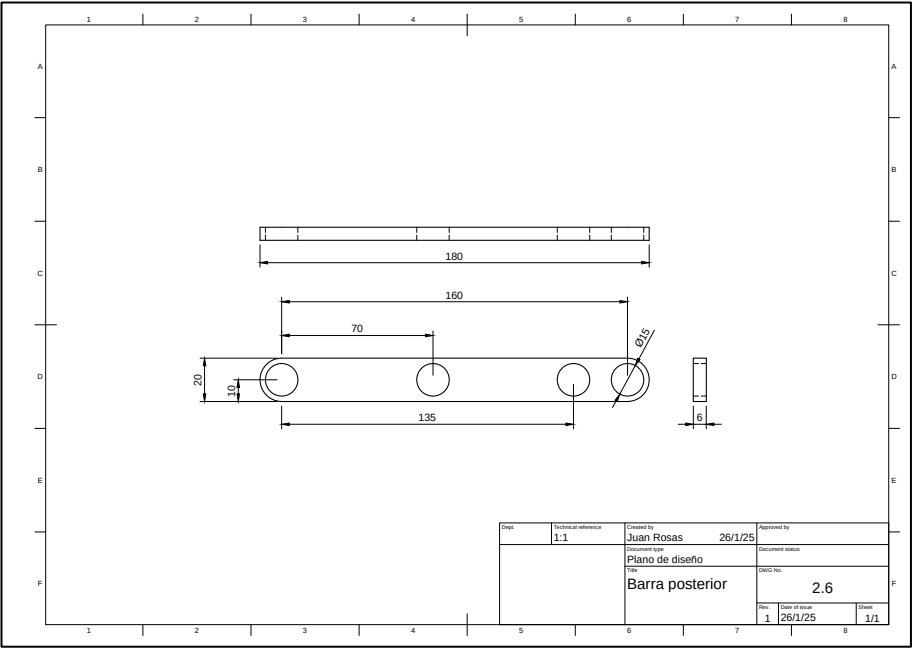


Figura B.11. Plano de diseño de barra posterior

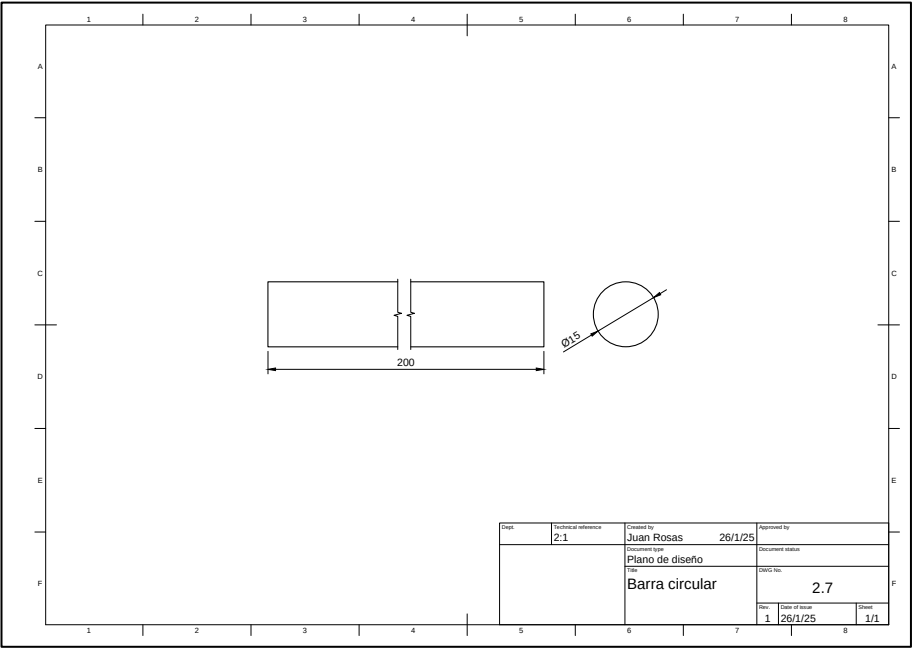


Figura B.12. Plano de diseño de barra circular

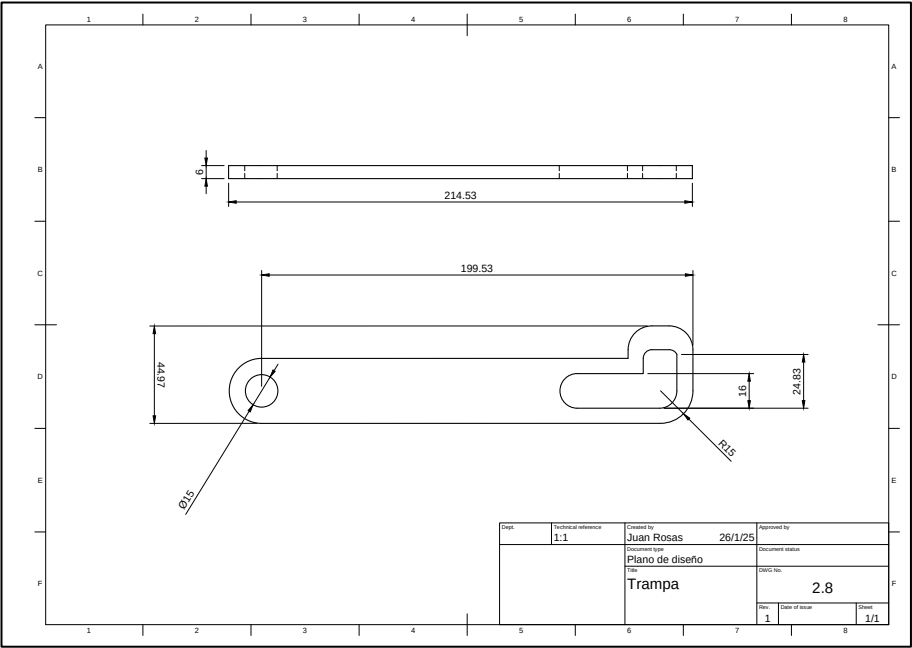


Figura B.13. Plano de diseño de trampa

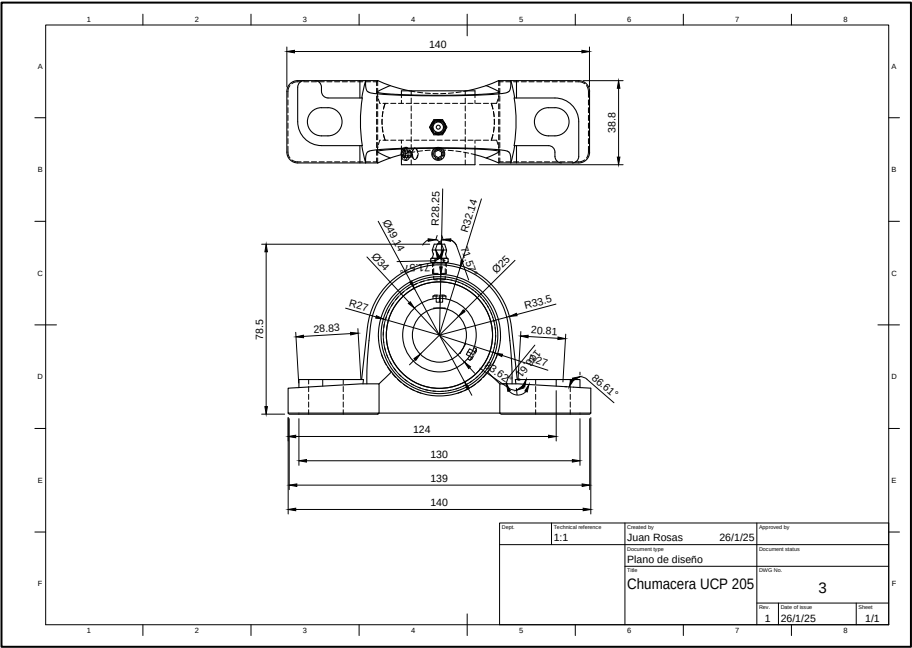


Figura B.14. Plano de diseño de chumacera UCP 205

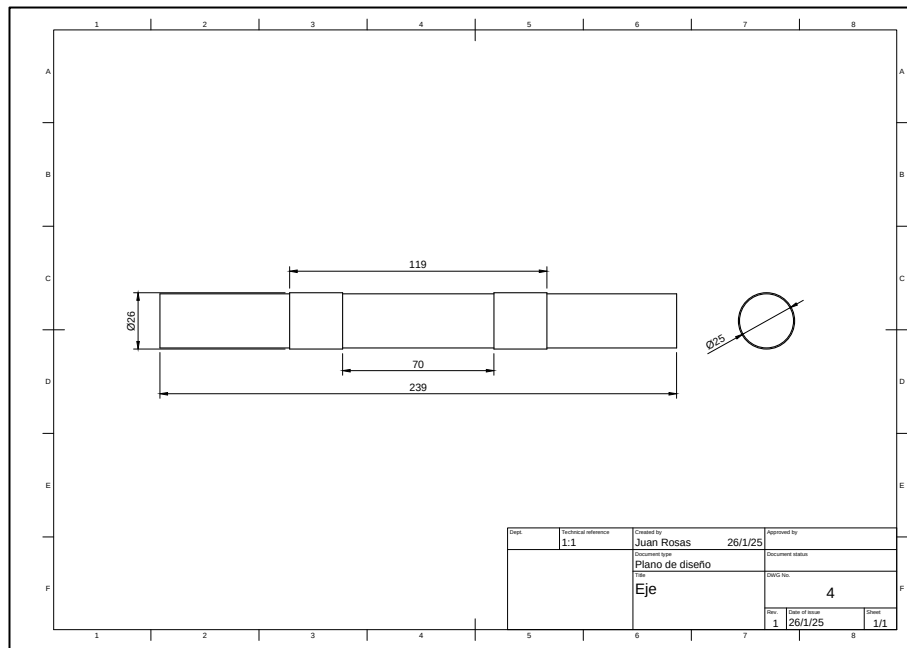


Figura B.15. Plano de diseño de eje

ANEXO C. SCRIPTS DE CÓDIGO DE SOLUCIÓN IMPLEMENTADA

1. BT_controller.py:

```
from os.path import exists
import numpy as np
import py_trees
from py_trees.composites import Sequence, Parallel, Selector
from mapping import Mapping
from navigation import Navigation
from planning import Planning
from controller import Robot, Supervisor
from map_utils import generate_wp

# crear una instancia del robot
robot = Supervisor()

# obtener el paso de tiempo del mundo actual
timestep = int(robot.getBasicTimeStep())

# generar los puntos de referencia (waypoints) para mapear el entorno
WP = generate_wp()

# verificar si el mapa ya existe antes de realizar el mapeo del entorno
class DoesMapExist(py_trees.behaviour.Behaviour):
    def update(self):
        file_exists = exists('cspace.npy') # verifica si el archivo del mapa existe
        if(file_exists):
            print("El mapa ya existe")
            map = np.load('cspace.npy') # cargar el mapa existente
            blackboard.write('map',map) # almacenar el mapa en la pizarra negra (blackboard)
            return py_trees.common.Status.SUCCESS
        else:
            print("El mapa no existe")
            map = np.zeros((300,300)) # Crear un mapa vacío si no existe
            blackboard.write('map',map)
            return py_trees.common.Status.FAILURE
```

clase para gestionar el blackboard, que almacena datos compartidos entre comportamientos

class Blackboard:

```
def __init__(self):
    self.data = {}
def write(self, key, value):
    self.data[key] = value
def read(self, key):
    return self.data.get(key)
```

blackboard = Blackboard()

blackboard.write('robot', robot)

blackboard.write('waypoints', WP)

crear el árbol de comportamiento con nodos de secuencia y selector

```
tree = Sequence("Main", children=[
    Selector("Existe el mapa?", children=[
        DoesMapExist("Buscar mapa"),
        Parallel("Mapping", policy=py_trees.common.ParallelPolicy.SuccessOnOne(), children=[
            Mapping("Mapear el entorno", blackboard),
            Navigation("Mover alrededor del aeródromo", blackboard)
        ])
    ], memory=True),
    Planning("Trasladar avioneta al hangar", blackboard, (-128, 5)),
], memory=True)
```

configurar el árbol de comportamiento

tree.setup_with_descendants()

bucle principal del simulador

```
while robot.step(timestep) != -1:
    # ejecutar un ciclo del árbol de comportamiento
    tree.tick_once()
```

2. d_star_lite.py:

```
from priority_queue import PriorityQueue, Priority
from grid import OccupancyGridMap
import numpy as np
from grid_utils import heuristic, Vertices
```

OBSTACLE = 1

UNOCCUPIED = 0

clase que implementa el algoritmo D* Lite para planificación de rutas en entornos dinámicos

class DStarLite:

def __init__(self, map: OccupancyGridMap, s_start: (int,int), s_goal: (int,int)):

"""

:param map: el mapa de referencia del entorno proporcionado por la interfaz gráfica

:param s_start: ubicación de inicio

:param s_goal: ubicación de destino

"""

self.new_edges_and_old_costs = Vertices()

inicio del algoritmo

self.s_start = s_start

self.s_goal = s_goal

self.s_last = s_start

self.k_m = 0 # acumulación

self.U = PriorityQueue()

self.rhs = np.ones((map.x_dim, map.y_dim)) * np.inf

self.g = self.rhs.copy()

mapa detectado basado en el entorno

self.sensed_map = OccupancyGridMap(map_data=map,
x_dim=map.x_dim,
y_dim=map.y_dim,
exploration_setting='8N')

inicializa el nodo objetivo con costo cero

self.rhs[self.s_goal] = 0

self.U.insert(self.s_goal, Priority(heuristic(self.s_start, self.s_goal), 0))

calcula la clave del nodo basado en su heurística y costos acumulados

def calculate_key(self, s: (int,int)):

"""

:param s: el vértice del que queremos calcular la clave

:return: clase priority de las dos claves

```

"""

k1 = min(self.g[s], self.rhs[s]) + heuristic(self.s_start, s) + self.k_m
k2 = min(self.g[s], self.rhs[s])
return Priority(k1, k2)

# calcula el costo entre dos nodos en función de si están ocupados o no
def cost(self, u: (int, int), v: (int, int)) -> float:
    """
    calcular el costo entre nodos
    :param u: vértice de origen
    :param v: vértice de destino
    :return: distancia euclidiana para recorrer. inf si hay un obstáculo en el camino
    """

    if not self.sensed_map.is_unoccupied(u) or not self.sensed_map.is_unoccupied(v):
        return float('inf')
    else:
        return heuristic(u, v)

# verifica si un nodo está en la cola de prioridad
def contain(self, u: (int, int)) -> (int, int):
    return u in self.U.vertices_in_heap

# actualiza el valor de un nodo en la cola de prioridad basado en su costo
def update_vertex(self, u: (int, int)):
    if self.g[u] != self.rhs[u] and self.contain(u):
        self.U.update(u, self.calculate_key(u))
    elif self.g[u] != self.rhs[u] and not self.contain(u):
        self.U.insert(u, self.calculate_key(u))
    elif self.g[u] == self.rhs[u] and self.contain(u):
        self.U.remove(u)

# calcula el camino más corto a partir de la información conocida
def compute_shortest_path(self):
    while self.U.top_key() < self.calculate_key(self.s_start) or self.rhs[self.s_start] > self.g[self.s_start]:
        u = self.U.top()
        k_old = self.U.top_key()
        k_new = self.calculate_key(u)

```

```

if k_old < k_new:
    self.U.update(u, k_new)
elif self.g[u] > self.rhs[u]:
    self.g[u] = self.rhs[u]
    self.U.remove(u)
    pred = self.sensed_map.succ(vertex=u)
    for s in pred:
        if s != self.s_goal:
            self.rhs[s] = min(self.rhs[s], self.cost(s, u) + self.g[u])
            self.update_vertex(s)
    else:
        self.g_old = self.g[u]
        self.g[u] = float('inf')
        pred = self.sensed_map.succ(vertex=u)
        pred.append(u)
        for s in pred:
            if self.rhs[s] == self.cost(s, u) + self.g_old:
                if s != self.s_goal:
                    min_s = float('inf')
                    succ = self.sensed_map.succ(vertex=s)
                    for s_ in succ:
                        temp = self.cost(s, s_) + self.g[s_]
                        if min_s > temp:
                            min_s = temp
                    self.rhs[s] = min_s
            self.update_vertex(u)

# función que mueve el robot y reajusta la planificación si es necesario
def move_and_replan(self, robot_position: (int, int)):
    path = [robot_position]
    self.s_start = robot_position
    self.s_last = self.s_start
    self.compute_shortest_path()

    while self.s_start != self.s_goal:
        assert (self.rhs[self.s_start] != float('inf')), "There is no known path!"

        succ = self.sensed_map.succ(self.s_start, avoid_obstacles=False)

```



```

min_s = float('inf')
arg_min = None
for s_ in succ:
    temp = self.cost(self.s_start, s_) + self.g[s_]
    if temp < min_s:
        min_s = temp
        arg_min = s_

self.s_start = arg_min
path.append(self.s_start)

# si algún costo de arista cambió
if self.new_edges_and_old_costs:
    self.k_m += heuristic(self.s_last, self.s_start)
    self.s_last = self.s_start

# para todas las aristas dirigidas (u,v) con costos cambiado
vertices = self.new_edges_and_old_costs.vertices
for vertex in vertices:
    v = vertex.pos
    succ_v = vertex.edges_and_c_old
    for u, c_old in succ_v.items():
        c_new = self.cost(u, v)
        if c_old > c_new:
            if u != self.s_goal:
                self.rhs[u] = min(self.rhs[u], self.cost(u, v) + self.g[v])
            elif self.rhs[u] == c_old + self.g[v]:
                if u != self.s_goal:
                    min_s = float('inf')
                    succ_u = self.sensed_map.succ(vertex=u)
                    for s_ in succ_u:
                        temp = self.cost(u, s_) + self.g[s_]
                        if min_s > temp:
                            min_s = temp
                    self.rhs[u] = min_s
                self.update_vertex(u)
    self.compute_shortest_path()
return path, self.g, self.rhs

```

3. grid_utils.py:

```
import math
from typing import List

# clase que representa un vértice en el grafo
class Vertex:
    def __init__(self, pos: (int, int)):
        self.pos = pos # coordenadas del vértice
        self.edges_and_costs = {} # diccionario de aristas y sus costos

    # método para agregar una arista con un costo asociado
    def add_edge_with_cost(self, succ: (int, int), cost: float):
        if succ != self.pos:
            self.edges_and_costs[succ] = cost

    # propiedad para acceder a las aristas y sus costos
    @property
    def edges_and_c_old(self):
        return self.edges_and_costs

# clase que almacena una colección de vértices
class Vertices:
    def __init__(self):
        self.list = [] # lista de vértices

    # método para agregar un vértice a la colección
    def add_vertex(self, v: Vertex):
        self.list.append(v)

    # propiedad para acceder a la lista de vértices
    @property
    def vertices(self):
        return self.list

# función para calcular la distancia euclidiana entre dos puntos
def heuristic(p: (int, int), q: (int, int)) -> float:
    """
```

función auxiliar para calcular la distancia entre dos puntos.

:param p: (x,y)

:param q: (x,y)

:return: distancia euclidiana

"""

return math.sqrt((p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)

función para obtener los movimientos con conectividad 4

def get_movements_4n(x: int, y: int) -> List:

"""

obtener todos los movimientos posibles con conectividad 4.

:return: lista de movimientos con costo [(dx, dy)]

"""

return [(x + 1, y + 0),

 (x + 0, y + 1),

 (x - 1, y + 0),

 (x + 0, y - 1)]

función para obtener los movimientos con conectividad 8

def get_movements_8n(x: int, y: int) -> List:

"""

obtener todos los movimientos posibles con conectividad 8.

:return: lista de movimientos con costo [(dx, dy)]

"""

return [(x + 1, y + 0),

 (x + 0, y + 1),

 (x - 1, y + 0),

 (x + 0, y - 1),

 (x + 1, y + 1),

 (x - 1, y + 1),

 (x - 1, y - 1),

 (x + 1, y - 1)]

4. grid.py:

import numpy as np

from grid_utils import get_movements_4n, get_movements_8n, heuristic, Vertices, Vertex

from typing import Dict, List

OBSTACLE = 1

UNOCCUPIED = 0

clase que representa un mapa de ocupación en cuadrícula

class OccupancyGridMap:

def __init__(self, map_data, x_dim, y_dim, exploration_setting='8N'):

"""

inicializa el mapa de ocupación con los datos o dimensiones proporcionadas.

"""

if map_data is not None:

self.occupancy_grid_map = map_data

self.x_dim, self.y_dim = x_dim, y_dim

else:

inicializa un mapa vacío si no se proporcionan datos

self.x_dim = x_dim

self.y_dim = y_dim

self.occupancy_grid_map = np.zeros((self.x_dim, self.y_dim), dtype=np.uint8)

self.exploration_setting = exploration_setting

def __getitem__(self, key):

return self.occupancy_grid_map[key]

def update_with_obstacles(self, obstacle_positions):

"""actualiza las celdas del mapa para marcar nuevos obstáculos."""

for pos in obstacle_positions:

if self.in_bounds(pos):

self.occupancy_grid_map[pos] = 1 # marcar como ocupado

def get_map(self):

"""

:return: devuelve el mapa de ocupación actual

"""

return self.occupancy_grid_map

def set_map(self, new_grid):

"""

:param new_grid: nuevo mapa de ocupación

```

"""

self.occupancy_grid_map = new_grid

def is_unoccupied(self, pos: (int, int)) -> bool:
    """
    verifica si una celda está ocupada
    :param pos: posición de la celda a verificar
    :return: true si la celda está libre, false si está ocupada
    """
    (x, y) = (int(pos[0]), int(pos[1])) # asegurarse de que pos sea int
    (row, col) = (x, y)
    return self.occupancy_grid_map[row][col] == UNOCCUPIED

def in_bounds(self, cell: (int, int)) -> bool:
    """
    verifica si las coordenadas proporcionadas están dentro de los límites del mapa
    :param cell: posición de la celda (x,y)
    :return: true si está dentro de los límites, false en caso contrario
    """
    (x, y) = cell
    return 0 <= x < self.x_dim and 0 <= y < self.y_dim

def filter(self, neighbors: List, avoid_obstacles: bool):
    """
    filtra los vecinos de una celda en función de si se deben evitar los obstáculos
    :param neighbors: lista de vecinos potenciales
    :param avoid_obstacles: si es true, se excluyen las celdas con obstáculos
    :return: lista de vecinos filtrados
    """
    if avoid_obstacles:
        return [node for node in neighbors if self.in_bounds(node) and self.is_unoccupied(node)]
    return [node for node in neighbors if self.in_bounds(node)]

def succ(self, vertex: (int, int), avoid_obstacles: bool = False) -> list:
    """
    obtiene los sucesores de un vértice
    :param vertex: vértice del que se quieren encontrar los sucesores
    :param avoid_obstacles: si es true, evita las celdas con obstáculos

```

```

: return: lista de sucesores
"""

(x, y) = vertex

if self.exploration_setting == '8N':
    movements = get_movements_8n(x=x, y=y)
else:
    movements = get_movements_4n(x=x, y=y)

# not needed. Just makes aesthetics to the path
if (x + y) % 2 == 0: movements.reverse()

filtered_movements = self.filter(neighbors=movements, avoid_obstacles=avoid_obstacles)
return list(filtered_movements)

def set_obstacle(self, pos: (int, int)):
    """
    establece una celda como obstáculo
    :param pos: posición de la celda
    """
    (x, y) = (int(pos[0]), int(pos[1])) # make sure pos is int
    (row, col) = (x, y)
    self.occupancy_grid_map[row][col] = OBSTACLE

def remove_obstacle(self, pos: (int, int)):
    """
    elimina un obstáculo de la celda especificada
    :param pos: posición de la celda
    """
    (x, y) = (int(pos[0]), int(pos[1])) # make sure pos is int
    (row, col) = (x, y)
    self.occupancy_grid_map[row][col] = UNOCCUPIED

def local_observation(self, global_position: (int, int), view_range: int = 20) -> Dict:#2
    """
    observa un área local en torno a la posición del robot
    :param global_position: posición del robot en el marco del mapa global
    :param view_range: rango de visión

```

:return: diccionario con las nuevas observaciones

"""

(px, py) = global_position

nodes = [(x, y) for x in range(px - view_range, px + view_range + 1)

for y in range(py - view_range, py + view_range + 1)

if self.in_bounds((x, y))]

return {node: UNOCCUPIED if self.is_unoccupied(pos=node) else OBSTACLE for node in nodes}

class SLAM:

def __init__(self, map: OccupancyGridMap, view_range: int):

self.ground_truth_map = map

self.slam_map = OccupancyGridMap(map_data=map,

x_dim=map.x_dim,

y_dim=map.y_dim)

self.view_range = view_range

def set_ground_truth_map(self, gt_map: OccupancyGridMap):

self.ground_truth_map = gt_map

def cost(self, u: (int, int), v: (int, int)) -> float:

"""

calcula el costo entre nodos

:param u: vértice de origen

:param v: vértice de destino

:return: distancia euclidiana o infinito si hay un obstáculo en el camino

"""

if not self.slam_map.is_unoccupied(u) or not self.slam_map.is_unoccupied(v):

return float('inf')

else:

return heuristic(u, v)

def rescan(self, global_position: (int, int)):

"""

vuelve a escanear el área local en torno al robot

"""

local_observation = self.ground_truth_map.local_observation(global_position=global_position,

view_range=self.view_range)

```
vertices = self.update_changed_edge_costs(local_grid=local_observation)
return vertices, self.slam_map
```

```
def update_changed_edge_costs(self, local_grid: Dict) -> Vertices:
```

```
"""
```

```
actualiza los costos de las aristas en función de los cambios detectados en el mapa
```

```
"""
```

```
vertices = Vertices()
```

```
for node, value in local_grid.items():
```

```
    if value == OBSTACLE:
```

```
        if self.slam_map.is_unoccupied(node):
```

```
            v = Vertex(pos=node)
```

```
            succ = self.slam_map.succ(node)
```

```
            for u in succ:
```

```
                v.add_edge_with_cost(succ=u, cost=self.cost(u, v.pos))
```

```
            vertices.add_vertex(v)
```

```
            self.slam_map.set_obstacle(node)
```

```
        else:
```

```
            if not self.slam_map.is_unoccupied(node):
```

```
                v = Vertex(pos=node)
```

```
                succ = self.slam_map.succ(node)
```

```
                for u in succ:
```

```
                    v.add_edge_with_cost(succ=u, cost=self.cost(u, v.pos))
```

```
                vertices.add_vertex(v)
```

```
                self.slam_map.remove_obstacle(node)
```

```
return vertices
```

5. map_utils.py:

```
import numpy as np
```

```
import math
```

```
def visualize_map(map_data):
```

```
"""
```

```
visualiza el mapa con una imagen.
```

```
"""
```

```
plt.figure(0)
```

```
plt.imshow(map_data)
```

```
plt.show()
```



```

def display_map(display, map):
    """
    muestra el mapa en una pantalla, marcando los obstáculos.
    """
    display.setColor(0xFFFFFF)
    for x in range(map.shape[0]):
        for y in range(map.shape[1]):
            if ((map[x][y]>0.9):
                display.drawPixel(x,y)

def obstacle_distance(lidar):
    """
    calcula la distancia al obstáculo más cercano usando datos de lidar.
    """
    ranges = np.array(lidar.getRangeImage())
    ranges[ranges == np.inf] = 100000
    ranges[ranges > 12.0] = 100000
    ranges = ranges[280:-280]
    # return closest_obstacle_distance
    closest_obstacle_distance = np.min(ranges)
    return closest_obstacle_distance

def world2map(xworld,yworld):
    """
    convierte coordenadas del mundo real a coordenadas de mapa.
    """
    xworld = max(min(xworld, 150), -150)
    yworld = max(min(yworld, 150), -150)

    pixelx = math.floor(xworld + 150)
    pixely = math.floor(150 - yworld)

    pixelx = max(min(pixelx, 299), 0)
    pixely = max(min(pixely, 299), 0)
    return [pixelx,pixely]

def map2world(pixelx,pixely):

```

```

"""
convierte coordenadas de mapa a coordenadas del mundo real.
"""

pixelx = max(min(pixelx, 299), 0)
pixely = max(min(pixely, 299), 0)

xworld = pixelx - 150
yworld = 150 - pixely

xworld = max(min(xworld, 150), -150)
yworld = max(min(yworld, 150), -150)

return [xworld,yworld]

def generate_wp():
    """
    genera una lista de waypoints para el movimiento del robot.
    """
    WP = []

    # rotación a la izquierda hacia la pared oeste
    y = -18
    for x in range(-134, -143, -2):
        WP.append((x,y))

    # rotación a la izquierda hacia la pared sur
    x = -142
    y = -20
    WP.append((x,y))

    for y in range(-22, -143, -4):
        WP.append((x,y))

    # rotación a la izquierda hacia la pared este
    x = -140
    y = -142
    WP.append((x,y))

```

```

for x in range(-138, 143, 4):
    WP.append((x,y))

# rotación a la izquierda hacia la pared norte
x = 142
y = -140
WP.append((x,y))

for y in range(-138, 143, 4):
    WP.append((x,y))

# rotación a la izquierda hacia la pared este
x = 140
y = 142
WP.append((x,y))

for x in range(138, -67, -4):
    WP.append((x,y))

# rotación a la izquierda hacia la esquina derecha del hangar
x = -66
y = 140
WP.append((x,y))

for y in range(138, -17, -2):
    WP.append((x,y))

# rotación a la derecha hacia el lado derecho de la puerta del hangar
x = -68
y = -16
WP.append((x,y))

for x in range(-70, -89, -3):
    WP.append((x,y))

# rotación a la derecha hacia el interior del hangar
x = -88
y = -14

```

```

WP.append((x,y))

for y in range(-14, -9, 3):
    WP.append((x,y))

# rotación a la derecha hacia el lado este del hangar
x = -86
y = -10
WP.append((x,y))

for x in range(-84, -73, 2):
    WP.append((x,y))

# rotación a la izquierda hacia el lado norte del hangar
x = -74
y = -8
WP.append((x,y))

for y in range(-6, 143, 2):
    WP.append((x,y))

# rotación a la izquierda hacia el lado oeste del hangar
x = -76
y = 142
WP.append((x,y))

for x in range(-78, -141, -2):
    WP.append((x,y))

# rotación a la izquierda hacia el lado sur del hangar
x = -140
y = 140
WP.append((x,y))

for y in range(140, -11, -3):
    WP.append((x,y))

# rotación a la izquierda hacia el lado izquierdo de la puerta del hangar

```

```

x = -138
y = -10
WP.append((x,y))

for x in range(-136, -125, 2):
    WP.append((x,y))

# rotación a la derecha hacia el exterior de la puerta del hangar
x = -126
y = -12
WP.append((x,y))

y = -16
WP.append((x,y))

return WP

```

6. mapping.py:

```

import py_trees
import math
import numpy as np
from matplotlib import pyplot as plt
from scipy import signal
from map_utils import world2map

# clase que maneja el mapeo del entorno
class Mapping(py_trees.behaviour.Behaviour):
    def __init__(self, name, blackboard):
        super(Mapping, self).__init__(name)
        self.hasrun = False
        self.robot = blackboard.read('robot')

    def setup(self):
        """ configura los dispositivos del robot """
        self.timestep = int(self.robot.getBasicTimeStep())
        self.lidar = self.robot.getDevice("Hokuyo UTM-30LX")
        self.lidar.enable(self.timestep)
        self.lidar.enablePointCloud()

```

```

self.display = self.robot.getDevice("display")
self.gps = self.robot.getDevice('gps')
self.gps.enable(self.timestep)
self.compass = self.robot.getDevice('compass')
self.compass.enable(self.timestep)

def initialise(self):
    """ inicializa los parámetros del mapeo """
    print(self.name)
    self.map = np.zeros((300,300))
    self.kernel = np.ones((30,30))
    self.angles = np.linspace(124*math.pi/180,-124*math.pi/180,1080)
    self.angles = self.angles[280:len(self.angles)-280]

def update(self):
    """ actualiza la información del mapeo """
    self.hasrun = True

    # obtener la pose del robot
    xw = self.gps.getValues()[0]
    yw = self.gps.getValues()[1]
    theta = np.arctan2(self.compass.getValues()[0],self.compass.getValues()[1])

    # convertir a coordenadas del mapa
    px, py = world2map(xw,yw)

    # obtener mediciones del lidar eliminando los primeros y últimos 80 datos
    ranges = np.array(self.lidar.getRangeImage())
    ranges[ranges == np.inf] = 100000
    ranges[ranges > 12.0] = 100000
    ranges = ranges[280:-280]

    # convertir datos del lidar a coordenadas del mundo
    w_T_r = np.array([[np.cos(theta),-np.sin(theta),xw],
                      [np.sin(theta),np.cos(theta),yw],
                      [0,0,1]])

    X_i = np.array([ranges*np.cos(self.angles)+0.202, ranges*np.sin(self.angles), np.ones(len(ranges))])

```

```
D = w_T_r @ X_i
```

```
# convertir de coordenadas del mundo a coordenadas del mapa
```

```
for d in D.transpose():
```

```
    px, py = world2map(d[0], d[1])
```

```
    if (map[px, py] < 1):
```

```
        map[px, py] += 0.0001
```

```
    v = int(map[px, py]*255)
```

```
    color = (v*256**2 + v*256 + v)
```

```
    self.display.setColor(int(color))
```

```
    self.display.drawPixel(px, py)
```

```
return py_trees.common.Status.RUNNING
```

```
# genera el mapa de obstáculos para planificación y navegación
```

```
def terminate(self, new_status):
```

```
    if(self.hasrun):
```

```
        cspace = signal.convolve2d(self.map,self.kernel,mode='same')
```

```
        plt.figure(0)
```

```
        plt.imshow(cspace)
```

```
        plt.figure(1)
```

```
        plt.imshow(cspace > 0.9)
```

```
        plt.show()
```

```
        np.save("cspace0", cspace)
```

```
        np.save("cspace", cspace > 0.9)
```

```
new_status = py_trees.common.Status.SUCCESS
```

```
return new_status
```

7. motors_utils.py:

```
def configure_speeds(name):
```

```
    """
```

```
    configura las velocidades máximas según la tarea
```

```
    """
```

```
if name == "Mover alrededor del aeródromo":
```

```

    MAX_SPEED = 10 # velocidad de simulación para mapeo
else:
    MAX_SPEED = 5.446 # velocidad real de las ruedas del robot
# p1 = MAX_SPEED * 0.5
# p2 = MAX_SPEED * 0.5
p1 = MAX_SPEED * 1
p2 = MAX_SPEED * 1
return p1, p2, MAX_SPEED

def set_motor_positions(leftBackMotor, leftFrontMotor, rightBackMotor, rightFrontMotor, position):
    """
    establece la posición de los motores
    """
    motors = [leftBackMotor, leftFrontMotor, rightBackMotor, rightFrontMotor]
    for motor in motors:
        motor.setPosition(position)

def set_motor_velocities(leftBackMotor, leftFrontMotor, rightBackMotor, rightFrontMotor, left_velocity,
right_velocity=None):
    """
    establece la velocidad de los motores
    """
    if right_velocity is None:
        right_velocity = left_velocity
    leftBackMotor.setVelocity(left_velocity)
    leftFrontMotor.setVelocity(left_velocity)
    rightBackMotor.setVelocity(right_velocity)
    rightFrontMotor.setVelocity(right_velocity)

```

8. navigation.py:

```

import py_trees
import math
import numpy as np
from map_utils import world2map
from motor_utils import configure_speeds, set_motor_positions, set_motor_velocities

# clase que maneja la navegación del robot
class Navigation(py_trees.behaviour.Behaviour):

```



```

def __init__(self, name, blackboard):
    super(Navigation, self).__init__(name)
    self.blackboard = blackboard
    self.robot = blackboard.read('robot')

def setup(self):
    """ configura los dispositivos del robot """
    self.timestep = int(self.robot.getBasicTimeStep())
    self.lidar = self.robot.getDevice("Hokuyo UTM-30LX")
    self.lidar.enable(self.timestep)
    self.lidar.enablePointCloud()
    self.display = self.robot.getDevice("display")
    self.gps = self.robot.getDevice('gps')
    self.gps.enable(self.timestep)
    self.compass = self.robot.getDevice('compass')
    self.compass.enable(self.timestep)
    self.leftBackMotor = self.robot.getDevice('wheel_left_back_joint')
    self.leftFrontMotor = self.robot.getDevice('wheel_left_front_joint')
    self.rightBackMotor = self.robot.getDevice('wheel_right_back_joint')
    self.rightFrontMotor = self.robot.getDevice('wheel_right_front_joint')
    set_motor_positions(self.leftBackMotor, self.leftFrontMotor, self.rightBackMotor, self.rightFrontMotor,
float("inf"))
    set_motor_velocities(self.leftBackMotor, self.leftFrontMotor, self.rightBackMotor, self.rightFrontMotor, 0.0)

def initialise(self):
    """ inicializa los parámetros de navegación """
    print(self.name)
    self.p1, self.p2, self.MAX_SPEED = configure_speeds(self.name)
    self.map = np.zeros((300,300))
    self.angles = np.linspace(124*math.pi/180,-124*math.pi/180,1080)
    self.angles = self.angles[280:len(self.angles)-280]
    self.index = 0
    self.marker = self.robot.getFromDef("marker").getField("translation")
    self.finish = False
    self.WP = self.blackboard.read('waypoints')

def update(self):
    """ actualiza la posición del robot y ajusta su movimiento """

```

```

if not self.finish:
    xw, yw = self.gps.getValues()[0], self.gps.getValues()[1]
    theta = np.arctan2(self.compass.getValues()[0], self.compass.getValues()[1])

    if self.name != "Mover alrededor del aeródromo":
        px,py = world2map(xw, yw)
        self.display.setColor(0xFF0000)
        self.display.drawPixel(px,py)

    self.marker.setSFVec3f([*self.WP[self.index], 0])

    rho = np.sqrt((xw - self.WP[self.index][0]) ** 2 + (yw - self.WP[self.index][1]) ** 2)
    alpha = np.arctan2(self.WP[self.index][1] - yw, self.WP[self.index][0] - xw) - theta
    alpha = (alpha + np.pi) % (2 * np.pi) - np.pi

    if rho < 0.3:
        self.index += 1
        if self.index == len(self.WP) - 1:
            self.finish = True

    phildot = -alpha * self.p1 + rho * self.p2
    phirdot = alpha * self.p1 + rho * self.p2

    phildot = max(min(phildot, self.MAX_SPEED), -self.MAX_SPEED)
    phirdot = max(min(phirdot, self.MAX_SPEED), -self.MAX_SPEED)

    set_motor_velocities(self.leftBackMotor, self.leftFrontMotor, self.rightBackMotor, self.rightFrontMotor,
        phildot, phirdot)
    return py_trees.common.Status.RUNNING
else:
    set_motor_velocities(self.leftBackMotor, self.leftFrontMotor, self.rightBackMotor, self.rightFrontMotor,
        0.0)
    return py_trees.common.Status.SUCCESS

```

9. planning.py:

```

import py_trees
import math
import numpy as np

```

```

from grid import OccupancyGridMap, SLAM
from d_star_lite import DStarLite
from map_utils import world2map, map2world, display_map, obstacle_distance
from motor_utils import configure_speeds, set_motor_positions, set_motor_velocities
import time
import csv

# clase que maneja la planificación de la ruta del robot
class Planning(py_trees.behaviour.Behaviour):
    def __init__(self, name, blackboard, goal):
        super(Planning, self).__init__(name)
        self.blackboard = blackboard
        self.robot = blackboard.read('robot')
        self.goal = goal

    def setup(self):
        """ configura los dispositivos del robot """
        self.timestep = int(self.robot.getBasicTimeStep())
        self.lidar = self.robot.getDevice("Hokuyo UTM-30LX")
        self.lidar.enable(self.timestep)
        self.lidar.enablePointCloud()
        self.display = self.robot.getDevice("display")
        self.gps = self.robot.getDevice('gps')
        self.gps.enable(self.timestep)
        self.compass = self.robot.getDevice('compass')
        self.compass.enable(self.timestep)
        self.leftBackMotor = self.robot.getDevice('wheel_left_back_joint')
        self.leftFrontMotor = self.robot.getDevice('wheel_left_front_joint')
        self.rightBackMotor = self.robot.getDevice('wheel_right_back_joint')
        self.rightFrontMotor = self.robot.getDevice('wheel_right_front_joint')
        set_motor_positions(self.leftBackMotor, self.leftFrontMotor, self.rightBackMotor, self.rightFrontMotor,
float('inf'))
        set_motor_velocities(self.leftBackMotor, self.leftFrontMotor, self.rightBackMotor, self.rightFrontMotor, 0.0)
        self.position_data = []
        self.event = False

    def initialise(self):
        """ inicializa los parámetros de planificación """

```

```

print(self.name)
self.p1, self.p2, self.MAX_SPEED = configure_speeds(self.name)
self.angles = np.linspace(124*math.pi/180, -124*math.pi/180, 1080)
self.angles = self.angles[280:len(self.angles)-280]
self.index = 0
self.marker = self.robot.getFromDef("marker").getField("translation")
self.finish = False
self.WP_distance = 1

self.map = self.blackboard.read('map')
display_map(self.display, self.map)
xw, yw = self.gps.getValues()[0], self.gps.getValues()[1]
px, py = world2map(xw, yw)
self.start = (px, py)
wx, wy = self.goal
gx, gy = world2map(wx, wy)
self.goal = (gx, gy)
self.grid_map = OccupancyGridMap(map_data=self.map, x_dim=self.map.shape[0],
y_dim=self.map.shape[1])
self.d_star_lite = DStarLite(map=self.grid_map, s_start=self.start, s_goal=self.goal)
self.slam = SLAM(map=self.grid_map, view_range=20)
initial_path, _, _ = self.d_star_lite.move_and_replan(robot_position=self.start)
# Convert the initial path to world coordinates and use it as the initial waypoints
self.WP = [map2world(x, y) for x, y in initial_path]
self.start_time = time.time()

def update(self):
    """ actualiza la planificación de la ruta """
    if obstacle_distance(self.lidar) < 12.0 or (len(self.WP) - self.index) < 10:
        self.WP_distance = 0.1
        if self.new_obstacle():
            self.recalculate_path()
    else:
        self.WP_distance = 1
    self.follow_waypoints()
    return py_trees.common.Status.RUNNING

def recalculate_path(self):

```

```

""" recalcula la ruta cuando se detecta un nuevo obstáculo """
self.event = True

self.slam.set_ground_truth_map(OccupancyGridMap(map_data=self.map, x_dim=self.map.shape[0],
y_dim=self.map.shape[1]))
xw, yw = self.gps.getValues()[0], self.gps.getValues()[1]
px,py = world2map(xw,yw)
self.start = (px,py)

wx, wy = self.WP[-1]
gx, gy = world2map(wx,wy)
self.goal = (gx,gy)

# SLAM
self.d_star_lite.new_edges_and_old_costs, self.d_star_lite.sensed_map =
self.slam.rescan(global_position=self.start)

# actualizar D* Lite con la nueva posición de inicio y recalcular la ruta
path, _, _ = self.d_star_lite.move_and_replan(robot_position=self.start)

# convertir la ruta de regreso a coordenadas del mundo y asignarla como waypoints
self.WP = [map2world(x, y) for x, y in path]
self.index = 1

def follow_waypoints(self):
    """ sigue los waypoints para llegar al destino """
    if not self.finish:
        xw, yw = self.gps.getValues()[0], self.gps.getValues()[1]
        theta = np.arctan2(self.compass.getValues()[0], self.compass.getValues()[1])

        px,py = world2map(xw, yw)
        self.display.setColor(0xFF0000)
        self.display.drawPixel(px,py)

        self.marker.setSFVec3f([*self.WP[self.index], 0])

        rho = np.sqrt((xw - self.WP[self.index][0]) ** 2 + (yw - self.WP[self.index][1]) ** 2)
        alpha = np.arctan2(self.WP[self.index][1] - yw, self.WP[self.index][0] - xw) - theta
        alpha = (alpha + np.pi) % (2 * np.pi) - np.pi

```

```

self.log_position(xw, yw, rho, alpha)

if rho < self.WP_distance:
    self.index += 1
    if self.index == len(self.WP):
        self.finish = True

phildot = -alpha * self.p1 + rho * self.p2
phirdot = alpha * self.p1 + rho * self.p2

phildot = max(min(phildot, self.MAX_SPEED), -self.MAX_SPEED)
phirdot = max(min(phirdot, self.MAX_SPEED), -self.MAX_SPEED)

set_motor_velocities(self.leftBackMotor, self.leftFrontMotor, self.rightBackMotor, self.rightFrontMotor,
phildot, phirdot)

self.event = False
return py_trees.common.Status.RUNNING
else:
    print("Llegué!")
    set_motor_velocities(self.leftBackMotor, self.leftFrontMotor, self.rightBackMotor, self.rightFrontMotor,
0.0)

self.event = False
self.save_position_data_to_csv()
return py_trees.common.Status.SUCCESS

def new_obstacle(self):
    """ detecta la presencia de nuevos obstáculos en el entorno """
    obstacles = 0
    new_obstacle = False
    self.updated_map = np.copy(self.map)
    xw, yw = self.gps.getValues()[0], self.gps.getValues()[1]
    theta = np.arctan2(self.compass.getValues()[0], self.compass.getValues()[1])

    # obtener mediciones del lidar eliminando los primeros y últimos 80 datos
    ranges = np.array(self.lidar.getRangeImage())
    ranges[ranges == np.inf] = 100000
    ranges[ranges > 12.0] = 100000

```

```

ranges = ranges[280:-280]

# convertir datos del lidar a coordenadas del mundo
w_T_r = np.array([[np.cos(theta),-np.sin(theta),xw],
                  [np.sin(theta),np.cos(theta),yw],
                  [0,0,1]])

X_i = np.array([ranges*np.cos(self.angles), ranges*np.sin(self.angles), np.ones(len(ranges))])#+0.202
D = w_T_r @ X_i

SAFETY_MARGIN_OBSTACLE = 6
margin_cells = int(SAFETY_MARGIN_OBSTACLE)

# convertir de coordenadas del mundo a coordenadas del mapa para visualizar obstáculos
self.display.setColor(0xFFFFF)
for d in D.transpose():
    dx, dy = world2map(d[0], d[1])
    min_x = max(0, dx - margin_cells)
    max_x = min(self.updated_map.shape[0], dx + margin_cells+1)
    min_y = max(0, dy - margin_cells)
    max_y = min(self.updated_map.shape[1], dy + margin_cells+1)
    for x in range(min_x, max_x):
        for y in range(min_y, max_y):
            if (self.updated_map[x,y] < 1):
                obstacles += 1
                new_obstacle = True
                self.updated_map[x,y] = 1
                self.display.drawPixel(x,y)

self.map = self.updated_map

return new_obstacle

def log_position(self, x, y, rho, alpha):
    """ registra la posición del robot junto con información relevante """
    x_formatted, y_formatted = f"{x:.2f}", f"{y:.2f}"
    wp_x, wp_y = self.WP[self.index][0], self.WP[self.index][1]
    rho_formatted = f"{rho:.2f}"

```

```

alpha_formatted = f'{alpha:.2f}'
timestamp = time.time() - self.start_time
timestamp_formatted = f'{timestamp:.2f}'

self.position_data.append([x_formatted, y_formatted, wp_x, wp_y, rho_formatted, alpha_formatted,
timestamp_formatted, self.event])

def save_position_data_to_csv(self):
    """ guarda los datos de posición en un archivo csv """
    with open("robot_position_data.csv", mode='w', newline=") as file:
        writer = csv.writer(file, delimiter=',')
        writer.writerow(["R_X", "R_Y", "WP_X", "WP_Y", "Distancia", "Orientacion", "Tiempo", "Recalculacion"])
        writer.writerows(self.position_data)

```

10. priority_queue.py:

```

class Priority:
    """
    maneja el orden lexicográfico de las claves
    """
    def __init__(self, k1, k2):
        """
        :param k1: valor de la clave
        :param k2: valor de la clave
        """
        self.k1 = k1
        self.k2 = k2

    def __lt__(self, other):
        """
        orden lexicográfico 'menor que'
        :param other: claves comparables
        :return: orden lexicográfico
        """
        return self.k1 < other.k1 or (self.k1 == other.k1 and self.k2 < other.k2)

    def __le__(self, other):
        """
        orden lexicográfico 'menor o igual que'
        :param other: claves comparables

```



```

        :return: orden lexicográfico
        """

        return self.k1 < other.k1 or (self.k1 == other.k1 and self.k2 <= other.k2)

class PriorityNode:
    """
    maneja el orden lexicográfico de los vértices
    """

    def __init__(self, priority, vertex):
        """
        :param priority: prioridad de un vértice
        :param vertex:
        """
        self.priority = priority
        self.vertex = vertex

    def __le__(self, other):
        """
        :param other: nodo comparable
        :return: orden lexicográfico
        """
        return self.priority <= other.priority

    def __lt__(self, other):
        """
        :param other: nodo comparable
        :return: orden lexicográfico
        """
        return self.priority < other.priority

class PriorityQueue:
    """
    estructura de datos de cola de prioridad basada en un montículo
    """

    def __init__(self):

```

```

"""
inicializa una cola de prioridad vacía
"""

self.heap = []
self.vertices_in_heap = []

def top(self):
    """
    devuelve el vértice con la mayor prioridad en la cola
    """
    return self.heap[0].vertex

def top_key(self):
    """
    devuelve la clave de mayor prioridad en la cola
    """
    if len(self.heap) == 0: return Priority(float('inf'), float('inf'))
    return self.heap[0].priority

def pop(self):
    """Fuente: Lib/heapq.py"""
    """
    extrae el elemento de mayor prioridad de la cola y lo elimina
    """
    lastelt = self.heap.pop() # raises appropriate IndexError if heap is empty
    self.vertices_in_heap.remove(lastelt)
    if self.heap:
        returnitem = self.heap[0]
        self.heap[0] = lastelt
        self._siftup(0)
    else:
        returnitem = lastelt
    return returnitem

def insert(self, vertex, priority):
    """Fuente: Lib/heapq.py"""
    """
    inserta un vértice con su prioridad en la cola
    """

```

```

"""
item = PriorityNode(priority, vertex)
self.vertices_in_heap.append(vertex)
self.heap.append(item)
self._siftdown(0, len(self.heap) - 1)

def remove(self, vertex):
    """
    elimina un vértice de la cola de prioridad
    """
    self.vertices_in_heap.remove(vertex)
    for index, priority_node in enumerate(self.heap):
        if priority_node.vertex == vertex:
            self.heap[index] = self.heap[len(self.heap) - 1]
            self.heap.remove(self.heap[len(self.heap) - 1])
            break
    self.build_heap()

def update(self, vertex, priority):
    """
    actualiza la prioridad de un vértice en la cola
    """
    for index, priority_node in enumerate(self.heap):
        if priority_node.vertex == vertex:
            self.heap[index].priority = priority
            break
    self.build_heap()

def build_heap(self):
    """
    reorganiza la cola para mantener la propiedad de montículo
    """
    n = len(self.heap)
    for i in reversed(range(n // 2)):
        self._siftup(i)

def _siftdown(self, startpos, pos):
    """

```

función auxiliar para reorganizar la estructura de montículo descendiendo

"""

```
newitem = self.heap[pos]
while pos > startpos:
    parentpos = (pos - 1) >> 1
    parent = self.heap[parentpos]
    if newitem < parent:
        self.heap[pos] = parent
        pos = parentpos
        continue
    break
self.heap[pos] = newitem
```

```
def _siftup(self, pos):
```

"""

función auxiliar para reorganizar la estructura de montículo ascendiendo

"""

```
endpos = len(self.heap)
startpos = pos
newitem = self.heap[pos]
childpos = 2 * pos + 1
while childpos < endpos:
    rightpos = childpos + 1
    if rightpos < endpos and not self.heap[childpos] < self.heap[rightpos]:
        childpos = rightpos
    self.heap[pos] = self.heap[childpos]
    pos = childpos
    childpos = 2 * pos + 1
self.heap[pos] = newitem
self._siftdown(startpos, pos)
```

ANEXO D. SCRIPTS DE CÓDIGO DE LOS ENSAYOS DE LAS SECCIONES 3.6.4.1 Y 3.6.4.2

En esta sección solo se incluirá un script, BT_controller.py, porque fue el único archivo necesario para llevar a cabo el ensayo de control de motores sin carga y con carga.

11. BT_controller.py:

```
from controller import Robot
import math
import time
import csv

# Tiempo de paso básico del simulador
TIME_STEP = 32

# Configuración inicial del robot
robot = Robot()

# Inicialización de motores
left_front_motor = robot.getDevice('wheel_left_front_joint')
left_back_motor = robot.getDevice('wheel_left_back_joint')
right_front_motor = robot.getDevice('wheel_right_front_joint')
right_back_motor = robot.getDevice('wheel_right_back_joint')

gps = robot.getDevice('gps')
gps.enable(TIME_STEP)

compass = robot.getDevice('compass')
compass.enable(TIME_STEP)

# Configurar los motores en modo de velocidad infinita
for motor in [left_front_motor, left_back_motor, right_front_motor, right_back_motor]:
    motor.setPosition(float('inf')) # Sin restricciones de posición
    motor.setVelocity(0.0)         # Velocidad inicial en 0

position_data = []
```

```

start_time = time.time()

def log_position():
    x, y, _ = gps.getValues()
    theta = math.atan2(compass.getValues()[0], compass.getValues()[1])
    timestamp = time.time() - start_time
    position_data.append([f"{x:.2f}", f"{y:.2f}", f"{theta:.2f}", f"{timestamp:.2f}"])

def save_position_data_to_csv():
    with open("robot_position_data.csv", mode='w', newline="") as file:
        writer = csv.writer(file, delimiter=',')
        writer.writerow(["R_X", "R_Y", "Orientacion", "Tiempo"])
        writer.writerows(position_data)

# Función para ejecutar un movimiento básico
def test_motor_movements():
    print("Iniciando ensayo de control de motores...")

    movements = [
        ("Movimiento de avance", 2.0, 2.0, 4000),
        ("Deteniendo motores", 0.0, 0.0, 1000),
        ("Movimiento de retroceso", -2.0, -2.0, 4000),
        ("Deteniendo motores", 0.0, 0.0, 1000),
        ("Rotación hacia la izquierda", -1.0, 1.0, 4000),
        ("Deteniendo motores", 0.0, 0.0, 1000),
        ("Rotación hacia la derecha", 1.0, -1.0, 4000),
        ("Deteniendo motores", 0.0, 0.0, 1000),
        ("Prueba de velocidad máxima", 5.445, 5.445, 4000),
        ("Deteniendo motores", 0.0, 0.0, 1000)
    ]

    for description, left_speed, right_speed, duration in movements:
        print(description)
        set_motor_speeds(left_speed, right_speed)
        for _ in range(int(duration / TIME_STEP)):
            robot.step(TIME_STEP)
            log_position()

```

```
save_position_data_to_csv()
```

```
# Función para establecer velocidades a los motores
```

```
def set_motor_speeds(left_speed, right_speed):
```

```
    left_front_motor.setVelocity(left_speed)
```

```
    left_back_motor.setVelocity(left_speed)
```

```
    right_front_motor.setVelocity(right_speed)
```

```
    right_back_motor.setVelocity(right_speed)
```

```
# Loop principal
```

```
test_motor_movements()
```